

Rational Verification for Stackelberg Security Games

Surasak Phetmanee

Submitted in fulfilment of the requirements for the
Degree of Doctor of Philosophy

School of Computing Science
College of Science and Engineering
University of Glasgow



University
of Glasgow

October 2025

Abstract

Stackelberg Security Games (SSGs) are a game theory model used to help defenders analyse threats, plan strategies, and estimate resource allocation in cybersecurity. In an SSG, a defender (the leader) commits to a defensive strategy, which an attacker (the follower) observes before choosing their optimal response. However, a key limitation of this model is the absence of a formal approach to verify that the security properties of a defensive strategy hold under Stackelberg equilibrium conditions, particularly against adaptive adversaries. Rational verification addresses this by checking if a temporal logic formula is satisfied in some or all computed equilibria. This thesis introduces a framework that integrates rational verification into SSGs, and defenders can design rational defensive strategies.

We propose a methodology for analysing adaptive adversaries by formalising SSGs, a model that uses rational verification to incorporate equilibria and payoffs to synthesise dynamic strategies. We implement **STEEVE** (**S**tackelberg **S**ecurity **G**ames and **E**quilibrium **V**erification), a tool that integrates game theory and formal verification for rationality in cybersecurity. The key features of **STEEVE** include Attack Defence Trees (ADTs) to model threats, formalising security properties using temporal logic rPATL (Probabilistic Alternating-time Temporal Logic with Rewards), and an extension of PRISM-games to compute Stackelberg equilibria. **STEEVE** gathers data from public vulnerability databases, such as Common Vulnerabilities and Exposures (CVEs), allowing defenders to tailor strategies to specific characteristics of cyber threat.

Our results show that the adaptive strategies produced by our framework yield a reduction in defender costs and system compromise compared to static approaches. The primary contribution of this work is a methodology and supporting tool that aim to improve the robustness of defensive strategies against adaptive threats by applying rational verification.

Contents

Abstract	i
Acknowledgements	vii
Declaration	viii
List of Notations	ix
1 Introduction	1
1.1 Motivation	1
1.2 Rational Verification and Security Games	2
1.3 Thesis Statement	4
1.4 Publications	5
1.5 Structure of the Thesis	5
2 Background	6
2.1 Game Theory	6
2.1.1 Security Games	6
2.1.2 Strategies	8
2.1.3 Equilibria	9
2.1.4 Stackelberg Security Games (SSGs)	12
2.2 Threat Modelling and Specification	14
2.2.1 Evolution of Cyber Threats	14
2.2.2 Threats, Vulnerabilities, and Requirements	15
2.2.3 The Role and Ecosystem of CVEs	19
2.2.4 Adversaries and Defenders	22
2.2.5 Attack Defence Trees (ADTs)	24
2.2.6 Alternative Threat Modelling Methods	27
2.3 Rational Verification of Security Games	28
2.3.1 Rational Verification	29
2.3.2 Temporal Logics for Strategic Verification	30

2.3.3	Related Tools	33
2.4	Summary	37
3	Modelling Threats	38
3.1	Introduction	38
3.1.1	Datasets	39
3.2	Mapping CVEs to ADTs	43
3.3	Transforming ADTs to SSGs	45
3.3.1	Case Study 1: Modelling CVE-2024-3400 as an SSG	51
3.4	Repeated Stackelberg Security Games (RSSGs)	57
3.4.1	Strategy Synthesis	58
3.4.2	Case Study 2: Modelling CVE-2024-4947 as an RSSG	59
3.5	Summary	64
4	Rational Verification of Security Games	65
4.1	Verifying Stackelberg Equilibria	66
4.2	Implementation of the STEVE Tool	67
4.3	Applying Rational Verification to the Case Studies	70
4.3.1	Verifying the Static Defence for CVE-2017-8759, CVE-2024-3400	70
4.3.2	Verifying the Adaptive Defence for CVE-2024-4947	73
4.4	Summary	76
5	Evaluation and Discussion	77
5.1	Introduction	77
5.2	Experimental Objectives and Metrics	78
5.3	Case Study Results	79
5.3.1	Static Strategy Verification	80
5.3.2	A Quantitative Comparison of Static vs. Dynamic Strategies	80
5.3.3	Equilibrium Guarantees and the Impact of Temporal Preferences	81
5.4	Discussion and Limitations	81
5.5	Summary	84
6	Conclusion	85
6.1	Future Work	86
A	Proofs of Correctness for the Procedures	87
A.1	Transforming an ADT to an SSG model: Correctness	87
A.2	Cost Estimation and Preferences	98

List of Tables

2.1	Common Vulnerabilities and Exposures (CVEs) reported as exploited by various Advanced Persistent Threat (APT) groups between 2010 and 2024.	18
2.2	Comparison of Features in Rational Verification Tools	34
3.1	CVE-2017-8759, a remote code execution vulnerability in Microsoft .NET. . .	42
3.2	Illustrative mapping from ADT elements to SSG components.	45
3.3	Summary of defensive & attacking strategies. Stackelberg equilibrium in bold. .	50
3.4	CVE-2024-3400, a critical command injection vulnerability in Palo Alto Networks.	52
3.5	Notation summary for strategy types in RSSGs.	59
3.6	CVE-2024-4947, a type confusion vulnerability in Google Chrome.	61
3.7	States and Actions	62
3.8	Transition Probabilities and Payoffs	63
4.1	Security metrics expressed in rPATL for rational verification in RSSGs, showing their mapping to properties under Stackelberg equilibrium semantics.	67
5.1	Experimental Results.	79
5.2	Comparison of Static vs. Dynamic Strategy for CVE-2024-4947 ($T = 10, \beta = 0.9$)	80
5.3	Comparison with verification tools.	81

List of Figures

2.1	Game Theory.	7
2.2	Tic-Tac-Toe.	11
2.3	A Stackelberg Security Game.	13
2.4	Bypassing Security	17
2.5	Example of an ADT	26
2.6	Equilibrium Checker Concept [Abate et al., 2021]. Preferences in this context denote the payoff ordering of each agent, used by the decision procedure to determine rational outcomes.	29
2.7	PRISM-games	35
2.8	EVE	36
3.1	Overview of the rational verification framework for SSGs.	38
3.2	CVE-2017-8759	41
3.3	Graphical representation of the ADTerm $c^d(\vee_d(b'_1, b'_2, b'_3), \wedge_a(b_1, b_2, b_3))$ generated from the CVE-2017-8759 example, which is then automatically transformed into an SSG. The defender (green nodes) can choose mitigation techniques for each of the following separately: b'_1 - patch management, b'_2 - email filtering, or b'_3 - backup and recovery. However, the attacker (red nodes) can initiate an attack by combining actions involving b_1 - initial malicious document delivery, b_2 - exploitation of the OLE object, and b_3 - payload delivery.	44
3.4	Graphical representation of the SSG for the CVE-2017-8759 example, as generated from the PRISM-games extension in StEVE. In this diagram, diamonds represent defenders and rectangles represent attackers. The strategy relies on the Stackelberg equilibrium, where the defender applies only patch management (action b'_1), which will cost the defender £24,750 and the attacker earns £4,425. Some states are omitted from the diagram; these are indicated by "...".	47
3.5	CVE-2024-3400	51
3.6	Extracting a CVE into an ADT	53
3.7	Transforming an ADT to SSGs	54
3.8	Assigning Preferences and Atomic Propositions	55

3.9	Graphical representation of the SSG for the CVE-2024-3400 example, as generated from the PRISM-games extension in StEVE. In this diagram, diamonds represent defenders and rectangles represent attackers. The strategy relies on the Stackelberg equilibrium, where the defender applies only threat prevention (action b'_2), which will cost the defender £13,000 and the attacker earns £3,150. Some states are omitted from the diagram; these are indicated by "...".	56
3.10	CVE-2024-4947	60
4.1	The StEVE paper.	68
4.2	Architecture and workflow of the StEVE tool.	69
4.3	Each set of states that satisfy security property, Stackelberg equilibrium, and rPATL.	71
4.4	Rational verification with StEVE returns "true". Verification results under Stackelberg equilibrium semantics. Figure 4.4 (<code>useThreatPrevention</code>) satisfies the rPATL property, whereas Figure 4.5 (<code>upgradePANOS</code>) does not, showing how different defender commitments affect the rational verification outcome.	72
4.5	Rational verification with StEVE returns "false". Verification results under Stackelberg equilibrium semantics. Figure 4.4 (<code>useThreatPrevention</code>) satisfies the rPATL property, whereas Figure 4.5 (<code>upgradePANOS</code>) does not, showing how different defender commitments affect the rational verification outcome.	73

Acknowledgements

I am deeply grateful to my supervisors, Michele Sevegnani and Oana Andrei, for their guidance, patience, and strong support. I also wish to acknowledge Colin Perkins, Syed Waqar Nabi, Muhammad Najib, Alice Miller, and Emma Li for their valuable feedback and encouragement. I appreciate my colleagues and friends for their support, and the School of Computing Science for providing the resources and assistance essential to this work. To my family, your love made everything possible. Finally, I am grateful to the Royal Thai Government Scholarship, the Royal Thai Embassy, and Thammasat University for their generous support.

Declaration

I hereby declare that the work presented in this thesis is my own and has been carried out under the guidance of my supervisors. Except where explicitly stated otherwise, this thesis contains no material that has been submitted for any other degree or qualification at this or any other university or academic institution. All sources of information and data used in this thesis have been acknowledged. Furthermore, I certify that this thesis is my own work, except where otherwise stated. AI generated content has been clearly acknowledged where used, and appropriate ethical and methodological standards have been maintained.

List of Notations

Glossary of key notations and symbols used in this thesis, covering game theory concepts, strategies, Stackelberg equilibrium, rPATL logic, and the ADT syntax.

Symbol	Description
$\mathcal{G}$	An extensive form game tuple $(N, A, H, Z, \chi, \pi, \gamma, u)$.
N	Set of players, $\{1, 2\}$ for Defender (1) and Attacker (2).
A	Set of all possible actions in the game.
H	Set of non-terminal histories (decision nodes).
Z	Set of terminal histories (outcome nodes).
S	Set of states in the RSSG model.
s_t	State of the game at the beginning of round t .
$A_i(s_t)$	Set of actions available to player i in state s_t .
χ	Action function in extensive form game: $\chi : H \rightarrow 2^A$.
π	Player function in extensive form game: $\pi : H \rightarrow N$.
γ	Successor function in extensive form game: $\gamma : H \times A \rightarrow H \cup Z$.
u_i	Utility function for player i in the extensive form game, $u_i : Z \rightarrow \mathbb{R}$.
U_i	Expected cumulative utility for player i from a state s_t in the repeated game, given actions a_1^t, a_2^t and assuming equilibrium strategies are followed thereafter.
$R_i(s_t, a_1^t, a_2^t)$	Immediate reward (payoff) for player i in round t given state s_t and actions a_1^t, a_2^t .
T	Total number of rounds in the repeated game (finite horizon).
t	Index for the current round, $t \in \{1, \dots, T\}$.
β	Discount factor, $0 \leq \beta \leq 1$.
R_i^T	Total discounted reward for player i over T rounds.
σ_i	Strategy for player i . In RSSGs, often a sequence $\sigma_i = (\sigma_i^1, \dots, \sigma_i^T)$.
Σ_i	Set of all possible strategies for player i .
σ_i^t	Action or decision rule for player i in round t (as part of strategy σ_i).
μ_1	A static strategy for the defender in the repeated game, $\mu_1 : S \rightarrow \Delta(A_1)$.

Continued on next page

Symbol	Description
$\Delta(A_1)$	Set of probability distributions over defender actions A_1 .
a_i^t	Action chosen by player i in round t .
$BR(\sigma_1)$	Set of best responses for the attacker (player 2) given defender strategy σ_1 .
(σ_1^*, σ_2^*)	Strategy profile in Stackelberg equilibrium.
$\mathbb{E}^{\sigma_1, \sigma_2}[\cdot]$	Expectation calculated under the strategy profile (σ_1, σ_2) .
$\arg \max$	The argument (input) that maximises a function.
$\sup$	Supremum (least upper bound).
AP	Set of atomic propositions.
L	Labelling function mapping histories/states to atomic propositions, $L : H \cup Z \rightarrow 2^{AP}$.
ϕ, ϕ_1, ϕ_2	State formulae (typically Boolean combinations of atomic propositions).
ψ	Path formula (temporal logic formula evaluated over paths).
ρ	Reward path formula (specifies how rewards are accumulated/checked).
$\langle\langle C \rangle\rangle$	rPATL coalition operator, where C is the coalition of players (e.g., $C = \{1\}$ for the defender).
$R_{SE=?}^r[\rho]$	rPATL reward operator for the optimal expected value of reward structure r satisfying ρ , under round by round Stackelberg equilibrium.
$P_{\sim p}[\psi]$	rPATL probability operator for checking if the probability of path ψ satisfies bound $\sim p$, under round by round Stackelberg equilibrium.
$P_{=?}[\psi]$	rPATL probability operator for computing the optimal probability of path ψ , under round by round Stackelberg equilibrium.
$X\phi$	Temporal operator: “ ϕ holds in the Next state”.
$F\phi$	Temporal operator: “Eventually (Finally) ϕ holds”.
$G\phi$	Temporal operator: “Globally (Always) ϕ holds”.
$\phi_1 \cup \phi_2$	Temporal operator: “ ϕ_1 holds Until ϕ_2 holds”.
$\phi_1 \cup^{\leq T} \phi_2$	Temporal operator: “ ϕ_1 holds Until ϕ_2 holds within T steps”.
$F^{\leq T}\phi$	Temporal operator: “Eventually ϕ holds within T steps/rounds”.
"total_cost", ...	Names of reward structures used in concrete rPATL/tool queries (e.g., “total_cost”, “steps”, “incident_cost”).
$c^d(\dots, \dots)$	Root node constructor in the thesis’s specific ADT syntax, linking defender components (1st arg) and attacker components (2nd arg).
b'_j, b_i	Base actions for defender (d) and attacker (a) in ADTs.
B_d, B_a	Sets of base actions for defender and attacker.
$\vee^d, \wedge^d$	Disjunction/Conjunction operators for defender strategies in ADTs.
$\vee^a, \wedge^a$	Disjunction/Conjunction operators for attacker strategies in ADTs.

Chapter 1

Introduction

1.1 Motivation

The National Cyber Security Centre (NCSC) [[National Cyber Security Centre, 2025b](#)] reminds us that online security is not only about awareness but also anticipating threats. The battle for cybersecurity is no longer fought in the shadows; it plays out in calculated moves between defenders and attackers. To plan their moves, many defenders [[Ng and Hasan, 2025](#)] turn to game theory, with the Stackelberg Security Game (SSG) [[Tambe, 2012](#)] as their playbook for deploying resources against adaptive attackers. The expressivity power of SSGs lies in their capacity of modelling the real world behaviour between a defender and an attacker. The SSG model allows a defender to compute a strategy that maximises their expected utility by reasoning about an attacker's best response to a given defensive strategy. This approach has yielded significant defensive benefits [[Sinha et al., 2018](#)], including reductions in attack success rates, improved utilisation of limited security resources. While an SSG can compute an optimal strategy, this result should be interpreted as a recommendation rather than a formal guarantee. The optimality holds only with respect to the model assumptions, such as utility functions, rational players, and perfect observability. Without a verification layer, there is no proof that the computed strategy satisfies the specified security objectives, such as ensuring that system downtime remains within a defined threshold. Therefore, formal verification is required to prove that the recommended strategy indeed fulfils the desired properties.

Recent incidents [[NHS England, 2024](#), [BBC News, 2023](#)] demonstrate that adversaries do not exploit simple technical flaws, but execute calculated campaigns that target weak points for maximum disruption. These are known as rational adversaries: attackers who have a plan, compare different options, and choose the one that gives them the biggest advantage. Practical guides such as The Hacker Playbook [[Kim, 2018](#)] illustrate how attackers plan and execute penetration testing campaigns. For example, the June 2024 ransomware attack [[Alder, 2025](#), [Financial Times, 2025](#)], a critical pathology services provider for NHS Trusts, demonstrated the vulnerability of the healthcare system through a supply chain attack. The disruption was not contained within a

single entity, but had a cascading effect leading to the cancellation of thousands of appointments and operations. This shows that a purely static defence is insufficient; defenders must strategically preparing for where rational adversaries will strike to cause the most significant impact. Similarly, the 2023 breach at the payroll provider [National Cyber Security Centre, 2025a, Progress Software, 2023], which compromised the data of major employers including British Airways, Boots, and the BBC, showed how a single shared vulnerability could be exploited across multiple organisations. This represents a strategic choice by an adversary to maximise their return on investment by exploiting a vulnerability with the broadest possible impact. For defenders, this highlights a critical resource allocation problem: security investment cannot be based solely on an organisation's internal vulnerabilities but must also account for the strategic value of interconnected systems. Such cases expose the limits of traditional defences and motivate a framework to reason formally about optimal strategies against adaptive adversaries.

These incidents show us that the ongoing attacks from advanced persistent threats still need to be tackled and challenged [Sharma et al., 2023]. Such adversaries conduct campaigns over extended periods, learning from defensive responses and continuously evolving their tactics. A defensive strategy computed as optimal for a single one-shot interaction becomes obsolete the moment it is deployed, as its implementation informs the adversary's next move. This creates a dynamic interplay where the defender's very actions change the strategic landscape, requiring them to constantly anticipate the attacker's adaptation. A truly strong defence strategy cannot be static. It must be a dynamic policy that adapts based on the evolving state of the security game. Therefore, a robust security framework must be capable of reasoning and adaptation.

These challenges motivate integrating formal methods into security games to reason about strategies and guarantees. Formal methods has provided mathematical and computational guarantees for software systems [Beyer and Podelski, 2022, Kwiatkowska et al., 2022a]. Model checking is a key technique within formal methods that allows the automatic verification of system properties expressed in temporal logic [Pnueli, 1977, Baier and Katoen, 2008]. This extension to the multiagent domain, known as rational verification [Wooldridge et al., 2016], makes it possible to reason formally about the behaviour of multiple interacting agents. This is the type of verification required for security games. We would expect to be able to apply these techniques to guarantee the outcomes of the strategies computed by the SSGs. However, security games (specifically SSGs) and formal verification (specifically rational verification) remain disconnected: no existing rational verification tool can reason Stackelberg equilibria, meaning SSG strategies cannot currently be formally guaranteed with available tools [Kulik et al., 2022].

1.2 Rational Verification and Security Games

This thesis integrates security games, threat modelling, and rational verification. The SSG is recognised as a foundational model for applying game theory to security [Tambe, 2012]. Surveys

and reviews e.g., Sinha et al. [Sinha et al., 2018], Ng & Hasan [Ng and Hasan, 2025] also describe SSGs as the central model in security game research.

Rational verification extends game-theoretic reasoning to the verification of strategic behaviour in multiagent systems. Notable contributions include the work of Abate et al. [Abate et al., 2021] and Najib [Najib, 2020], who introduced equilibrium based verification using temporal logic, and Aslanyan et al. [Aslanyan et al., 2016], who proposed a quantitative verification framework for cybersecurity scenarios. Zheng et al. [Zheng et al., 2022] directly studied Stackelberg strategies in partially observable stochastic environments by analysing the existence and structure of equilibrium in a turn-based, one-sided zero sum game setting, addressing challenges such as information leakage from public actions. However, with the exception of Zheng et al., these works do not explicitly address Stackelberg equilibria, which are central to security modelling where the defender commits to a strategy.

Attack Defence Trees (ADTs) are a commonly used formalism to represent the relationship between attack activities and countermeasures [Kordy et al., 2010, Kordy et al., 2013]. Early work in this area primarily focused on cost-benefit evaluation and graphical representation. More recent studies have introduced game-theoretic reasoning into ADTs, including quantitative verification [Aslanyan et al., 2016], asynchronous multiagent models [Arias et al., 2020], and partially observable security games [Khakpour and Parker, 2024]. Widel et al. [Widel et al., 2019] provide a survey of how formal methods have been applied to attack trees and their extensions in security modelling, covering interpretation, automated construction, and quantitative analysis. While these advances bring formal semantics closer to adversarial reasoning, they do not offer equilibrium computation or verification under the Stackelberg equilibrium.

This gap is also evident in verification tools. PRISM-games [Kwiatkowska et al., 2020] is a model checker for stochastic multiplayer games with support for Nash equilibrium, and EVE [Gutierrez et al., 2018] enables verification of temporal properties in multiagent systems, also limited to Nash reasoning. However, neither tool supports Stackelberg equilibria, which are essential for modelling defender commitment in cyber defence scenarios. Several works have addressed rational or equilibrium verification more broadly. For example, Abate et al. [Abate et al., 2021] provide a theoretical foundation for rational verification using game-theoretic temporal logic, while Chen et al. [Chen et al., 2013] focus on competitive stochastic systems, without specifically targeting Stackelberg or ADT-based reasoning. Khakpour and Parker [Khakpour and Parker, 2024] model attacker-defender interactions under uncertainty using partially observable stochastic games, yet do not address equilibrium strategies.

To the best of our knowledge, no existing framework maps known vulnerabilities to attack defence trees and then uses Stackelberg equilibrium reasoning with rPATL in a verification tool. Our work addresses this limitation by extending PRISM-games with Stackelberg equilibrium reasoning. We introduce the first framework that integrates ADTs, SSGs, and the temporal logic rPATL for the rational verification of security game properties.

1.3 Thesis Statement

This thesis proposes that integrating ADTs, Rational Verification, and rPATL into the SSG model can significantly improve security outcomes for defenders in cybersecurity scenarios. The primary objectives are to develop algorithms for verifying the Stackelberg equilibrium in these security games and to create a supportive tool for this process. We aim to demonstrate through theoretical analysis and empirical studies that our integrated approach outperforms traditional SSG models in various cybersecurity contexts. This research will help develop effective strategies, enabling defenders to anticipate and respond to cyber threats by understanding and predicting the rational behaviours of attackers.

The key research questions are as follows:

RQ1 How can the integration of ADTs, Rational Verification, and rPATL into the SSG model enhance the defender's capabilities and provide an understanding of the attacker's potential actions compared to traditional methods in cybersecurity?

To answer this question, we will propose a method integrating ADTs, Rational Verification, and rPATL into the SSG model. We aim to combine their strengths, enabling defenders to not only compute optimal resource allocation strategies but also verify that these strategies presume rational behaviour.

RQ2 How can algorithms be designed to verify Stackelberg equilibria in security games, along with creating a supporting tool, improve security outcomes and outperform traditional SSG models in various cybersecurity scenarios?

To answer this question, we will create algorithms and tools to verify that the computed equilibrium satisfies rational behaviour properties in security games, verify rationality and synthesis strategies. Additionally, we will evaluate the effectiveness of these integrated approaches in improving security outcomes compared to traditional SSG models.

RQ3 How can this framework be extended to enable the analysis of the tool beyond single shot games to analyse scenarios where an attacker combines multiple vulnerabilities to achieve their objectives?

To answer this question, we will extend the algorithms to support a diverse range of vulnerabilities, including repeated SSGs.

1.4 Publications

1. Phetmanee, S. (2023). An extended abstract on towards verifying Stackelberg security games. In *Doctoral Symposium of the 25th International Symposium on Formal Methods (FM 2023)*, Lübeck, Germany. [↗](#)
2. Phetmanee, S., Sevegnani, M., & Andrei, O. (2024). StEV_E: A rational verification tool for Stackelberg security games. In *Proceedings of the 19th International Conference on Integrated Formal Methods (IFM 2024)* (Vol. 15234, pp. 267–275). Springer. [↗](#)
3. Phetmanee, S., Sevegnani, M., & Andrei, O. (2025). Rational verification in repeated security games. In *Computer Safety, Reliability, and Security: SAFECOMP 2025 Workshops* (Vol. 15955, pp. 312–326). Springer. [↗](#)

1.5 Structure of the Thesis

This thesis is structured across six chapters, progressing from foundational concepts to the development, implementation, and evaluation of a new framework for rational verification.

- **Chapter 1 Introduction:** This chapter presents the motivation for the research, establishing the gap between security modelling and formal methods. It outlines the primary research questions, the key contributions of the thesis and the list of publications that have resulted from this work.
- **Chapter 2 Background:** This chapter reviews the foundations and related work. It covers the three main pillars of the research, including the use of game theory and Stackelberg games, the principle of threat modelling with ADTs, and the rational verification.
- **Chapter 3 Modelling Threats:** This chapter details transforming vulnerabilities into security games. It defines the processes from CVE to constructing ADTs, and then formalises SSGs.
- **Chapter 4 Rational Verification:** This chapter presents the idea of verifying rationality in security games and introduces an extension to the rPATL temporal logic to express properties under Stackelberg equilibrium.
- **Chapter 5 Evaluation and Discussion:** This chapter conducts the experiments rely on the proposed framework. It presents a detailed comparison of static and dynamic defence strategies, analysing metrics such as defender cost, system risk, and the impact of strategy setting. The chapter discusses the results and acknowledges the limitations of the work.
- **Chapter 6 Conclusion:** This chapter concludes the thesis by summarising the key findings and contributions. It revisits the research questions and outlines directions for future work.

Each research question is addressed across chapters. RQ1 (integration of ADTs, rational verification, and rPATL) is examined conceptually and formally in Chapters 2–3. RQ2 (design and implementation of verification algorithms and the StEV_E tool) is developed and demonstrated in Chapter 4 and further evaluated in Chapter 5. RQ3 (extension to repeated and multiple vulnerability settings) is evaluated experimentally in Chapter 5 and summarised in Chapter 6.

Chapter 2

Background

2.1 Game Theory

Game theory [[Osborne and Rubinstein, 1994](#)] is a toolkit that helps us make sense of how people or agents make decisions when they're interacting with each other. These tools can be applied to study the behaviour of individuals, groups, animals, and even machines, across domains such as markets, politics, and computer science. Games can be classified in several ways, including cooperative vs non-cooperative, simultaneous vs sequential, perfect vs imperfect information, and zero-sum vs non-zero-sum. The figure 2.1 below shows the general idea when we use game theory to play a game. In this figure, represented by chess, it is a sequential game. The player needs to plan a strategy, for example, the queen's gambit¹, before making a move. In cybersecurity, game theory offers a variety of methods to model and analyse interactions between attackers and defenders. For example, defenders can block attacker moves to optimise protection strategies, while attackers may choose tactics in response to observed defences.

2.1.1 Security Games

Security games [[Tambe, 2012](#)], and in particular Stackelberg Security Games (SSGs), are typically modelled in *strategic form*², where the defender (leader) commits to a mixed strategy and the attacker (follower) observes this commitment before responding. However, these games can also be represented in *extensive form* to capture their sequential structure, where the leader moves before the follower. This representation makes the order of play and information available to each player which is relevant in cybersecurity settings where an attacker may adapt to observed defensive measures. The study of security games emerged in the 2000s from research at the intersection of artificial intelligence and game theory [[Tambe, 2025](#)], motivated by the challenge of allocating limited security resources against adaptive adversaries. The SSG, where the defender

¹The queen's gambit is one of the oldest openings for white that starts after the moves 1.d4 d5 2.c4.

²Also referred to as the *normal form* representation of a game.

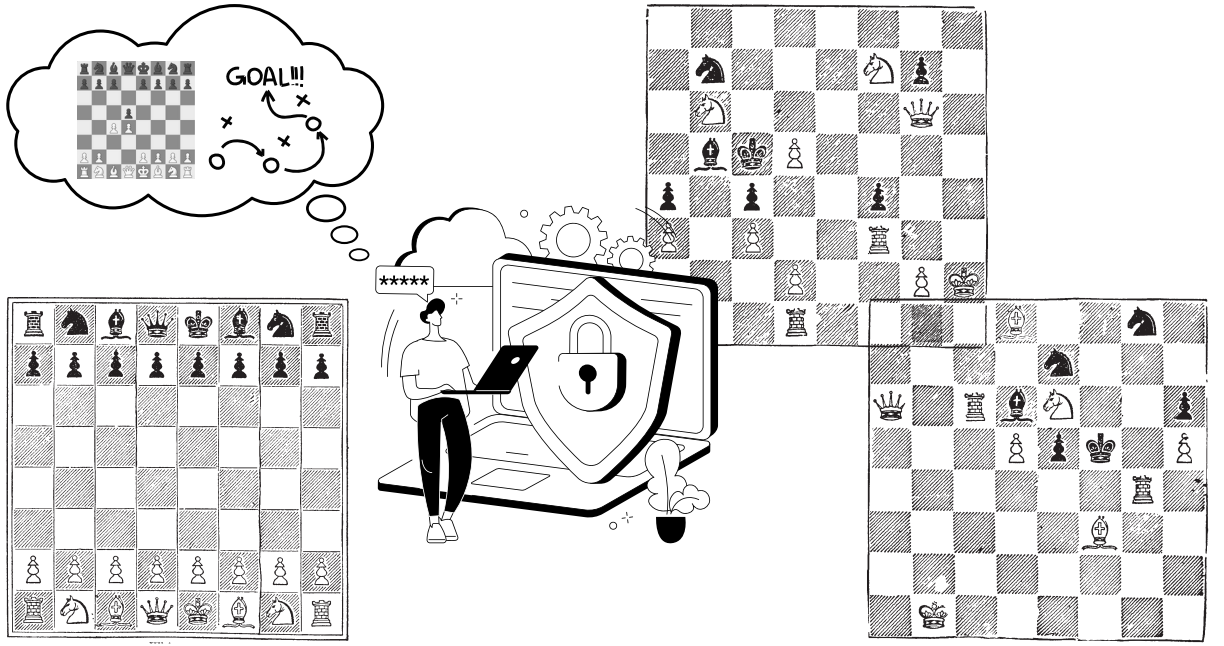


Figure 2.1: Game Theory.

commits first, became the core model. Early real world deployments demonstrated the practicality of this approach: ARMOR [Pita et al., 2009] for police checkpoint placement at Los Angeles International Airport, IRIS [Tsai et al., 2009] for scheduling US Federal Air Marshal missions, and PROTECT [Shieh et al., 2012] for optimising US Coast Guard patrols. Later research extended the framework to handle uncertainty, repetition, multiple attackers, and cyber defence.

The formalism for representing sequential move in multiagent systems is the *extensive form game*. It represents the game as a tree, with decision nodes corresponding to points where a player selects an action and edges corresponding to possible actions. Terminal nodes indicate the end of the game with payoffs assigned to each player.

Definition 1. An *extensive form game* [Leyton-Brown and Shoham, 2008] is a tuple $\mathcal{G} = (N, A, H, Z, \chi, \pi, \gamma, u)$ where:

- N is a set of n players where $N = \{i \mid 1 \leq i \leq n\}$, $n \geq 2$ and $n \in \mathbb{N}$;
- A is a finite set of actions;
- H is a set of non-terminal history or choice nodes;
- Z is a set of terminal history nodes disjoint from H , $H \cap Z = \emptyset$;
- $\chi : H \rightarrow 2^A$ is the action function, which determines the set of possible actions for each non-terminal history node;
- $\pi : H \rightarrow N$ is the player function, which assigns to each non-terminal history node a player $i \in N$ who chooses an action at that node;

- $\gamma : H \times A \rightarrow H \cup Z$ is the successor function, which maps a choice node and an action to a new non-terminal or terminal node such that for all $h_1, h_2 \in H$ and $a_1, a_2 \in A$, if $\gamma(h_1, a_1) = \gamma(h_2, a_2)$ then $h_1 = h_2$ and $a_1 = a_2$;
- $u = (u_1, \dots, u_n)$ is a profile of utility functions, where $u_i : Z \rightarrow \mathbb{R}$ is a utility function for player i that assigns a real-valued utility to each terminal history node.

In cybersecurity applications, such a model might represent a scenario in which the defender commits to a security configuration, the attacker observes this choice, and then selects an attack method. We will demonstrate the concrete example of SSGs in Example 4. The solution concept of interest in SSGs is the *Stackelberg equilibrium* [von Stackelberg, 2011], in which the leader selects a strategy anticipating the follower's optimal response.

2.1.2 Strategies

In game theory, a *strategy* is a complete plan that tells a player what action to take at every point where they might need to decide, even if that situation never actually happens during the game [Osborne and Rubinstein, 1994]. In *extensive form games* [Kuhn, 1953], these decision points are organised into *information sets*.

- In *perfect information games*, each information set contains only one decision point, so the player always knows exactly where they are in the game.
- In *imperfect information games*, an information set may contain several decision points, meaning the player cannot tell them apart.

A strategy must specify an action for every information set belonging to the player. Once all players strategies are fixed, the outcome of the game is determined.

Types of Strategies

Strategies are usually grouped into three main types [Osborne and Rubinstein, 1994]:

1. **Pure strategy** – a fixed, deterministic choice of action for every information set.

Definition 2. A pure strategy s_i for player i is a function

$$s_i : \mathcal{I}_i \rightarrow A$$

where $\mathcal{I}_i$ is the set of player i 's information sets and A is the set of available actions.

2. **Mixed strategy** – a probability distribution over the set of pure strategies S_i . The player randomly selects one pure strategy at the start of the game and sticks with it.
3. **Behavioural strategy** – a probability distribution over the actions at each information set. The player may randomise differently at different decision points.

We denote pure strategies by s_i and mixed or behavioural strategies by σ_i . When we write Σ_i , we mean the set of strategies of the type under discussion. In games with *perfect recall*³, mixed and behavioural strategies are equivalent outcomes [Kuhn, 1953].

Information sets and strategy profiles. An *information set* groups together decision nodes that a player cannot distinguish due to limited information. A *strategy profile* $\sigma = (\sigma_1, \dots, \sigma_n)$ is a tuple containing one complete strategy for each player, specifying their choices at every information set. Given a strategy profile, the outcome of the game is uniquely determined by the successor function γ .

Example 1. In a simplified security game, the defender might have to decide between *Scan* or *Monitor* at multiple stages. A pure strategy could be: “Scan at the first stage, monitor at the second stage, scan again at the third.” A mixed strategy could be: “Choose one of two complete security plans at random, each with probability 0.5.” A behavioural strategy could be: “At each stage, scan with probability 0.2 and monitor with 0.8, independent of previous choices.”

Strategies form the building blocks for equilibrium concepts such as the Nash equilibrium and Stackelberg equilibrium, which describe stable outcomes where no player has an incentive to change their plan. These concepts are discussed in the next section.

2.1.3 Equilibria

The analysis of a game often focuses on identifying *equilibria* in which no player can gain by changing their strategy. Different equilibrium concepts apply to different classes of games and strategy settings. A solution concept specifies conditions for equilibrium strategies and below are examples of equilibria. The analysis of a game often focuses on identifying equilibria *strategy profiles*⁴ in which no player can improve their payoff by changing their strategy. Different equilibrium concepts apply to different classes of games and strategic settings. A *solution concept* specifies the conditions that define such outcomes; computational algorithms⁵ can then be employed to determine strategies that satisfy these conditions.

Zero-sum Games and the Minimax Theorem

In *zero-sum* games, the players’ interests are directly opposed for every outcome (σ_1, σ_2) , the sum of their payoffs satisfies

$$u_1(\sigma_1, \sigma_2) + u_2(\sigma_1, \sigma_2) = 0.$$

³A player has perfect recall if they never forget what they knew or did earlier in the game.

⁴A strategy profile is a list of strategies, one for each player.

⁵The algorithms used to compute these equilibria are discussed in Chapter 4.

Example 2. In Tic-Tac-Toe, players take turns placing their marks (X or O) on a 3×3 grid. At the end of the game:

- If Player 1 wins, they receive payoff $+1$ and Player 2 receives -1 .
- If Player 2 wins, they receive payoff $+1$ and Player 1 receives -1 .
- In the case of a draw, both receive 0 .

In each possible outcome, the total payoff $u_1 + u_2 = 0$, which makes Tic-Tac-Toe a zero-sum game.

Let Σ_i denote the set of all strategies available to player i , and let $\sigma_i \in \Sigma_i$ denote a specific strategy for player i . The function $u_i : \Sigma_1 \times \Sigma_2 \rightarrow \mathbb{R}$ gives the payoff to player i . The classical solution concept for such games is based on the *minimax* principle [Von Neumann, 1928].

Definition 3. A *minimax equilibrium* is a strategy profile $(\sigma_1^*, \sigma_2^*)^6 \in \Sigma_1 \times \Sigma_2$ such that

$$\max_{\sigma_1 \in \Sigma_1} \min_{\sigma_2 \in \Sigma_2} u_1(\sigma_1, \sigma_2) = \min_{\sigma_2 \in \Sigma_2} \max_{\sigma_1 \in \Sigma_1} u_1(\sigma_1, \sigma_2) = v,$$

where $v \in \mathbb{R}$ is the *value* of the game. Player 1 can guarantee themselves a payoff of at least v , while Player 2 can hold Player 1 to at most v , when mixed strategies are allowed.

The *minimax theorem* states that every finite two-player zero-sum game has a minimax equilibrium in mixed strategies, and that the maximin value for Player 1 equals the minimax value for Player 2.

Nash Equilibrium

The concept of equilibrium was generalised beyond zero-sum games by John Nash [Nash, 1950, Nash, 1951]. A Nash equilibrium is the most widely used solution concept in noncooperative game theory. It is a strategy profile in which each player's strategy is a *best response* to the strategies of the others.

Definition 4. Let Σ_i be the set of strategies available to player i and $\Sigma = \Sigma_1 \times \dots \times \Sigma_n$ be the set of all strategy profiles. A strategy profile $\sigma^* = (\sigma_1^*, \dots, \sigma_n^*) \in \Sigma$ is a *Nash equilibrium* if, for every player $i \in N$,

$$\sigma_i^* \in \text{BR}_i(\sigma_{-i}^*),$$

where

$$\text{BR}_i(\sigma_{-i}) = \arg \max_{\sigma_i \in \Sigma_i} u_i(\sigma_i, \sigma_{-i})$$

is the set of best responses of player i to σ_{-i} . Equivalently,

$$u_i(\sigma_i^*, \sigma_{-i}^*) \geq u_i(\sigma_i, \sigma_{-i}^*) \quad \text{for all } \sigma_i \in \Sigma_i.$$

In a Nash equilibrium, no player can gain by unilaterally changing their strategy, given the strategies of the others. For example, In tic-tac-toe, if the board is full and the result is a draw,

⁶An asterisk in a strategy profile, e.g., $(\sigma_1^*, \dots, \sigma_n^*)$, indicates that it is an equilibrium profile.

the current strategy profile is a Nash equilibrium, neither player can improve their outcome by changing their last move, given the other player's moves. Nash proved that every finite game has at least one equilibrium in mixed strategies. Zero-sum and Nash refer to different aspects of game theory: zero-sum describes a class of games in which one player's gain equals the other's loss, while Nash equilibrium is a solution concept defining a stable outcome in any non cooperative game. In cybersecurity contexts, purely zero-sum reasoning is too restrictive and Nash equilibria often imply compromise between attacker and defender, which is undesirable. Therefore, the Stackelberg equilibrium offers a more appropriate representation of strategic in cyber defence.

Stackelberg Equilibrium

The *Stackelberg equilibrium* formalises the optimal strategies in such leader follower settings.

Definition 5. Let Σ_1 and Σ_2 be the sets of strategies available to each player, and let

$$\text{BR}_2(\sigma_1) = \arg \max_{\sigma_2 \in \Sigma_2} u_2(\sigma_1, \sigma_2)$$

denote the set of best responses of the follower to the leader's strategy σ_1 . A *Stackelberg equilibrium* is a strategy profile (σ_1^*, σ_2^*) such that:

1. $\sigma_2^* \in \text{BR}_2(\sigma_1^*)$ (the follower best responds to the leader's choice);
2. $\sigma_1^* \in \arg \max_{\sigma_1 \in \Sigma_1} u_1(\sigma_1, \sigma_2)$ for some $\sigma_2 \in \text{BR}_2(\sigma_1)$ (the leader chooses the strategy that maximises their payoff, anticipating the follower's best response).

In the *strong* Stackelberg equilibrium assumption, if the follower has multiple best responses, they choose the one that is most favourable to the leader. This equilibrium concept is especially relevant in security settings, where the defender (leader) commits to a strategy in advance, and the attacker (follower) reacts optimally after observing this commitment.

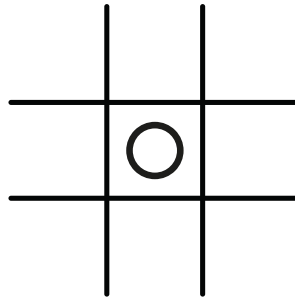


Figure 2.2: Tic-Tac-Toe.

Example 3. Consider a variant of tic-tac-toe as shown in Fig. 2.2 where Player 1 (the leader) commits to a strategy before the game starts and Player 2 (the follower) observes this commitment before making their own first move.

- **Leader's strategy:** Choose a starting square (centre, corner, or edge) and commit to it.
- **Follower's strategy:** Given the leader's committed starting square, choose their own first square to maximise their chance of winning or forcing a draw.

If the leader commits to the centre square (σ_1^*), the follower's best response (σ_2^*) is to choose a corner. This is because the centre is a strong opening, but with optimal defence, the follower can still guarantee at least a draw. In Stackelberg terms:

1. $\sigma_2^* \in \text{BR}_2(\sigma_1^*)$: the follower chooses the optimal counter to the leader's committed opening.
2. $\sigma_1^* \in \arg \max_{\sigma_1 \in \Sigma_1} u_1(\sigma_1, \sigma_2)$ for some $\sigma_2 \in \text{BR}_2(\sigma_1)$: the leader anticipates the follower's best reply and selects the opening that maximises their payoff usually the centre start in tic-tac-toe.

2.1.4 Stackelberg Security Games (SSGs)

Building on the idea of the Stackelberg equilibrium, earlier studies [Tambe, 2012, Sinha et al., 2018] recognised its potential in defence problems and integrated it into the solution concept of security games. A classic example is an airport security setting, where the defender must decide how to deploy a limited number of officers across multiple checkpoints. Before any passengers arrive, the defender commits to a deployment plan. An attacker, quietly blending into the crowd, observes this arrangement and then chooses the checkpoint that offers the best opportunity for a successful breach. SSGs help the defender in such scenarios by modelling the interaction as a leader follower game. The defender can compute an optimal allocation of resources that maximises security effectiveness under the constraint of limited resources.

The same story applies in cybersecurity. Here, the defender may need to allocate intrusion detection systems, patch management efforts, or firewall rules across different network. Once deployed, the attacker can scan to identify the weakest point of entry. In both cases, the defender must anticipate the attacker's response before committing resources.

SSGs provide a key solution to address resource allocation problems in security contexts. This is done by solving for Stackelberg equilibrium, which is a strategy for the leader that takes into account the follower's best response.

Representation for model checking. In this work, the SSG model is represented as a tuple $(\mathcal{G}, AP, L)$ suitable for formal verification where $\mathcal{G}$ is the extensive form game, AP is a set of atomic propositions, and L is a labelling function for states. The defender is the leader, and the attacker is the follower; after the defender mitigates or patches, the attacker chooses a strategy that maximises their utility. This work uses the strong Stackelberg equilibrium, where if the follower has more than one best move, they choose the one that is most advantageous to the leader.

Example 4. A Stackelberg security game adapted from the previous literature [Tambe, 2012], involving a *Bank* (the leader) and a *Hacker* (the follower). The cybersecurity team can choose

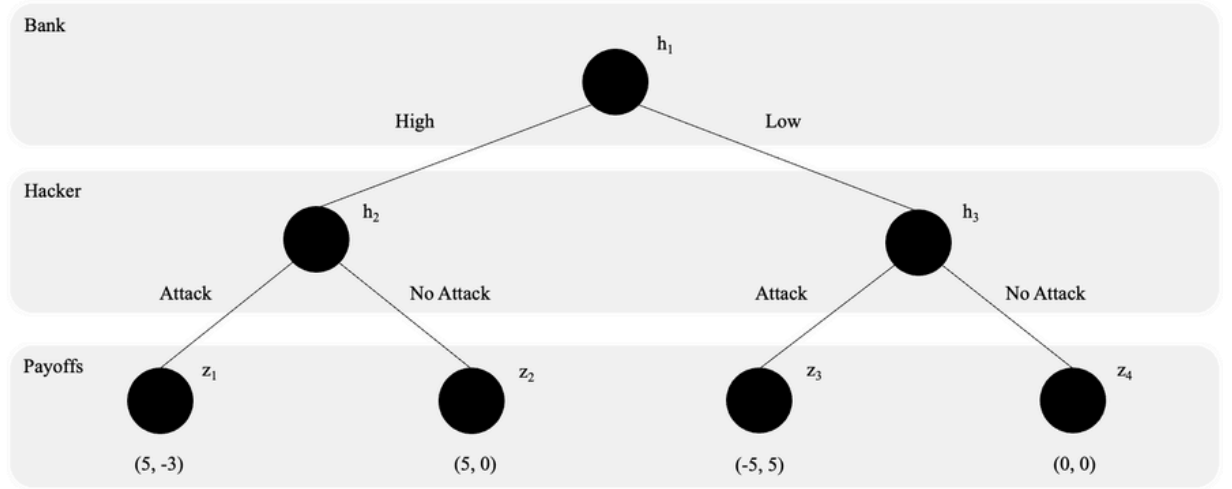


Figure 2.3: A Stackelberg Security Game.

between two strategies including invest heavily in security measures (*high*) or invest minimally (*low*). After observing, the *Hacker* can choose to either launch an attack (*attack*) or not (*noAttack*).

This game can be represented in extensive form, shown in Definition 1, as follows.

- The set of players $N = \{\text{Bank}, \text{Hacker}\}$.
- The set of actions $A_1 = \{\text{high}, \text{low}\}$ for the player *Bank* and $A_2 = \{\text{attack}, \text{noAttack}\}$ for the player *Hacker*, with $A = A_1 \cup A_2$.
- The set of non-terminal history nodes $H = \{h_1, h_2, h_3\}$, where h_1 represents the initial state of the game before the *Bank* makes its move; h_2 and h_3 represent the state of the game before the *Hacker* moves.
- The set of terminal history nodes $Z = \{z_1, z_2, z_3, z_4\}$, where
 - z_1 represents the state after the *Bank* chooses *high* and the *Hacker* chooses *attack*,
 - z_2 represents the state after the *Bank* chooses *high* and the *Hacker* chooses *noAttack*,
 - z_3 represents the state after the *Bank* chooses *low* and the *Hacker* chooses *attack*,
 - z_4 represents the state after the *Bank* chooses *low* and the *Hacker* chooses *noAttack*.
- The action function $\chi(h_1) = \{\text{high}, \text{low}\}$ for the *Bank*.
- The action function $\chi(h_2) = \chi(h_3) = \{\text{attack}, \text{noAttack}\}$ for the *Hacker*.
- The player function $\pi(h_1) = \text{Bank}$ and $\pi(h_2) = \pi(h_3) = \text{Hacker}$.
- The successor function $\gamma(h_1, \text{high}) = h_2$, $\gamma(h_1, \text{low}) = h_3$, $\gamma(h_2, \text{attack}) = z_1$, $\gamma(h_2, \text{noAttack}) = z_2$, $\gamma(h_3, \text{attack}) = z_3$, and $\gamma(h_3, \text{noAttack}) = z_4$.
- The utility functions $u_{\text{Bank}}(z_1) = 5$, $u_{\text{Bank}}(z_2) = 5$, $u_{\text{Bank}}(z_3) = -5$, $u_{\text{Bank}}(z_4) = 0$ for the *Bank* and $u_{\text{Hacker}}(z_1) = -3$, $u_{\text{Hacker}}(z_2) = 0$, $u_{\text{Hacker}}(z_3) = 5$, $u_{\text{Hacker}}(z_4) = 0$ for the *Hacker*.

Analysis and Result. This game can be represented as a game tree in Fig. 2.3.

- If the Bank chooses high and the Hacker chooses attack, the payoffs are 5 for the Bank and -3 for the Hacker. If the Bank chooses high and the Hacker chooses noAttack, the payoffs are 5 for the Bank and 0 for the Hacker.
- If the Bank chooses low and the Hacker chooses attack, the payoffs are -5 for the Bank and 5 for the Hacker. If the Bank chooses low and the Hacker chooses noAttack, the payoffs are 0 for the Bank and 0 for the Hacker.

Given these payoffs, the Bank would choose high since its worst outcome (5) is better than that of low (-5), anticipating that the Hacker would then choose noAttack because 0 is better than -3 . Thus, the Stackelberg equilibrium is high for the Bank and noAttack for the Hacker, with payoffs of 5 and 0 respectively.

2.2 Threat Modelling and Specification

This section outlines how formal methods are traditionally applied in security, focusing on modelling and specifying properties, but without considering strategic, rational adversaries. Cybersecurity is a concern for individuals, organisations, and governments. As technology becomes more integrated into all aspects of life, the need to protect our data and systems from cyber threats has never been more important.

For example, in a recent case [[BBC News, 2025](#)], Marks & Spencer was the target of a severe ransomware attack that disrupted IT infrastructure, disabled online shopping and leaked sensitive customer and employee data. Threat modelling is a process that helps identify, analyse, and mitigate potential security threats before they can be exploited by adversaries. There are several types of cyberattack we will discuss further.

2.2.1 Evolution of Cyber Threats

Cyber threats have become more sophisticated and harder to detect. Initially, cyberattacks were relatively simple, viruses spread through emails or software vulnerabilities. Cybercriminals and state sponsored hackers use more advanced techniques, often combining multiple types of attacks to achieve their objectives. For example, in the 2000s, many attacks targeted individual computers or websites, but over time, attackers have shifted to more complex, large scale attacks involving entire networks or infrastructure. More recently, the rise of ransomware has enabled even less skilled attackers to launch devastating attacks. The timeline shows how cyber threats have evolved:

- 1980s – 1990s: Early viruses and worms that infected individual computers.
- 2000s: The growth of phishing attacks, botnets, and large scale data breaches.
- 2010s: Increased use of ransomware, targeted attacks, and cyber espionage.
- 2020s – **Present**: AI, cloud based vulnerabilities, and ransomware as a service.

Cybercrimes are organised and often operate in groups using modern tools and techniques to attacks [Buczak and Seeker, 2016, Kuppa et al., 2021]. In some cases, these groups may target companies for financial gain, stealing sensitive data or extorting businesses. Cyber warfare is another growing threat. State sponsored cyberattacks target not just private organisations but governments, critical infrastructure, and national security. These attacks are more strategic aiming to disrupt systems. For example, *WannaCry* exploited CVE-2017-0144 [NIST, 2017], a critical vulnerability in Microsoft’s Server Message Block (SMB). The attack used the *EternalBlue* exploit, which had been leaked from the National Security Agency (NSA). *WannaCry* spread rapidly across networks. The attack affected over 230,000 computers in more than 150 countries, disrupting organisations such as the National Health Service (NHS). A security researcher discovered a kill switch which significantly slowed the spread of the ransomware. The attack highlighted the importance of patching systems.

2.2.2 Threats, Vulnerabilities, and Requirements

A *threat* is anything that could potentially exploit a weakness in a system, while a *vulnerability* is a weakness or flaw in the system that could be exploited by a threat. Requirements, specifically *security requirements*, are the policy or goal for protecting systems.

Threats

Cyber threats [Sharma et al., 2023] have evolved significantly over the years, and understanding these threats is crucial for protecting systems. The most common types of threats include:

Types of Threats

 **Malware:** Malicious software such as viruses, ransomware, trojans, and spyware designed to disrupt operations or gain unauthorised access to systems.

Example: Firefox advisories [[Mozilla Foundation, 2025](#)], vehicle hacking [[Lim, 2012](#)].

 **Phishing and Social Engineering:** Attacks trick users into receiving sensitive information (e.g., passwords, credit card numbers). Phishing commonly uses emails that look like communications from trusted organisations, such as banks or government agencies.

Example: Bring Your Own Device (BYOD) [[National Cyber Security Centre, 2021](#)], password weakness lists [[NordPass, 2023](#)].

 **Distributed Denial of Service (DDoS):** DDoS attacks use many systems to flood a target with traffic, blocking legitimate access.

Example: DDoS attacks on the healthcare sector [[Bulasara et al., 2025](#)].

 **Advanced Persistent Threats (APTs):** Attacks often conducted by state sponsored attackers aimed to steal sensitive information or access to critical systems.

Example: Stuxnet SCADA attack [[Langner, 2011](#)].

 **Supply Chain Attacks:** Exploiting vulnerabilities in third party vendors, contractors, or software providers to compromise the target.

Example: Default router and modem passwords [[PcWinTech.com, 2021](#)].

 **Zero-Day Exploits:** Attacks that exploit vulnerabilities unknown to the systems and the exploits can cause harm until a patch is released.

Example: OWASP Top Ten threats [[OWASP Foundation, 2025](#)], Log4Shell in Apache (CVE-2021-44228) [[National Institute of Standards and Technology \(NIST\), 2021](#)].

Fig. 2.4 shows the abstract example of the threat of the system that we can see people trying to bypass the security to make the home or this area weak. This example refers to the threat in our cybersecurity today that we need to learn to think like a hacker. Defenders need to analyse and create a reasonable security strategy.

Vulnerabilities

Vulnerabilities appear around us. One essential tool for tracking and managing vulnerabilities is the Common Vulnerabilities and Exposures (CVE)⁷, a database that catalogues vulnerabilities. Each vulnerability is assigned a unique identifier, enabling security experts to monitor and address specific weaknesses. Furthermore, the Common Vulnerability Scoring System (CVSS) assigns scores based on factors such as exploitability and potential impact. Together, CVE and CVSS

⁷<https://www.cve.org>



Figure 2.4: Bypassing Security [[Liedtke, 2024](#)].

help prioritise vulnerabilities and implement timely fixes before they can be exploited. Table [2.1](#) shows the dataset of CVEs we are interested in this research. This list highlights the recurring use of specific vulnerabilities in real world campaigns and serves as a basis for selecting relevant scenarios for modelling repeated security games in this work. Data compiled from public sources including Kuppia et al. [[Kuppia et al., 2021](#)], the MITRE ATT&CK knowledge base, the National Vulnerability Database (NVD), and the CISA Known Exploited Vulnerabilities (KEV) catalogue. The complete dataset is archived and available for download from this repository [↗](#).

Security Requirements

Security requirements [[Firesmith, 2003](#)] are the goals or policies for protecting systems. We can specify that a software system should protect against cyber threats in appropriate ways. One widely accepted principles that a system must follow is the CIA Triad [[Shostack, 2014](#)]:

- **Confidentiality:** Restrict access to data via encryption, authentication, and access controls.
- **Integrity:** Prevent unauthorised changes using hashing, checksums, and digital signatures.
- **Availability:** Ensure systems remain accessible through redundancy, failover, and DDoS.

Table 2.1: Common Vulnerabilities and Exposures (CVEs) reported as exploited by various Advanced Persistent Threat (APT) groups between 2010 and 2024.

Attack Group	Common Vulnerabilities and Exposures (CVE)
APT28 (Fancy Bear)	CVE-2014-4076, CVE-2015-1701, CVE-2015-2424, CVE-2015-2590, CVE-2015-3043, CVE-2015-5119, CVE-2016-7255, CVE-2016-7855
APT29 (Cozy Bear)	CVE-2023-38831
APT3 (Gothic Panda)	CVE-2010-3962, CVE-2013-4979, CVE-2014-1776, CVE-2014-4113, CVE-2014-6332, CVE-2014-8439, CVE-2015-2387, CVE-2015-2419, CVE-2015-2545, CVE-2015-3043, CVE-2015-3105, CVE-2015-3113, CVE-2015-5119, CVE-2015-5122, CVE-2015-7645, CVE-2015-8651, CVE-2016-1019, CVE-2016-4117, CVE-2016-4119, CVE-2016-7255, CVE-2017-0199, CVE-2017-11882, CVE-2018-0802, CVE-2018-4878
APT32 (OceanLotus)	CVE-2016-7255
APT33 (Elfin)	CVE-2017-11774, CVE-2018-20250
APT34 (OilRig)	CVE-2014-0640, CVE-2015-7547, CVE-2017-0199
APT35 (Charming Kitten)	CVE-2018-20250
APT37 (Reaper)	CVE-2013-4979, CVE-2014-8439, CVE-2015-2387, CVE-2015-2419, CVE-2015-2545, CVE-2015-3043, CVE-2015-3105, CVE-2015-5122, CVE-2015-7645, CVE-2016-1019, CVE-2016-4117, CVE-2017-0199, CVE-2018-0802
APT38	CVE-2015-8651, CVE-2016-1019, CVE-2016-4119
APT39 (Chafer)	CVE-2018-20250
APT41 (Double Dragon)	CVE-2012-0158, CVE-2015-1641, CVE-2017-0199, CVE-2017-11882, CVE-2019-3369, CVE-2019-3396
Carbanak	CVE-2012-0158, CVE-2013-3660, CVE-2013-3906, CVE-2014-6352, CVE-2015-1641, CVE-2015-1701, CVE-2015-1770, CVE-2015-2545, CVE-2016-5165, CVE-2016-5195, CVE-2017-5638
Cleaver	CVE-2010-0232
Dragonfly	CVE-2012-1723, CVE-2012-4792, CVE-2013-1347, CVE-2013-2465
Elderwood	CVE-2010-0249, CVE-2011-0609, CVE-2011-0611, CVE-2011-2110, CVE-2012-0779, CVE-2012-1535, CVE-2012-1875, CVE-2012-1889
FIN7	CVE-2017-11882
Gamaredon (Primitive Bear)	CVE-2012-0158, CVE-2017-0199, CVE-2017-11882, CVE-2018-0802
Gorgon Group	CVE-2017-0199
Group5	CVE-2014-4114
Ke3chang	CVE-2010-2883, CVE-2010-3333, CVE-2012-4681, CVE-2015-2545
Lazarus Group	CVE-2017-0144, CVE-2017-0145, CVE-2024-4947
Lotus Blossom	CVE-2017-11882
Molerats	CVE-2017-0199
MuddyWater	CVE-2017-5689, CVE-2017-11882, CVE-2019-2725
Naikon	CVE-2010-3333, CVE-2012-0158, CVE-2012-1856
NEODYMIUM / PROMETHIUM	CVE-2008-2551, CVE-2010-0188, CVE-2010-0840, CVE-2010-1297, CVE-2010-2568, CVE-2010-3336, CVE-2010-3653, CVE-2011-0097, CVE-2011-0611, CVE-2011-1823, CVE-2012-0056, CVE-2012-0158, CVE-2012-0507, CVE-2012-1723, CVE-2012-1889, CVE-2013-0074, CVE-2013-0422, CVE-2013-1493, CVE-2013-2423, CVE-2013-2460, CVE-2013-2551, CVE-2013-3896, CVE-2014-1761, CVE-2014-6332, CVE-2015-0072, CVE-2015-0310, CVE-2015-0311, CVE-2015-1671, CVE-2015-5119, CVE-2015-8651, CVE-2016-0034, CVE-2016-0165, CVE-2016-0167, CVE-2016-0189, CVE-2016-1010, CVE-2016-1019, CVE-2016-4117, CVE-2016-4171
OILRIG	CVE-2014-0640, CVE-2015-7547, CVE-2017-0199
PATCHWORK	CVE-2012-1856, CVE-2014-4114, CVE-2015-1641, CVE-2015-2545, CVE-2017-0199, CVE-2017-0261, CVE-2017-8570, CVE-2017-8750, CVE-2017-12824
PLATINUM	CVE-2013-1331, CVE-2013-7331, CVE-2015-2545, CVE-2015-2546
RANCOR	CVE-2018-0798
ROMCOM	CVE-2024-5274
SILENCE	CVE-2008-4250, CVE-2017-0199, CVE-2017-0262, CVE-2017-0263, CVE-2017-11882, CVE-2018-0802, CVE-2018-8174
SUCKFLY	CVE-2014-6332
TROPIC TROOPER	CVE-2017-5689, CVE-2017-11882, CVE-2018-0802
TURLA	CVE-2009-3129, CVE-2012-1723, CVE-2012-4681, CVE-2013-2729, CVE-2013-3346, CVE-2013-5065

Technologies provide the tools for security but the human element remains a critical factor. Many successful cyberattacks, particularly phishing and social engineering campaigns, exploit human psychology rather than complex software vulnerabilities. As a result, *cybersecurity awareness* [Krombholz et al., 2015] is a fundamental layer of an organisation's defence. This involves continuous training and education for all employees to help them recognise potential threats, understand secure practices such as, using strong passwords, identifying malicious emails, and know how to respond to a suspected security incident. Proactive organisations often run simulated phishing campaigns to test and improve employee vigilance. To support such efforts, several recognised standards and frameworks have been established:

- MITRE ATT&CK Framework: The ATT&CK (Adversarial Tactics, Techniques, and Common Knowledge) framework is a knowledge base of adversary tactics and techniques derived from real world observations [The MITRE Corporation, 2024]. It provides a common taxonomy for describing how attackers operate. Security teams use ATT&CK to validate their defences, improve threat detection, and simulate adversary behaviour.
- ISO/IEC 27000 Series: This standards led by the ISO/IEC 27001 [International Organisation for Standardisation, 2022], specifies the requirements for *establishing, implementing, maintaining, and continually* improving an Information Security Management System (ISMS). Certification against ISO 27001 is a best practice for business contracts to information security management.
- NIST Cybersecurity Framework (CSF): The CSF, developed by the U.S. National Institute of Standards and Technology, is one of the most widely adopted frameworks [National Institute of Standards and Technology, 2018]. It provides a high level, strategic view of cybersecurity risk management through five functions including *identify, protect, detect, respond, and recover*.

2.2.3 The Role and Ecosystem of CVEs

The CVE is managed by MITRE in collaboration with a community of over 300 CNAs.⁸ that collectively focus on vulnerability tracking. MITRE, a non profit organisation sponsored by the U.S. Cybersecurity and Infrastructure Security Agency (CISA), operates the CVE programme as the editor and moderator, maintaining the master CVE List and setting the rules for participation. The programme scales through a federated system of CNAs, which include major software vendors such as Microsoft, Google, and Apple; open source projects such as the Apache Software Foundation; research groups; and bug bounty platforms. These CNAs are authorised to assign CVE IDs to vulnerabilities within their respective scopes, enabling faster and more decentralised assignment and publication.

⁸CVE Numbering Authorities (CNAs) such as Google, Microsoft, Apple.

CVE data is integrated into several domains.

- **Cybersecurity Operations:** This is the primary consumer domain. Security teams use CVEs for vulnerability management to scan systems, identify weaknesses, and prioritise patching. CVEs are a core component of threat intelligence where they are correlated with active exploits listed in resources like CISA's Known Exploited Vulnerabilities (KEV) [[Cybersecurity and Infrastructure Security Agency \(CISA\), 2025](#)].
- **Software Engineering (DevSecOps):** In modern software development, CVEs are foundational to software analysis. Automated tools like GitHub's Dependabot^a or Snyk^b continuously scan project dependencies for libraries with known CVEs, alerting developers to risks before code is deployed. This is making a proactive move in the development lifecycle.
- **Others such as cyber insurance:** Insurers are increasingly using CVE data to assess the security of potential clients. An organisation's speed and effectiveness at patching critical CVEs can directly impact its insurance.

^a<https://github.com/dependabot>

^b<https://snyk.io>

Related Standards

- **CVSS (Common Vulnerability Scoring System):** Rates the severity of vulnerabilities. ^a
- **CWE (Common Weakness Enumeration):** This describes the type of vulnerability. For example, CVE-2021-44228 (Log4Shell) is an instance of CWE-502 (Deserialisation of Untrusted Data). ^b
- **CPE (Common Platform Enumeration):** A standardised naming scheme for IT products. ^c
- **CAPEC (Common Attack Pattern Enumeration and Classification):** This framework describes the patterns attackers use to exploit weaknesses. CAPEC helps connect a vulnerability (CVE/CWE) to real world adversary behaviour. ^d

^a<https://www.first.org/cvss>

^b<https://cwe.mitre.org>

^c<https://nvd.nist.gov/products/cpe>

^d<https://capec.mitre.org>

CVEs

The primary function of a CVE is to serve as a unique, authoritative reference for a single vulnerability. Its key features are designed to ensure clarity and consistency.

- **Unique Identifier:** The “CVE-YEAR-SEQUENTIAL_NUMBER” format provides a single, unambiguous name for a vulnerability, eliminating confusion and allowing security tools and professionals to correlate information from different sources. For example, CVE-2017-8759 [↗](#), CVE-2024-3400 [↗](#), and CVE-2024-4947 [↗](#).
- **Standardised Description:** Each entry contains a concise, neutral description of the vulnerability, outlining the nature of the flaw, the affected component, and the potential impact of its exploitation.
- **Scope Definition:** Using standards like CPE (Common Platform Enumeration), a CVE record specifies the exact products, vendors, and versions affected by the vulnerability.
- **Centralised References:** CVE entries act as a hub, providing a list of external references such as vendor advisories, bug tracker IDs, and third party security reports for more detailed technical information.
- **Lifecycle State:** Each CVE has a status such as RESERVED (the ID has been allocated but details are not yet public), PUBLISHED (the entry is public), or REJECTED.

KEV

The KEV catalogue [[Cybersecurity and Infrastructure Security Agency \(CISA\), 2025](#)] is not a list of all vulnerabilities; it is a highly action focused on immediate risk.

- **Evidence of Active Exploitation:** A vulnerability is only added to the KEV catalogue if CISA has reliable evidence that it is being actively used by malicious actors.
- **Prescriptive Remediation Deadlines:** The KEV catalogue is a directive not just a list. It mandates specific remediation deadlines for U.S. Federal Civilian Executive Branch agencies. These deadlines serve as a strong benchmark for all organisations to patching.
- **High Signal Prioritisation Tool:** The KEV catalogue cuts through the noise of thousands of CVEs disclosed. It allows security teams to focus their efforts on the vulnerabilities that pose the most direct and urgent danger.

CAPEC

It focus is on the adversary's perspective, the methods of attack rather than the vulnerabilities.

- **Adversary Centric Viewpoint:** It provides a dictionary of attack patterns describing how an attacker can exploit a certain type of weakness. This helps defenders think like an attacker.
- **Hierarchical Abstraction:** Attack patterns are organised in a hierarchy from broad categories such as, "Data Manipulation", specific techniques such as, "CAPEC-49: Password Brute Forcing" [↗](#). This allows for analysis at different levels of detail.
- **Linkages to Weaknesses:** CAPEC patterns are directly mapped to the CWEs they typically exploit. This creates a link between the attacker's method (CAPEC) and the underlying software flaw (CWE).
- **Rich Contextual Detail:** Each pattern includes fields describing the typical severity, required attacker skills, prerequisites for the attack to succeed, and likely consequences, making it a valuable resource for threat modelling, penetration testing, and developer training.

Gaps and Limitations of the CVE System

Scope Limitations: CVEs only cover publicly disclosed vulnerabilities in specific products. They do not cover system misconfigurations, procedural flaws, undiscovered zero day exploits, or vulnerabilities in bespoke, private software.

Timeliness: There can be a significant delay between the private discovery of a vulnerability, its public disclosure, and the final publication of a detailed CVE entry. This gap leaves a window of opportunity for attackers.

Quality and Consistency: The level of detail in CVE descriptions varies widely between different CNAs. Some entries are technical detail, while others can be vague mitigation.

Context Free Scoring: The base CVSS score associated with a CVE lacks environmental context. A "critical" 9.8 vulnerability in an internet facing server is a disaster which the same vulnerability in an air gapped, non critical system may be a low priority issue.

2.2.4 Adversaries and Defenders

The interaction between a malicious actor and a protecting entity can be modelled as a strategic game. We describe the characteristics, goals, and constraints of each player that are fundamental to analysing security scenarios and developing effective defences.

The Attacker

The term “attacker,” also known as an “adversary”, refers to any entity that attempts to compromise the security of a system. We model an attacker as a rational agent seeking to maximise their utility which is a function of achieving their objective while minimising their costs and risks. Attackers [Verizon Business, 2024] can be classified into primary types as follows:

-  **Cybercriminals:** The largest group motivated primarily by financial gain. Their tactics include ransomware, phishing for banking credentials, business email compromise, and stealing data to sell on darknet markets.
-  **Nation Actors:** These are highly resourced groups sponsored by governments with objectives tied to geopolitical influence often referred to as Advanced Persistent Threats (APTs).
-  **Hacktivists:** Motivated by a political or social agenda, these actors use cyberattacks to promote their cause. Their methods often include website defacement, DDoS attacks to silence opponents, and leaking sensitive data to cause reputational damage.
-  **Insider Threats:** An insider is any individual with authorised access to a system who misuses that access. Insiders are dangerous because they operate from behind existing security.

An attacker’s strategy is dictated by their objectives and capabilities. They only need to find and exploit a single vulnerability whereas the defender must protect the entire system against all possible attack vectors [Schneier, 1999]. Their actions are a trade off between the payoff of a successful attack and the cost of resources (time, tools, expertise) and the risk of detection.

The Defender

The defender is the organisation or individual responsible for protecting systems. In game theory, the defender is also a rational player but one whose utility is derived from minimising risk and maintaining system availability while operating under resource constraints. The main objectives of the defender are as follows.

-  **Identify:** Develop and manage cybersecurity risk. This includes asset management, risk assessment, and defining policies.
-  **Protect:** Implement safeguards to ensure the services. This involves access control, encryption, network security such as firewalls and security training.
-  **Detect:** This relies on continuous monitoring using tools like Intrusion Detection Systems (IDS) to alert when possible attacks are found.
-  **Respond:** Take action upon detecting an incident. This includes incident analysis, containment of the threat, and recovery.
-  **Recover:** Implement resilience plans to restore any services that were impaired due to an incident and communicate with stakeholders.

The defender operates under inherent disadvantages. They face significant resource constraints (budgets, personnel, and time) that make comprehensive protection impossible. This forces them to make strategic decisions about risk prioritisation, deciding which assets are most critical and which vulnerabilities pose the most urgent threat.

2.2.5 Attack Defence Trees (ADTs)

ADTs [Kordy et al., 2014] are a formal graphical model that combines attack trees and defence trees to describe and study security situations. Nodes show actions or conditions, and edges show how they are connected. The root is the main security goal and the leaves are basic actions or conditions. ADTs can be used for both descriptive and numerical analysis helping to find weaknesses, check how well defences work, and decide where to focus security efforts. They also represent the behaviour of attackers and defenders, and their formal definitions support reasoning in security games [Kordy et al., 2010, Kordy et al., 2013, Aslanyan et al., 2016, Wideł et al., 2019]. Each node can be broken down into smaller goals using “and” or “or” links and can have at most one countermeasure of the opposite type, showing the operation between attack and defence.

Components and Notation

- **Nodes and Players:** Every node in the tree represents a goal. The root node is the main goal of the primary player (typically the attacker). Each goal is owned by a player: either the *proponent* (the player trying to achieve the goal) or the *opponent* (the player trying to prevent it). In a security context, the attacker is the proponent of their attack goal, while the defender is the opponent.
- **Refinement:** A player can achieve a goal in different ways. This is captured by refining a node into child nodes. The relationship is defined by a logical operator:

An OR refinement (disjunctive, shown as a $\vee$ shape) means the proponent only needs to achieve one of the child goals. A rational proponent will choose the most beneficial child.

An AND refinement (conjunctive, shown as a $\wedge$ shape) means the proponent must achieve all of the child goals.

- **Basic Actions:** The leaves of the tree are basic actions. These represent atomic steps like ‘send a virus’ or ‘invest in firewalls’.
- **Countermeasures:** A key feature of ADTs is the ability to model defences. A goal node belonging to one player can be countered by an action of the opposing player. This is the explicit “defence” in an ADT and is represented by the ‘c’ operator in the formal syntax. In Figure 2.5, the defender’s action ‘invest high performance firewalls’ is a countermeasure to the attacker’s main goal.

The model can be created with attributes to evaluate security properties beyond simple success or failure. The process involves two steps:

1. **Attributing the Leaves:** Basic actions are assigned values from a specific attribute domain. Common examples include:
 - **Cost:** The financial cost or effort required to perform the action.
 - **Time:** The duration needed to complete the action.
 - **Success Probability:** The likelihood that the action succeeds.
 - **Skill:** The level of expertise required.
2. **Evaluation:** Semantic functions are defined to calculate the value of parent nodes based on the values of their children. The rules depend on the attribute and the logical operator. For example, when calculating the minimal cost for the attacker:
 - The cost of an AND-goal is the sum of the costs of its children.
 - The cost of an OR-goal is the minimum of the costs of its children as a rational attacker will choose the cheapest path to success.
 - For countermeasures, the attacker's cost may be adjusted depending on the defence value.

This propagation allows for the automatic calculation of high level properties, such as the minimal cost for an attacker to compromise the system, the probability of a successful defence, or the time. These capabilities are implemented in tools like ADTool [Kordy et al., 2013].

From ADTs to Security Games

ADTs provide a structured model for bridging the gap between informal security descriptions and formal analysis. The transformation from an ADT to a security game involves several conceptual mappings:

- **Players:** The proponent and opponent in the ADT directly map to players in the security game.
- **Strategies:** A player's strategy in the game corresponds to a set of choices within the ADT.

For the attacker, a strategy is a plan of basic actions that satisfies their top level goal.

For the defender, a strategy is a selection of countermeasures to deploy.

- **Payoffs:** The utility functions for the players are derived from given values.

In our work, the ADT captures the action, structure, and countermeasure dependencies of the scenario. This model is then transformed into a security game, allowing the application of solution concepts such as the Stackelberg equilibrium to compute optimal defensive strategies.

Definition 6 (Attack Defence Terms (ADTerms) [Kordy et al., 2014]). Given a set S , we denote S^* for the set of all strings over S and ϵ for the empty string. ADTerms are typed ground terms over a signature $\Sigma = (S, F)$, where

- $S = \{p, o\}$ is a set of types,
- $F = \{(\vee_k^p)_{k \in \mathbb{N}}, (\wedge_k^p)_{k \in \mathbb{N}}, (\vee_k^o)_{k \in \mathbb{N}}, (\wedge_k^o)_{k \in \mathbb{N}}, c^p, c^o\} \cup \mathbb{B}^p \cup \mathbb{B}^o$ is a set of function equipped with a mapping type: $F \rightarrow S^* \times S$, which expresses the type of each function as follows. For $k \in \mathbb{N}$,

$$\begin{aligned} \text{type}(\vee_k^p) &= (p^k, p) & \text{type}(\vee_k^o) &= (o^k, o) \\ \text{type}(\wedge_k^p) &= (p^k, p) & \text{type}(\wedge_k^o) &= (o^k, o) \\ \text{type}(c^p) &= (po, p) & \text{type}(c^o) &= (op, o) \\ \text{type}(b) &= (\epsilon, p), \text{ for } b \in \mathbb{B}^p & \text{type}(b) &= (\epsilon, o), \text{ for } b \in \mathbb{B}^o \end{aligned}$$

The component of $\mathbb{B}^p$ and $\mathbb{B}^o$ are typed constants representing the proponent's and the opponent's basic actions, respectively. This definition represents disjunctive ($\vee$) and conjunctive ($\wedge$) refinement operators of arity k , for a proponent (p) and an opponent (o) by the functions $\vee_k^p, \wedge_k^p, \vee_k^o, \wedge_k^o$. There is a binary function c^s (counter), where $s \in S$, connects a term of the type s with a countermeasure. The definition denotes the set of all ADTerms by T_Σ . We can partition into T_Σ^p and T_Σ^o for the set of terms of the proponent and opponent, respectively. We can denote the type of a term with a function $\tau : T_\Sigma \rightarrow S$ by $\tau(t) = s$ if $t \in T_\Sigma^s$. For the purpose of our work, we restrict the ADT models to ADTs with only one level of child nodes.

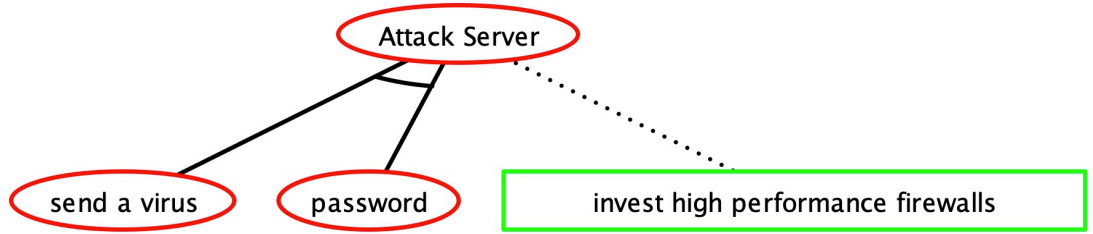


Figure 2.5: Example of an ADT

Example 5. We can apply the simple Stackelberg security game from Fig. 2.3 and transform it to the ADTerm $t = c^p(\wedge^p(\text{send a virus}, \text{password}), \text{invest high performance firewalls}) \in T_\Sigma^p$. This term can be shown with an ADT in Fig. 2.5.

Figure 2.5 presents an ADT including root node, basic actions, and operators. The root node, Attack Server, represents the objective of hackers. This ADT includes the attack actions such as send a virus and password, along with the defence action of invest high performance firewalls. This means that the server can be attacked by combining send a virus and password. However, the defender has the option to defend if he invests in high performance firewalls.

In this work, we consider a restricted form of ADTs, which simplifies the modelling process by allowing only a single level of nesting for child nodes. This restriction is sufficient for representing many practical security scenarios and automated processing and transformation into SSGs.

Definition 7. The restriction syntax consists of four types of terms:

$$\begin{aligned} c^d(b', b) \\ c^d(b', f(b_1, \dots, b_k)) \\ c^d(f'(b'_1, \dots, b'_{k'}), b) \\ c^d(f'(b'_1, \dots, b'_{k'}), f(b_1, \dots, b_k)) \end{aligned}$$

where:

- $b', b'_1, \dots, b'_{k'} \in B_d$ are defender actions;
- $b, b_1, \dots, b_k \in B_a$ are attacker actions;
- $f' \in \{\vee_d, \wedge_d\}$ denotes disjunction or conjunction for defender actions;
- $f \in \{\vee_a, \wedge_a\}$ denotes disjunction or conjunction for attacker actions;
- B_a and B_d are disjoint sets of attacker and defender actions;
- The operator c^d denotes a countermeasure relationship between defender and attacker.

Definition 8. The semantics of ADT describe how attacker and defender actions interact:

- $\wedge$: Conjunction – all listed actions must succeed for the goal to succeed.
- $\vee$: Disjunction – any one of the listed actions is sufficient for success.
- A term $c^d(b', \wedge_a(b_1, b_2))$ means the defender deploys countermeasure b' to stop a coordinated attack that requires both b_1 and b_2 .
- $c^d(\vee_d(b'_1, b'_2), b)$ means that either b'_1 or b'_2 is sufficient to counter a single attacker action b .

ADTs provide a formal yet intuitive way to capture the interplay between threats and defences. In our framework, they serve as an intermediate representation that is automatically transformed into security game models for further analysis using rational verification.

2.2.6 Alternative Threat Modelling Methods

The STRIDE Framework

The STRIDE [Shostack, 2014] framework is one of the most established and used methodologies for threat modelling. Its primary goal is to help software architects and developers identify and categorise potential security threats during the design phase. STRIDE provides a pattern for the six key categories of threats, each corresponding to the violation of a desired security property.

- **Spoofing:** This threat involves an adversary illegally assuming the identity of another person.
Example: An attacker creates a fake login page that looks identical to a banking website to steal user credentials.

- **Tampering:** This involves the unauthorised modification of data.
Example: An attacker intercepts a network communication and alters the amount of a financial transaction.
- **Repudiation:** This threat arises when a user denies having performed an action that they did, in fact, perform, and the system cannot prove otherwise.
Example: A user deletes log files to hide their malicious activity making it impossible to attribute the actions back to them.
- **Information Disclosure:** This involves the exposure of sensitive information to individuals who are not authorised to see it.
Example: A software bug allows an unauthenticated user to access a database containing personal customer information. This was the threat in the fictional Marks & Spencer attack mentioned earlier.
- **Denial of Service (DoS):** This type of attack aims to make a system or service unavailable.
Example: An attacker floods a web server with an overwhelming amount of traffic causing it to crash or become unresponsive.
- **Elevation of Privilege:** This occurs when a user with limited privileges manages to gain access to permissions and functions reserved for higher privilege accounts.
Example: A user exploits a vulnerability in an application to execute commands with root permissions on the underlying server.

Other methodologies include **LINDDUN** [Wuyts et al., 2014], which targets privacy threats by mapping data flows to risks and suggesting mitigations, and the **Cyber Kill Chain** [Hutchins et al., 2011] developed by Lockheed Martin which models seven stages of an attack to identify points for detection and disruption. While **STRIDE** offers an approach for identifying and categorising threats, its nature makes it less suitable for the game theory analysis we aim to perform. Consequently, we employ the structured and hierarchical framework of ADTs to support formal modelling and analysis.

2.3 Rational Verification of Security Games

We first review existing work on rational and equilibrium verification. Earlier tools such as *PRISM-games* [Kwiatkowska et al., 2020] and *EVE* [Gutierrez et al., 2018] can verify probabilistic or Nash equilibria in stochastic multiplayer games, but none provide reasoning under Stackelberg semantics. Subsequent work by Abate et al. [Abate et al., 2021] and Wooldridge et al. [Wooldridge et al., 2016] established theoretical frameworks for rational verification, but they remained limited to Nash or cooperative settings. These limitations motivate our extension of *PRISM-games* with explicit Stackelberg equilibrium reasoning.

2.3.1 Rational Verification

Rational verification [Abate et al., 2021, Najib, 2020] is a concept in the field of multi-agent systems which combines elements of game theory and formal verification. It involves determining whether a given temporal logic formula is satisfied in some or all game equilibria of a system. This means checking whether the system will exhibit the behaviour represented by the formula under the assumption that agents within the system act rationally in pursuit of their preferences. In the context of cybersecurity, rational verification can be a tool for analysing and predicting the behaviour of systems where multiple agents interact with their own strategies. These agents could represent different types, such as users, devices, or even potential attackers.

Definition 9. Let $\mathcal{G}$ be a game model and ϕ be a temporal logic property. *Rational verification* asks whether the property ϕ holds in some or all of the equilibria of $\mathcal{G}$ [Wooldridge et al., 2016]. The result is often expressed with respect to specific equilibrium concepts such as Nash equilibrium or Stackelberg equilibrium.

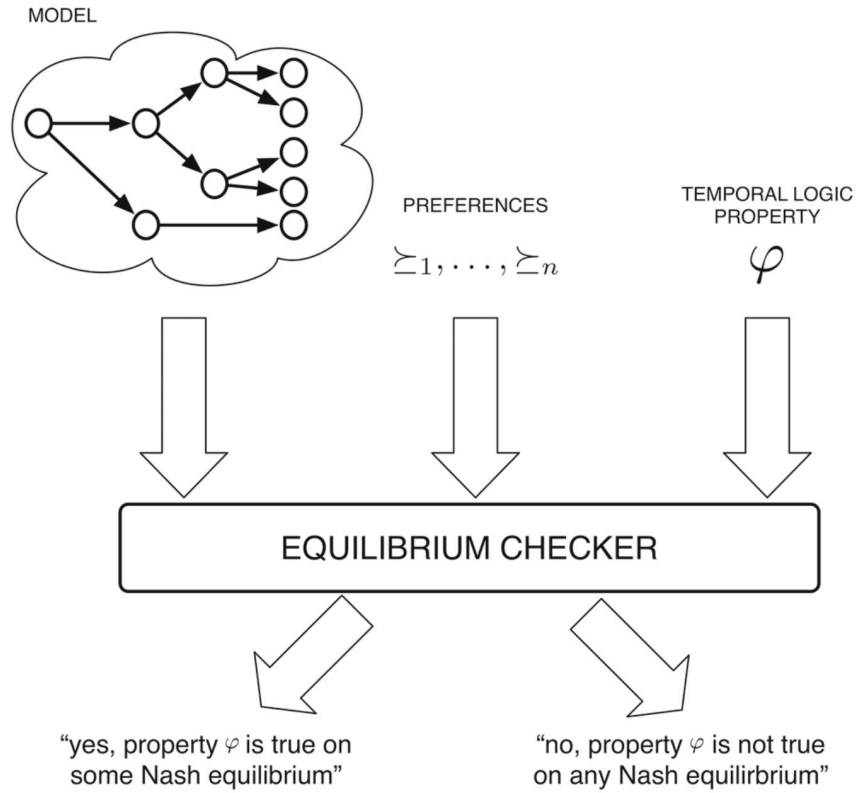


Figure 2.6: Equilibrium Checker Concept [Abate et al., 2021]. Preferences in this context denote the payoff ordering of each agent, used by the decision procedure to determine rational outcomes.

For example, consider a network where agents represent different devices. Each device has a strategy that determines how it responds to potential security threats, such as malware attacks. A rational verification process might involve checking whether, under the assumption of rational behaviour, all devices will correctly identify and respond to a malware attack. The temporal

logic formula in this case could represent the property “all devices correctly identify and respond to a malware attack”, and the game equilibrium could represent a state of the system where no device can benefit from changing its response strategy given that all other devices are responding correctly. This shows how rational verification can be used to analyse and predict the behaviour of systems in a cybersecurity context. We can gain insights into how the system will behave under different threat scenarios and use this information to design more effective security strategies.

Figure 2.6 represents the equilibrium checker concept, a key component of rational verification. The equilibrium checker is a decision procedure that determines whether a temporal logic property holds in at least one equilibrium outcome of the game. Recent works [Abate et al., 2021, Kwiatkowska et al., 2022a] on the Equilibrium checker have expanded its application to probabilistic systems, particularly concurrent stochastic games (CSGs). These games can model uncertainty and randomness in complex multi-agent environments. The rational verification problem has been studied for both non-cooperative and cooperative games in the qualitative probabilistic setting [Wooldridge et al., 2016, Abate et al., 2021]. However, we found that the previous works did not consider Stackelberg security games. This technique is particularly useful in cybersecurity, where defenders must anticipate the most likely responses from intelligent adversaries. By verifying properties under rational behaviour, security analysts can ensure that defence strategies remain effective even when attackers optimise their actions. Rational verification has been formalised and studied in a range of settings, particularly for multi-agent systems and security games [Wooldridge et al., 2016, Abate et al., 2021]. It provides an approach for reasoning about questions such as:

- Does there exist a defence strategy such that the attacker cannot reach a critical state?
- What is the expected damage under optimal attack behaviour?
- Can we guarantee a security threshold is met in equilibrium?

This approach supports the synthesis of strategies that are not only correct with respect to the system specification, but also optimal given the rational actions.

2.3.2 Temporal Logics for Strategic Verification

Temporal logics are formal languages used to specify and reason about how systems evolve over time [Pnueli, 1977]. In the context of rational verification, temporal logics such as Linear Temporal Logic (LTL) provide the property language, while equilibrium checking ensures that these properties are evaluated under rational play. We begin with the LTL before introducing its strategic extension, rPATL.

LTL: Syntax and Semantics

LTL is a temporal logic used to specify properties of execution paths in transition systems [Pnueli, 1977, Baier and Katoen, 2008]. However, it reasons about a single path at a time and lacks constructs for agents or strategies.

Definition 10 (LTL Syntax and Semantics [Baier and Katoen, 2008]). Given a set of atomic propositions AP , the syntax of LTL formulas is defined by the grammar:

$$\phi ::= \top \mid a \mid \neg\phi \mid \phi \wedge \phi \mid X\phi \mid \phi \cup \phi$$

where $a \in AP$. The core temporal operators are Next (X) and Until (U). Other key operators are:

- Eventually (Finally): $F\phi \equiv \top \cup \phi$ (ϕ eventually holds).
- Globally: $G\phi \equiv \neg F\neg\phi$ (ϕ always holds).

LTL semantics are defined over an infinite path $\pi = s_0, s_1, \dots$, where $L(s_i)$ is the set of propositions true at state s_i . The satisfaction $\pi, i \models \phi$ (formula ϕ holds at position i on path π) is defined as:

- $\pi, i \models a \iff a \in L(s_i)$
- $\pi, i \models X\phi \iff \pi, i+1 \models \phi$
- $\pi, i \models \phi_1 \cup \phi_2 \iff \exists j \geq i \text{ s.t. } \pi, j \models \phi_2 \text{ and } \forall k, i \leq k < j, \pi, k \models \phi_1$.

For example, if a means “the system is secure”, $G a$ specifies that the system is always secure. While useful for defining goals, LTL cannot express properties like “the defender *has a strategy* to keep the system secure.” For this, we need a logic that can reason about agents and strategies.

rPATL: Logic for Strategic and Probabilistic Systems

To overcome the limitations of LTL, we use rPATL [Chen et al., 2013] (probabilistic alternating-time temporal logic with rewards). It is a branching-time logic that extends temporal logic to reason about the abilities of coalitions of agents to achieve probabilistic and reward-based objectives. The definition below covers turn-based stochastic multi-player games (SMGs or TSGs)⁹ for zero-sum properties [Chen et al., 2013] and non-zero-sum cases [Kwiatkowska et al., 2019, Kwiatkowska et al., 2022b].

Definition 11. An SMG is a tuple $G = \langle \Pi, S, A, (S_i)_{i \in \Pi}, \Delta, AP, \chi \rangle$, where:

- Π is a finite set of players, and S a finite, non-empty set of states;
- A is a finite set of actions, with $A(s) = \{a \in A \mid \Delta(s, a) \text{ defined}\} \neq \emptyset$ for all $s \in S$;
- $(S_i)_{i \in \Pi}$ is a partition of S so that, in each state $s \in S_i$, player i chooses the action;
- $\Delta : S \times A \rightarrow D(S)$ is a (partial) probabilistic transition function (with $D(S)$ the set of discrete distributions over S);
- AP is a finite set of atomic propositions and $\chi : S \rightarrow 2^{AP}$ is a labelling function.

⁹<https://www.prismmodelchecker.org/games/modelling.php#smgs>

The syntax of rPATL is given by the following grammar:

Definition 12 (rPATL Syntax [Chen et al., 2013, Kwiatkowska et al., 2019, Kwiatkowska et al., 2022b]). The syntax of rPATL distinguishes between state and path formulae. The grammar is given by:

$$\begin{aligned}\phi &::= \top \mid a \mid \neg\phi \mid \phi \wedge \phi \mid \langle\langle C \rangle\rangle P_{\sim q}[\psi] \mid \langle\langle C \rangle\rangle R_{\sim x}^r[F^*\phi] && \text{(state formulae)} \\ \psi &::= X\phi \mid \phi U^{\leq k} \phi \mid \phi U \phi && \text{(path formulae)}\end{aligned}$$

where:

- $a \in AP$ is an atomic proposition and ϕ is a state formula.
- C is a coalition (set) of players.
- ψ is a path formula specifying a temporal property Next (X), Until (U), or Bounded Until ($U^{\leq k}$).
- r is a reward structure and $F^*\phi$ specifies a cumulative reward objective.
- $\sim \in \{<, \leq, \geq, >\}$, and q, x are rational valued bounds.

The key operators allow us to formally express complex strategic properties:

- **Probability:** $\langle\langle C \rangle\rangle P_{\sim q}[\psi]$ states that “coalition C has a strategy to ensure that the probability of path formula ψ holding meets the bound $\sim q$, regardless of how other players act”.
- **Reward:** $\langle\langle C \rangle\rangle R_{\sim x}^r[F^*\phi]$ states that “coalition C has a strategy to ensure the expected cumulative reward associated with structure r until a state satisfying ϕ is reached meets the bound $\sim x$ ”.

How Strategies are Verified

In our extension, these operators will later be interpreted under the assumption that players act according to a Stackelberg equilibrium strategy profile. Verifying a property involves a process called rational verification^a, which checks if a model of a system satisfies a given formula. The process consists of three main steps:

1. **Model Construction:** The security scenario is translated into a formal game model. This model defines the players, states, the actions available in each state, the probabilistic transitions between states resulting from actions, and the rewards associated with those transitions.
2. **Property Specification:** The desired security objective is formally written as an rPATL formula. For example, to verify if a defender can limit the probability of a breach, one might specify: $\langle\langle\text{defender}\rangle\rangle P_{<0.01} [F \text{“breached”}]$. This asks: “Does the defender have a strategy to ensure the probability of eventually being breached is less than 1%?”.
3. **Algorithmic Verification:** A model checker, such as PRISM-games, takes the game model and the rPATL formula as input. It computes the set of states that satisfy the formula. For rPATL, this typically involves solving a simpler two-player zero-sum game derived from the original model and the property. The tool returns a boolean answer (true/false) and provides a strategy or counterexample.

^aRational verification extends model checking by evaluating properties under equilibrium strategies.

2.3.3 Related Tools

The section shows the related applications that can analyse models from game theory against temporal logic specifications. In recent years, several specialised verification tools have been developed in this domain; however, their underlying algorithms and models reveal a critical and consistent limitation: no existing tool provides native support for the computation or verification of the Stackelberg equilibrium. This section reviews the capabilities of PRISM-games and EVE. Table 2.2 shows the comparison of their features.

PRISM-games

PRISM-games [Kwiatkowska et al., 2020] is a probabilistic model checker for the analysis of concurrent and turn-based stochastic games. As an extension of the PRISM [Kwiatkowska et al., 2011], it inherits a capacity for modelling systems with probabilistic and non-deterministic behaviour including game-theoretic domain. The tool supports the stochastic multiplayer game,

Table 2.2: Comparison of Features in Rational Verification Tools

Feature	PRISM-games	EVE
Model	Stochastic Games	Concurrent Games
Temporal Specification	rPATL	LTL
Quantitative Analysis (Probability & Rewards)	✓	-
Nash Equilibrium	✓	✓
Stackelberg Equilibrium	-	-

which allows for the representation of turn-based interactions, probabilistic transitions, and rewards. The recent version 3.0, as shown in Fig. 2.7, at that moment released further extends this to concurrent stochastic games where players can simultaneously make decisions.

The PRISM-games use the temporal logic rPATL [Chen et al., 2013]. This enables the specification and verification of complex quantitative properties against a game model. A user can formulate queries to compute a wide range of values such as the maximum probability of reaching a target state, the minimum expected cumulative cost to achieve an objective, or the minimum expected time to an event. A key feature is its ability to reason about strategic behaviour under various equilibrium concepts. The tool provides support for computing and verifying properties under zero-sum assumptions and for non zero-sum games under the Nash equilibrium concept. In technical term, the tool can synthesise strategies for social welfare optimal Nash equilibria, which identify outcomes where no player has a unilateral incentive to deviate and the collective social welfare (sum of rewards) is maximised [Kwiatkowska et al., 2019, Kwiatkowska et al., 2022b].

The computational approach for this equilibrium verification is predicated on finding a strategic fixed point. For zero-sum properties, the verification relies on value iteration, where at each state, a matrix game is solved using linear programming to compute a minimax strategy. For non zero-sum Nash equilibria at each state, the tool solves a bimatrix game (a two-player game where each player has their own payoff matrix). The implementation uses techniques such as an SMT based approach to find all equilibria values. The verification process is designed to identify a state of play from which no rational agent would choose to deviate on their own.

However, this is also the limitation to security games. The verification of a Stackelberg equilibrium is not a fixed point problem but a sequential optimisation problem. An algorithm designed to verify a Stackelberg game performs a nested computation:

1. Follower: For every possible strategy the leader, the algorithm must first compute the follower's optimal best response. This requires solving an optimisation problem for the follower for each of the leader's potential commitments.
2. Leader: The algorithm must then reason over this entire set of leader follower pairs to identify the initial commitment that maximises the leader's own utility.

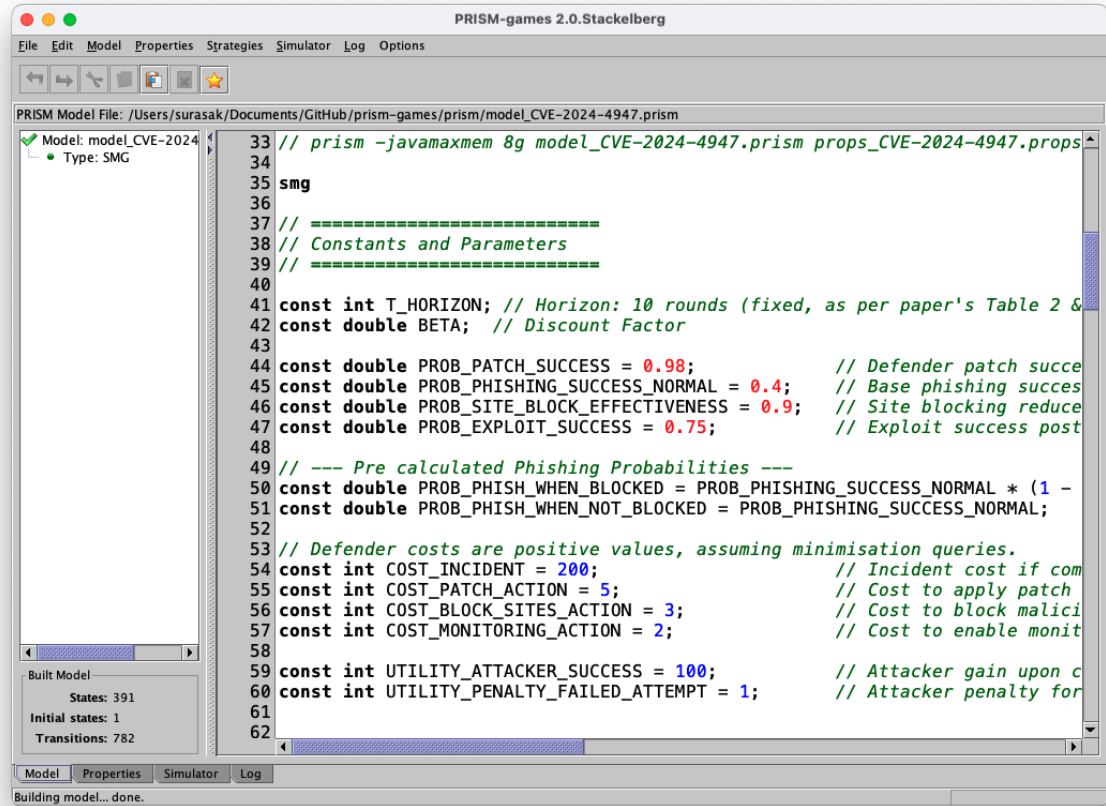


Figure 2.7: PRISM-games

This nested procedure is distinct from the methods used for Nash equilibrium analysis. The standard semantics of rPATL, as implemented in PRISM-games, are not designed to express this structure. The logic can ask, “what is the outcome if players play a Nash equilibrium?”, but it cannot ask, “what is the outcome if player 1 commits to a strategy that is optimal, given that player 2 will play their best response?”. Consequently, PRISM-games does not provide a built in mechanism to compute a Stackelberg equilibrium or to verify properties under the equilibrium.

EVE (Equilibrium Verification Environment)

EVE [Gutierrez et al., 2018] is another tool in this domain, focusing specifically on the temporal equilibrium analysis of concurrent and multi-agent systems. Rather than the probabilistic and reward-based focus of PRISM-games, EVE’s strength lies in the logical verification of temporal properties in concurrent game structures. EVE models are specified using the Simple Reactive Module Language (SRML), where system components are represented as players whose goals are expressed as LTL formulae. This approach is suited for analysing the correctness of systems where components are viewed as rational agents.

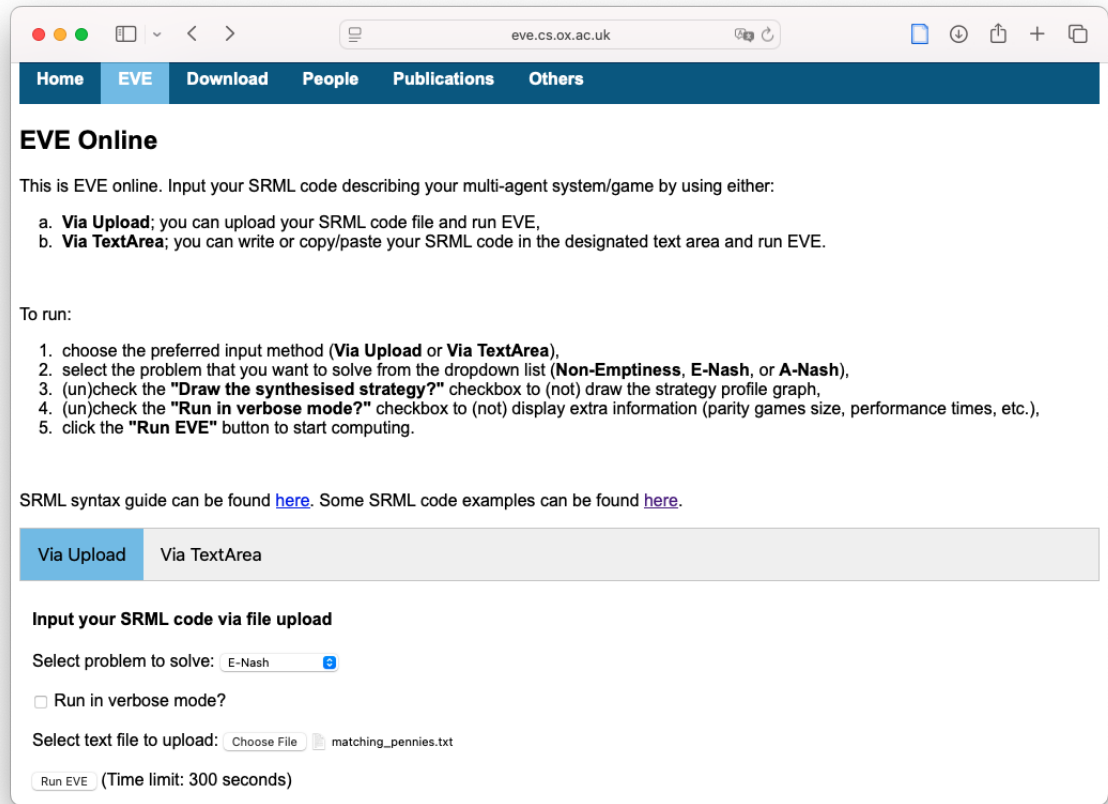


Figure 2.8: EVE

The primary function of EVE is to solve key decision problems related to the existence and properties of pure strategy Nash equilibria. The tool can solve the following problems:

- NON-EMPTINESS: Does a given game have at least one pure strategy Nash equilibrium?
- E-NASH: Does a given LTL property hold on *some* Nash equilibrium of the game?
- A-NASH: Does a given LTL property hold on *all* Nash equilibria of the game?

To achieve this, EVE, as shown in Fig. 2.8, uses a technique based on solving parity games. It translates the problem of finding a Nash equilibrium into a parity game, which can then be solved algorithmically. Beyond verification, recent research has considered the problem of equilibrium design [Gutierrez et al., 2019a], where the aim is to construct system specifications such that all equilibria satisfy desirable temporal properties. Later work extends this into a fully automated framework for both verification and synthesis of multi-player games, using equilibrium analysis automation [Gutierrez et al., 2020]. The computational difficulty of these rational verification problems has also been studied [Gutierrez et al., 2019b]. While the general case is hard, in that study shows that several useful fragments can be solved efficiently.

EVE represents an important advance in equilibrium verification, but it addresses a different class of problems than those encountered in SSGs. Its verification process is built around identifying pure strategy Nash equilibria, where players' goals are expressed as LTL formulae. This creates fundamental mismatches with the requirements of SSGs. First, EVE focuses on qualitative goal satisfaction in LTL, whereas SSGs require quantitative reasoning about payoffs and expected utilities such as financial loss or mission success. Second, EVE translates equilibrium verification to parity games for pure strategy Nash equilibria, a method that differs from the sequential optimisation needed for Stackelberg equilibria, where a leader commits and a follower best responds. Extending EVE to incorporate quantitative objectives and leader follower reasoning could make it more suitable for security game analysis.

In conclusion, the analysis of both PRISM-games and EVE confirms the existence of a clear and consistent *Stackelberg Verification Gap*. While both tools provide valuable capabilities, their underlying models and algorithms are suited to equilibrium concepts that do not align with the hierarchical structure¹⁰ of security games. In this thesis, we take a first step towards closing this gap by extending rational verification within PRISM-games to support Stackelberg equilibria. As a direction for future research, EVE could also be extended to handle Stackelberg equilibria.

2.4 Summary

This chapter has presented the foundations for this research. In particular, SSGs were discussed as an effective framework for representing sequential decision making, where a defender commits to a strategy before a rational attacker responds. We then introduced the concept of ADTs, which provide a structured way to model potential attacks and defensive countermeasures. The chapter further explored the principles of rational verification by verifying whether temporal logic properties hold under rational behaviour. We also reviewed existing tools that support these concepts. These gaps motivate the research contributions that follow. The next chapters build on this foundation to formalise SSGs, introduce a rational verification framework for single shot and repeated games, and present the algorithms, tools, and evaluations that demonstrate the effectiveness of our approach.

¹⁰See Definition 1 on Extensive Form Games

Chapter 3

Modelling Threats

3.1 Introduction

The challenge in applying game theory to practical security is bridging the gap between descriptive threat information and a structured model. This chapter introduces the first core contribution of this thesis that is a methodology for transforming threats into verifiable game models. The idea is that given unstructured vulnerability information, we aim to systematically produce a verifiable model that captures interactions in both single-shot and repeated scenarios.

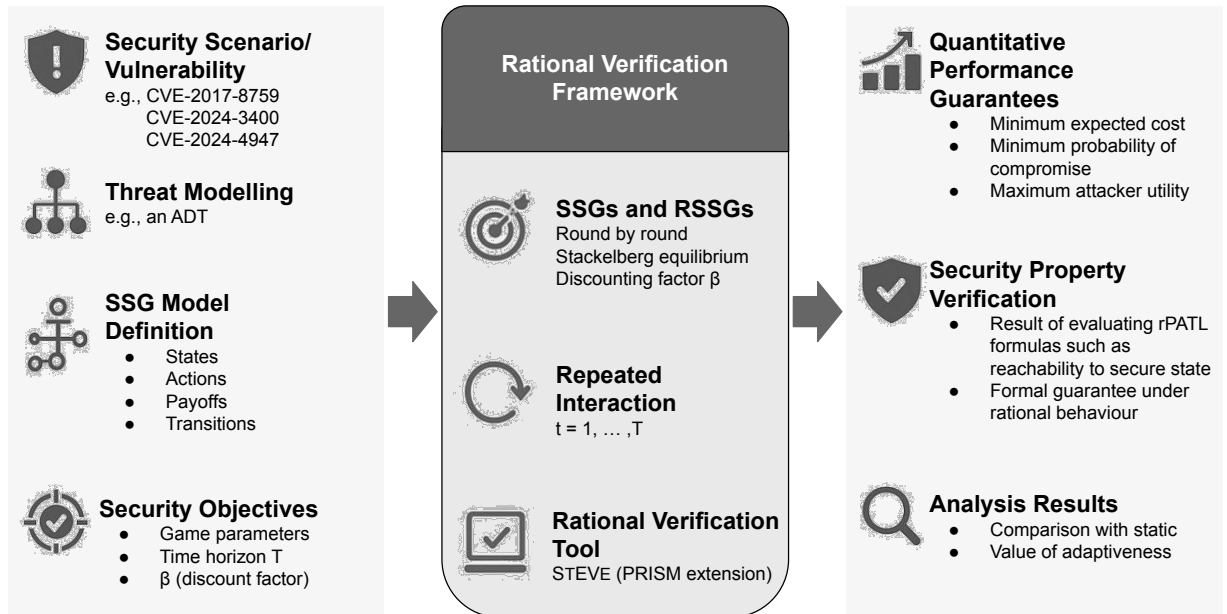


Figure 3.1: Overview of the rational verification framework for SSGs.

The Fig. 3.1 represents an overview of our framework. This picture is divided into three portions: input on the left, process in the centre, and output on the right. The input consists of a security scenario or vulnerability, which comes from CVEs such as CVE-2017-8759,

CVE-2024-3400, and CVE-2024-4947. The threat modelling we use is ADTs, which are then transformed into an SSG model that contains states, actions, payoffs, and transitions. Security objectives include game parameters, time horizon, and discount factor. The centre shows the elements of our verification process. In both SSGs and RSSGs, we verify properties stick with round by round Stackelberg equilibrium and discounting factor. This approach can support finite repeated interaction. At the bottom, we use our tool `StEVE` that extends the `PRISM`-game ability to handle this task. After the process, the output consists of quantitative performance guarantees including minimum expected cost, minimum probability of compromise, and maximum attacker utility. Result of evaluating rPATL formulae such as reachability of security state under the Stackelberg equilibrium. Finally, analysis results can be compared with static strategies that we can see the value of adaptiveness.

Existing approaches such as STRIDE, the Cyber Kill Chain, and MITRE ATT&CK are highly valuable for classification and planning, but lack a direct formal semantics that would enable verification. ADTs provide a structured, formal representation of adversarial behaviour with explicit countermeasures making them a suitable intermediate language between descriptive information and game models. The methodology proceeds in two main stages. First, we describe a process for mapping unstructured data from public vulnerability databases, specifically the CVE system into the structure of an ADT. Second, we present the transformation of these ADTs into two classes of game models including the static, one-shot SSG for analysing isolated interactions¹, and the more expressive RSSG for modelling adaptive adversaries. Throughout the chapter, we use concrete case studies based on CVEs to illustrate each stage of the modelling process.

3.1.1 Datasets

We created a dataset that reflects realistic vulnerabilities, particularly those exploited by Advanced Persistent Threats (APTs), to use in our rational verification framework. This list builds upon the work by Kuppa et al. [Kuppa et al., 2021] and has been updated with more recent information.² The dataset comprises 126 CVEs associated with these APT activities collected based on a list provided in Table 2.1. Data for each CVE was gathered programmatically using Python scripts interacting with publicly available resources, primarily the NVD API 2.0 for technical details and CVSS scoring, and the CISA KEV catalogue JSON feed to identify actively exploited vulnerabilities. Information regarding associated APT groups was gathered from public reporting and sources like MITRE ATT&CK. The collected data for each CVE includes attributes such as the attack sequences, mitigation, and severity. The dataset schema is summarised as follows.

¹It refers to a single encounter between defender and attacker.

²Updates were gathered from the knowledge base: <https://attack.mitre.org/groups/>

Dataset Schema

- **cve_id**: Unique identifier of the vulnerability (e.g., CVE-2024-4947).
- **is_key**: True/False flag showing whether the CVE is listed in the KEV catalogue.
- **description**: Official description of the vulnerability.
- **cvss_v3_score**: Numerical CVSS v3.1 base score (0.0–10.0).
- **cvss_v3_severity**: Severity rating (e.g., Low, Medium, High, Critical).
- **cvss_v3_vector**: Full CVSS v3.1 vector string.
- **attack_vector, attack_complexity, privileges_required, user_interaction, scope, confidentiality_impact, integrity_impact, availability_impact**:
Individual components parsed from the CVSS v3.1 vector.
- **cwe**: CWE identifier linked to the weakness.
- **cpes**: List of affected CPE strings describing vulnerable products.
- **published_date, last_modified_date**:
Dates when the CVE was published and last updated.
- **mitre_techniques**: Relevant MITRE ATT&CK techniques mapped to the CVE.
- **attack_sequences**: Description of exploitation steps.
- **mitigations_controls**: Suggested strategies to reduce or prevent exploitation.

This dataset was compiled to support research into rational verification, providing vulnerability data for modelling defensive interactions. The list of CVEs was compiled from public sources including Kuppa et al. (2021) [Kuppa et al., 2021], and we update to date using information from the MITRE ATT&CK knowledge base. Core technical details for each CVE (description, CVSS, CWE, CPE) were retrieved programmatically via Python scripts³ interacting with the official NVD API 2.0. Exploitation status was determined by the CISA KEV JSON feed. The dataset includes the fields `mitre_techniques`, `attack_sequences`, and `mitigations_controls` for all CVEs. However, for accuracy we recommend manual review.

This dataset can support a wide range of cybersecurity research and analysis tasks, including the analysis of trends in vulnerabilities exploited by APT groups and the correlation of vulnerability characteristics. It also provides inputs for simulating cyberattacks and defences in security game models. In addition, it might be used for training machine learning models aimed at vulnerability prioritisation or threat prediction, as well as for educational purposes in cybersecurity courses. However, this dataset relies on publicly available information from NVD, CISA, and MITRE. While efforts were made to ensure accuracy during collection, users should verify critical information against primary sources. The association between APT groups and CVEs is based on public reporting and may not be exhaustive.

³The Python scripts used for data collection and processing are also available at our repository.

CVE-2017-8759 Detail

Description

Microsoft .NET Framework 2.0, 3.5, 3.5.1, 4.5.2, 4.6, 4.6.1, 4.6.2 and 4.7 allow an attacker to execute code remotely via a malicious document or application, aka ".NET Framework Remote Code Execution Vulnerability."

Metrics

CVSS Version 4.0CVSS Version 3.xCVSS Version 2.0

NVD enrichment efforts reference publicly available information to associate vector strings. CVSS information contributed by other sources is also displayed.

CVSS 3.x Severity and Vector Strings:

 NIST: NVD	Base Score: 7.8 HIGH	Vector: CVSS:3.1/AV:L/AC:L/PR:N/UI:R/S:U/C:H/I:H/A:H
ADP: CISA-ADP	Base Score: 7.8 HIGH	Vector: CVSS:3.1/AV:L/AC:L/PR:N/UI:R/S:U/C:H/I:H/A:H

References to Advisories, Solutions, and Tools

By selecting these links, you will be leaving NIST webspace. We have provided these links to other web sites because they may have information that would be of interest to you. No inferences should be drawn on account of other sites being referenced, or not, from this page. There may be other web sites that are more appropriate for your purpose. NIST does not necessarily endorse the views expressed, or concur with the facts presented on these sites. Further, NIST does not endorse any commercial products that may be mentioned on these sites. Please address comments about this page to nvd@nist.gov.

QUICK INFO

CVE Dictionary Entry:
[CVE-2017-8759](#)

NVD Published Date:
09/12/2017

NVD Last Modified:
04/19/2025

Source:
Microsoft Corporation

CVSS v3.1 Severity and Metrics:
Base Score: 7.8 HIGH
Vector: AV:L/AC:L/PR:N/UI:R/S:U/C:H/I:H/A:H
Impact Score: 5.9
Exploitability Score: 1.8

Attack Vector (AV): Local
Attack Complexity (AC): Low
Privileges Required (PR): None
User Interaction (UI): Required
Scope (S): Unchanged
Confidentiality (C): High
Integrity (I): High
Availability (A): High

Figure 3.2: CVE-2017-8759

As introduced earlier in this section, we consider CVE-2017-8759, CVE-2024-3400, and CVE-2024-4947 to demonstrate the capability of the proposed framework and supporting tool. CVE-2017-8759 and CVE-2024-3400 were analysed using SSGs in our previous work [Phetmanee et al., 2025], while CVE-2024-4947 was analysed using RSSGs in [Phetmanee et al., 2026]. We begin with CVE-2017-8759 as shown in Fig. 3.2, used in a simplified example. This vulnerability is among the top ten exploited between 2016–2019 according to the record of CISA [Cybersecurity and Infrastructure Security Agency (CISA), 2020]. The details of this vulnerability, including the approach to attack and defence, are shown in Table 3.1. On the other hand, CVE-2024-3400 will be used as the first case study presenting single shot games, and CVE-2024-4947 will serve as the second case study illustrating repeated games. Our code and the examples presented in this thesis are publicly available as follows.

Source

Phetmanee, S., Sevegnani, M., & Andrei, O. (2025). Rational verification in repeated security games. In *Computer Safety, Reliability, and Security: SAFECOMP 2025 Workshops* (Vol. 15955, pp. 312–326). Springer. [Dataset](#) | [Tool](#) | [Paper](#)

Table 3.1: CVE-2017-8759, a remote code execution vulnerability in Microsoft .NET.

Attribute	Value
CVE	CVE-2017-8759
CVE Description	A remote code execution vulnerability in the .NET Framework (versions 2.0, 3.5, 3.5.1, 4.5.2, 4.6, 4.6.1, 4.6.2, 4.7). When parsing specially crafted documents or applications, the WSDL parser fails to sanitise CRLF sequences, leading to code injection into dynamically generated C# source. The file is compiled via <code>csc.exe</code> and executed, allowing arbitrary code execution. This vulnerability was exploited in the wild to deliver spyware.
Attack Sequences	The attack requires a sequence of actions: - Initial malicious document delivery (b_1): attacker sends a crafted Office/RTF file embedding a SOAP WSDL with CRLF injection. - Exploitation of the OLE object (b_2): document parsing triggers the WSDL parser to generate malicious C# code. - Payload delivery (b_3): the malicious C# code is compiled by <code>csc.exe</code> into a DLL, which is loaded to achieve spyware installation and remote code execution.
Mitigations and Controls	We consider three primary mitigation options: - Patch management (b'_1): apply Microsoft patches (released September 12, 2017) and update to secure .NET versions. - Email filtering (b'_2): block or quarantine suspicious Office/RTF attachments at the mail gateway. - Backup and recovery (b'_3): maintain regular system backups and recovery plans to mitigate impact in case of compromise. Additional measures include monitoring for abnormal use of <code>csc.exe</code>, deploying YARA signatures for SOAP/WSDL exploits, and applying application whitelisting with behavioural monitoring.
Characteristics	• Authentication: Not Required • Availability Impact: High (User Interaction Required) • Classification: Code Injection (WSDL Parser / CRLF Injection) • Credibility: Vendor Confirmed, Exploited in the Wild • Ease: Exploit Available (Targeted) • Application Type: .NET Framework, Office document parsing • Vendors: Microsoft • LocalCode: No, RemoteCode: Yes • CWE: CWE-94 (Improper Control of Generation of Code) • CVSS: - CVSS2.0 Vector: AV:N/AC:M/Au:N/C:C/I:C/A:C - Score: 9.3 (High) - CVSS3.0 Vector: AV:L/AC:L/PR:N/UI:R/S:U/C:H/I:H/A:H - Score: 7.8 (High)

3.2 Mapping CVEs to ADTs

We require a structure that represents attack and defensive countermeasures. Our methodology programmatically collects CVE details, CVSS scores, and KEV data from the NVD into the dataset. In this step we need two key elements:

- **Attacker Actions (B_a):** These are derived from the description of the exploit sequence. For example, an attack might require ‘initial malicious document delivery’, ‘exploitation of an OLE object’, and ‘payload delivery’. These steps become the basic attacker actions in the ADT.
- **Defender Actions (B_d):** These are derived from the official mitigation advice or recommended countermeasures for the CVE. For example, ‘patch management’, ‘email filtering’, and ‘backup and recovery’ are common defensive actions.

These sets of actions are then structured into an ADTerm⁴ using the restricted ADT syntax defined in Chapter 2. Def. 7. The relationship between the actions (conjunctive or disjunctive) is inferred from the CVE description. An attack that requires multiple steps to succeed is modelled with a conjunction ($\wedge_a$), whereas a set of alternative mitigation options for the defender is modelled with a disjunction ($\vee_d$). This automated mapping provides a systematic and repeatable method for translating unstructured threat intelligence into an ADT representation, which serves as the input for the next stage. As a result, we identify nine cases as follows.

1. $c^d(b', b)$
2. $c^d(b', \wedge_a(b_1, \dots, b_k))$
3. $c^d(b', \vee_a(b_1, \dots, b_k))$
4. $c^d(\wedge_d(b'_1, \dots, b'_k), b)$
5. $c^d(\vee_d(b'_1, \dots, b'_k), b)$
6. $c^d(\wedge_d(b'_1, \dots, b'_k), \wedge_a(b_1, \dots, b_k))$
7. $c^d(\wedge_d(b'_1, \dots, b'_k), \vee_a(b_1, \dots, b_k))$
8. $c^d(\vee_d(b'_1, \dots, b'_k), \wedge_a(b_1, \dots, b_k))$
9. $c^d(\vee_d(b'_1, \dots, b'_k), \vee_a(b_1, \dots, b_k))$

Example 6 (CVE-2017-8759). *The CVE-2017-8759, as shown in Table 3.1, we can see the sequence of attack actions: ‘initial malicious document delivery’ (b_1), ‘exploitation of the OLE object’ (b_2), and ‘payload delivery’ (b_3). The defender has several mitigation options, of which we consider three primary ones: ‘patch management’ (b'_1), ‘email filtering’ (b'_2), and ‘backup and recovery’ (b'_3). If we consider all the above actions, both the defender and attacker mean the possible ADTs can be cases.*

- $$\begin{aligned}
 &c^d(\wedge_d(b'_1, b'_2, b'_3), \wedge_a(b_1, b_2, b_3)) \\
 &c^d(\wedge_d(b'_1, b'_2, b'_3), \vee_a(b_1, b_2, b_3)) \\
 &c^d(\vee_d(b'_1, b'_2, b'_3), \wedge_a(b_1, b_2, b_3)) \\
 &c^d(\vee_d(b'_1, b'_2, b'_3), \vee_a(b_1, b_2, b_3))
 \end{aligned}$$

⁴See the Definition 6

The natural language descriptions in CVEs might be ambiguous, and the logical relationship between attack steps or defensive actions is not always explicit. Our methodology employs heuristics, such as identifying sequential dependencies (“first A, then B”) to infer conjunctive relationships ($\wedge_a$) and lists of alternative mitigations (“patch X or Y”) to infer disjunctive relationships ($\vee_d$). In cases of ambiguity, a manual review by a security analyst is used to confirm the ADT structure. This semi-automated approach balances the need for a repeatable process with the complexity of real world threats. According to the information from the dataset, this vulnerability is analysed to the case $c^d(\vee_d(b'_1, b'_2, b'_3), \wedge_a(b_1, b_2, b_3))$, shown in Fig. 3.3.

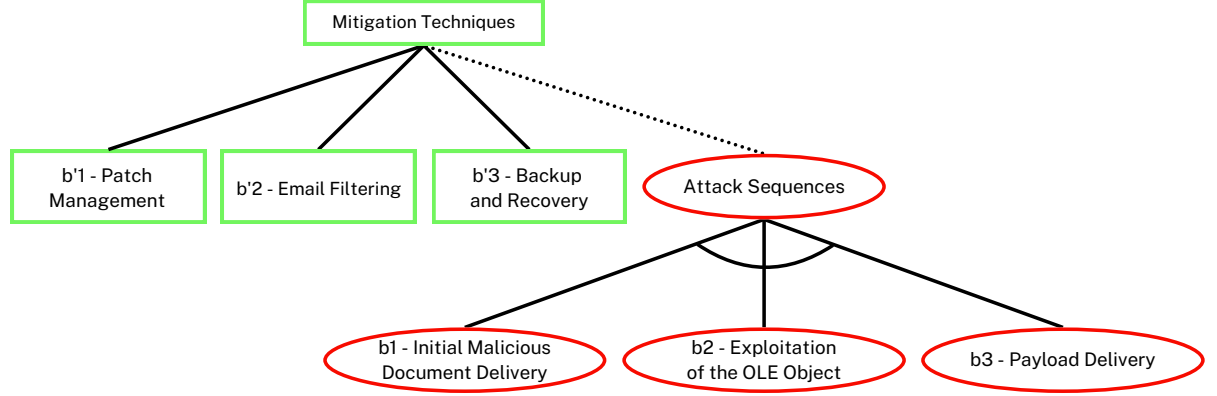


Figure 3.3: Graphical representation of the ADTerm $c^d(\vee_d(b'_1, b'_2, b'_3), \wedge_a(b_1, b_2, b_3))$ generated from the CVE-2017-8759 example, which is then automatically transformed into an SSG. The defender (green nodes) can choose mitigation techniques for each of the following separately: b'_1 - patch management, b'_2 - email filtering, or b'_3 - backup and recovery. However, the attacker (red nodes) can initiate an attack by combining actions involving b_1 - initial malicious document delivery, b_2 - exploitation of the OLE object, and b_3 - payload delivery.

Generality and Preconditions. Our CVE $\rightarrow$ ADT $\rightarrow$ SSG/RSSG pipeline is template driven and therefore general across vulnerabilities that satisfy three preconditions: (i) a structured CVE entry with CVSS/CWE/CPE fields; (ii) at least one describable *attack sequence* and corresponding *mitigations*; and (iii) parameterisation of costs/impacts (from CVSS and practitioner estimates). Under these inputs, our mapping templates (Def. 6) systematically instantiate attacker/defender primitives and logical connectives ($\wedge_a, \vee_a, \wedge_d, \vee_d$), and the ADT $\rightarrow$ SSG procedure (Alg. 1) deterministically yields a turn-based leader follower model for verification. The approach thus scales by data coverage, not by bespoke modelling: we retrieved a dataset of 126 CVEs and associated attributes (Sec. 3.1.1), which our pipeline can process without manual game logic.

3.3 Transforming ADTs to SSGs

Once a threat is represented as an ADT, the next stage is to transform this structure into a formal game model suitable for strategic analysis. The graphical representation of transformation is shown in Fig. 3.4. Each case of the ADT will transform into the game tree. This section details the transformation into an SSG. Our transformation algorithm takes an ADT as input and constructs the components of the SSG tuple $(\mathcal{G}, AP, L)$ where $\mathcal{G} = (N, A, H, Z, \chi, \pi, \gamma, u)$. The transformation, detailed in Algorithm 1, systematically maps the ADT's structure to the game's structure. The defender and attacker actions from the ADT's B_d and B_a sets directly define the action sets for the players in the game. Here, F denotes the set of logical operators (e.g. $\wedge^d, \wedge^a, \vee^d, \vee^a$) extracted from the ADT, which determines how defender and attacker conjunctions and disjunctions are mapped into the corresponding SSG components. The tree structure of the game, with the defender as the leader moving first from the initial node h_0 , is constructed based on the countermeasure relationship in the ADT. The payoffs u_i at the terminal nodes Z are derived from the CVSS impact scores and provided estimates for the costs of defensive actions and the damages from an attack.

Addressing ambiguity in CVE-ADT translation. Converting textual vulnerability descriptions into ADTs involves interpretation. Descriptions in the CVE database vary in granularity and may omit contextual assumptions such as the attacker's required privileges or system configuration. To minimise subjectivity, the mapping process follows a set of transformation templates that associate each MITRE CWE category with specific attack or defence primitives. Nevertheless, residual ambiguity remains when different analysts could model the same narrative using slightly different decompositions. For example, CVE-2024-3400 can be represented either as a single command injection node or as a two stage escalation (input validation $\rightarrow$ execution). Our framework quantifies this variation during validation by comparing equivalent sub trees and highlighting inconsistent interpretations.

Example mapping. Table 3.2 shows how an ADT fragment is transformed into the SSG tuple.

Table 3.2: Illustrative mapping from ADT elements to SSG components.

ADT Element	SSG Component
$\vee_d$ (Patch Chrome or Enable Monitoring)	Defender action set A_1
$\wedge_a$ (Exploit + Payload Delivery)	Attacker action set A_2
Base action node b'_1 (Patch)	State transition labelled <i>patch</i>
Base action node b_1 (Exploit V8)	State transition labelled <i>exploit</i>

Example 7 (CVE-2017-8759). *In this experiment, we assume that the defender takes the following disjunctive actions: b'_1, b'_2 , and b'_3 . The attacker uses the following conjunctive actions: b_1, b_2 ,*

Algorithm 1 Transforming an ADT to an SSG**Input:** ADT with defender root node $\iota(T)$, payoff matrix U **Output:** SSG as $(N, A, H, Z, \chi, \pi, \gamma, u, AP, L)$

```

1:  $N \leftarrow \{defender, attacker\};$ 
2: if  $\wedge^d \in F$  then
3:    $Action_{defender} \leftarrow \{\mathbb{B}^d, \emptyset\};$ 
4: else if  $\wedge^a \in F$  then
5:    $Action_{attacker} \leftarrow \{\mathbb{B}^a\};$ 
6: else
7:    $Action_{defender} \leftarrow 2^{\mathbb{B}^d};$ 
8:    $Action_{attacker} \leftarrow 2^{\mathbb{B}^a} \setminus \{\emptyset\};$ 
9: end if
10:  $A \leftarrow Action_{defender} \cup Action_{attacker};$ 
11: Create defender node  $h_0$  and set its properties;
12:  $H \leftarrow \{h_0\};$ 
13:  $\chi(h_0) \leftarrow Action_{defender};$ 
14:  $L(h_0) \leftarrow AP;$ 
15:  $\pi(h_0) \leftarrow defender;$ 
16: for all  $B' \in Action_{defender}$  do
17:   Create attacker node  $h_{attacker};$ 
18:    $H \leftarrow H \cup \{h_{attacker}\};$ 
19:    $\chi(h_{attacker}) \leftarrow Action_{attacker};$ 
20:    $L(h_{attacker}) \leftarrow AP;$ 
21:    $\pi(h_{attacker}) \leftarrow attacker;$ 
22:    $\gamma(h_0, B') \leftarrow h_{attacker};$ 
23:   for all  $B \in Action_{attacker}$  do
24:     Create terminal node  $z;$ 
25:      $Z \leftarrow Z \cup \{z\};$ 
26:      $L(z) \leftarrow AP;$ 
27:      $\gamma(h_{attacker}, B) \leftarrow z;$ 
28:     Assign  $u_{defender}(z)$  and  $u_{attacker}(z)$  using  $U;$ 
29:   end for
30: end for

```

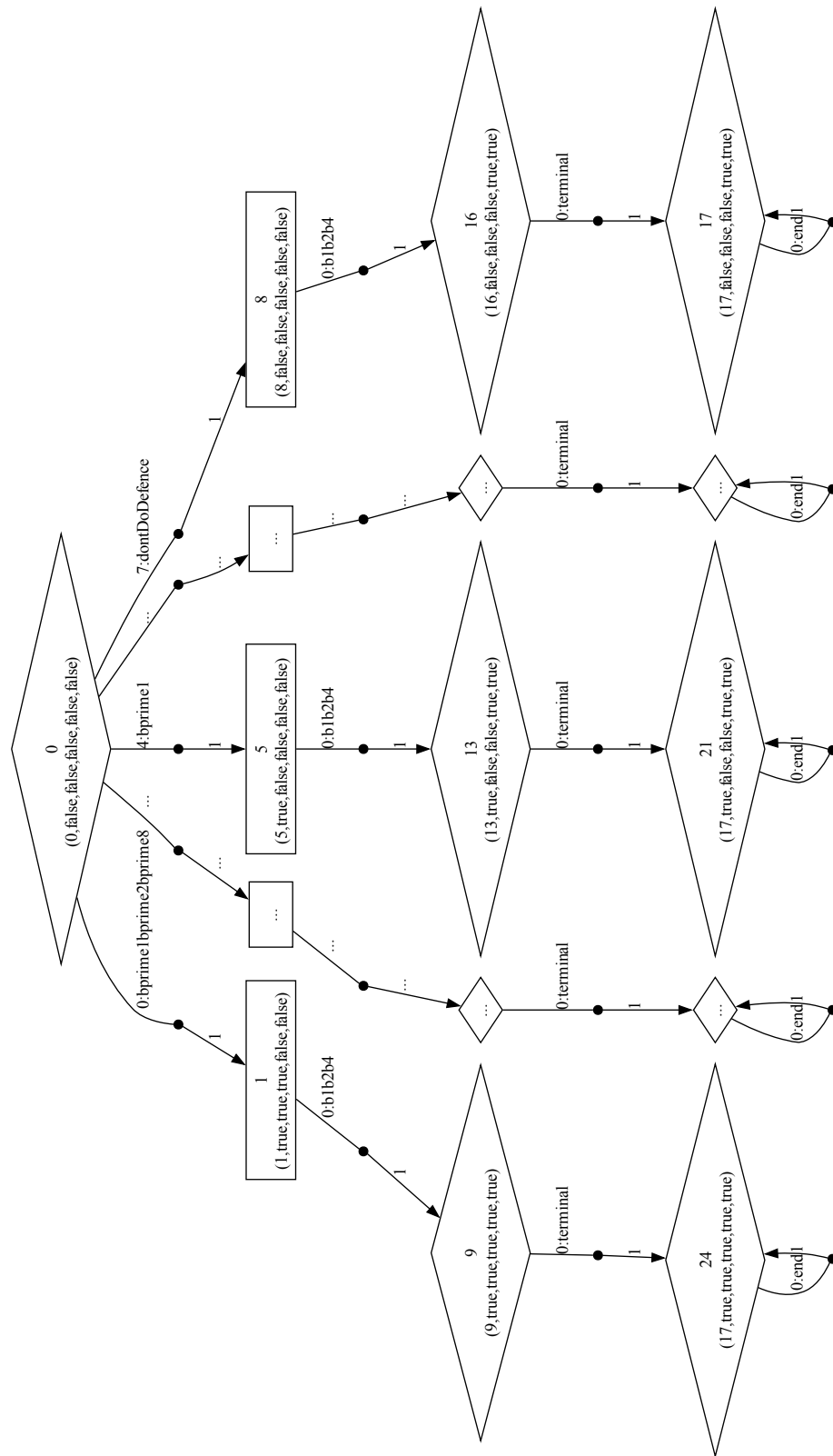


Figure 3.4: Graphical representation of the SSG for the CVE-2017-8759 example, as generated from the PRISM-games extension in StEVE. In this diagram, diamonds represent defenders and rectangles represent attackers. The strategy relies on the Stackelberg equilibrium, where the defender applies only patch management (action b'_1), which will cost the defender £24,750 and the attacker earns £4,425. Some states are omitted from the diagram; these are indicated by “...”.

and b_3 . The ADTerm encoding this scenario is $c^d(\vee_d(b'_1, b'_2, b'_3), \wedge_a(b_1, b_2, b_3))$ and graphically represented in Fig. 3.3, which is then transformed into an SSG with payoffs derived from adjustment costs. Each of the defender's strategies $\sigma_1 \in 2^{\{b'_1, b'_2, b'_3\}}$ is verified against the security requirements $\rho_1 = \mathbf{F} \text{ applyPatch}$, $\rho_2 = \mathbf{F} \text{ filterEmail}$, and $\rho_3 = \mathbf{F} \text{ backup}$, where `applyPatch`, `filterEmail`, and `backup` are user-defined atomic propositions corresponding to the actions b'_1 , b'_2 , b'_3 respectively. The tool generates the three corresponding rPATL formulae $\langle\langle C \rangle\rangle \mathbf{R}'_{SE=?}[\rho_i]$, for $i \in \{1, 2, 3\}$. This game model is now suitable for the equilibrium analysis that will be, for example, calculated in Chapter 4. Following Algorithm 1, this ADTerm is transformed into the SSG shown in Fig. 3.4.

The SSG in Fig. 3.4 is structured as an extensive form game. This transformation can be used to analyse security strategies, where the actions of the attacker and defender are modelled as a game, with the assumption that the attacker must take at least one action. The next step is how we are *assigning preferences*, sometimes referred to as payoffs. For example, the financial implications are also outlined with the assets valued at £100,000 and the potential maximum gain for the attacker capped at £50,000. Costs associated with specific defensive actions, along with the impact values, and the impact score 5.9. The analysis focuses on evaluating the effectiveness of defensive strategies against cyber threats by assuming that the potential damage from an attack without any defence is a full 100% risk.

The preference can be assigned by analysing risk and adjusting the cost of loss and gain. We utilise a risk based approach [Jaewon Lee et al., 2014] to analysis software security vulnerabilities. The formula involves multiplying the criticality of the vulnerability, which is determined by the CVSS. For example, risk = criticality (likelihood $\times$ vulnerability scores) $\times$ impact. This allows us to evaluate software security vulnerabilities while also considering the cost effectiveness of potential solutions. The preferences will be assigned to the terminal nodes. The following examples will highlight step by step how to assign the preferences for each strategy.

Cost and Impact Values (CVE-2017-8759)

- Defender's Asset Value: £100,000
- Attacker's Maximum Gain: £50,000
- Attacker's Gain Multipliers: (b_1, b_2, b_4): 60%, No Action: 0%
- Patch (b'_1): Estimated Cost £1,000 - £10,000 annually
- Patch (b'_2): Estimated Cost £5,000 - £20,000 annually
- Patch (b'_3): Estimated Cost £10,000 - £50,000 annually
- CVSS Impact Score: 5.9

Assigning Preferences

🎯 **Strategy:** Defender: b'_1, b'_2, b'_3 , Attacker: b_1, b_2, b_3

Defender's Cost: £10,000 + £20,000 + £50,000 = £80,000

Given:

Patch (b'_1): Impact Reduction: 60-90%

Patch (b'_2): Impact Reduction: 50-70%

Patch (b'_3): Impact Reduction in terms of prevention: 0-30%

Calculation Steps:

1. Start with 100% risk.
2. Apply Patch b'_1 (60-90% reduction):
Using the average, let's say 75% reduction.
New risk = 100% - 75% of 100% = 25%
3. Apply Patch b'_2 (50-70% reduction) to the remaining risk:
Using the average, let's say 60% reduction.
New risk = 25% - 60% of 25% = 10%
4. Apply Patch b'_3 (0-30% prevention) to the remaining risk:
Using the average, let's say 15% reduction.
Final risk = 10% - 15% of 10% = 8.5%

Total Impact Reduction Calculation:

Original Risk: 100%

Final Risk: 8.5%

Total Impact Reduction: 100% - 8.5% = 91.5%

The impact score from CVSS will be reduced to $5.9 * (1 - 0.915) = 0.5015$.

Defender's adjusted loss: $(0.5015 / 10) * £100,000 = £5,015$

Attacker's gain: $(0.5015 / 10) * £50,000 * 0.6 = £1,504.5$

👤 Defender's payoff: $-£80,000 - £5,015 = -£85,015$

👤 Attacker's payoff: £1,504.5

The results are summarised in the following Table 3.3, showing the losses for the defender and the gains for the attacker in various scenarios. The example of calculation above will place in the strategy 1 that the defender will lose £85,015 while the attacker will gain £1,504.5 if both of them play this strategy. The remaining strategies are calculated in the same manner.

Table 3.3: Summary of defensive & attacking strategies. Stackelberg equilibrium in bold.

Strategy	Def. Actions	Att. Actions	Def. Loss (£)	Att. Gain (£)
σ_1	$\{b'_1, b'_2, b'_3\}$	$\{b_1, b_2, b_3\}$	-85,015	1,504.5
σ_2	$\{b'_1, b'_2\}$	$\{b_1, b_2, b_3\}$	-35,900	1,770
σ_3	$\{b'_2, b'_3\}$	$\{b_1, b_2, b_3\}$	-76,018	6,018
σ_4	$\{b'_1, b'_3\}$	$\{b_1, b_2, b_3\}$	-72,537.5	3,761.25
σ_5	$\{b'_1\}$	$\{b_1, b_2, b_3\}$	-24,750	4,425
σ_6	$\{b'_2\}$	$\{b_1, b_2, b_3\}$	-43,600	7,080
σ_7	$\{b'_3\}$	$\{b_1, b_2, b_3\}$	-100,150	15,045
σ_8	$\emptyset$	$\{b_1, b_2, b_3\}$	-59,000	17,700

This chapter ends with the SSG model. In the next, it will become the basis for rational verification. For now, the story of CVE-2017-8759 concludes with its final outcome. The next chapter will take us from modelling to verification. The cost of each strategy will be calculated from deployment and maintenance expenses, while the attacker's gain will be estimated based on potential financial damages from successful exploits, including data loss and business disruption. These estimates will be informed by historical data collected by the company. The strategies will be evaluated based on their effectiveness in minimising the defender's loss and the attacker's gain. The Stackelberg equilibrium will be highlighted in bold. Among the evaluated strategies, σ_5 (patch management) will be shown as the optimal choice as it satisfies the Stackelberg equilibrium, resulting in the least loss for the defender and a moderate gain for the attacker. Rational verification with STEVE will return "true" when a security requirement satisfies some of the rPATL formulae and the Stackelberg equilibrium, and "false" when it fails to meet these criteria or does not achieve the Stackelberg equilibrium. The example below shows the abstract idea of how rational verification works.

Rational Verification

Given:

A Stackelberg security game (SSG)

A defender strategy (σ)

F "applyPatch"

A temporal formula that path of the game must satisfy (φ)

$\langle\langle C \rangle\rangle R_{SE=?}^r[F \text{ "applyPatch"}]$

$\langle\langle C \rangle\rangle R_{SE=?}^r[F \text{ "filterEmail"}]$

$\langle\langle C \rangle\rangle R_{SE=?}^r[F \text{ "backup"}]$

Question:

Is there a defender strategy $\sigma \in SSG$ such that the path $\models \varphi$?

Result: True

3.3.1 Case Study 1: Modelling CVE-2024-3400 as an SSG

In late March 2024, a silent weakness opened the firewalls guarding thousands of organisations. Embedded in the GlobalProtect feature of Palo Alto Networks PAN-OS, a single crafted request could hack the boundary and hand over the keys with *root* privileges. At first there were only whispers unusual network traces, failed logins that felt too deliberate. By early April, the whispers became a storm: the flaw, assigned CVE-2024-3400 and rated at the highest severity, was being exploited in the wild under the campaign known as *MidnightEclipse* [National Institute of Standards and Technology (NIST), 2024, Volexity, 2024].

For defenders, every moment was a high risk. Should they rush to disable GlobalProtect, apply fragile workarounds, or wait in hope that a patch would arrive before the attackers struck? Security agencies urged immediate mitigations, including restricting exposure, enabling Threat Prevention signatures, and applying vendor patches as soon as available [Palo Alto Networks, 2024, Cybersecurity and Infrastructure Security Agency (CISA), 2024]. This is exactly the dilemma that SSGs capture: a defender commits to a strategy in the open, knowing the attacker is watching, weighing the odds, and preparing a countermove.

CVE-2024-3400 Detail

Description

A command injection as a result of arbitrary file creation vulnerability in the GlobalProtect feature of Palo Alto Networks PAN-OS software for specific PAN-OS versions and distinct feature configurations may enable an unauthenticated attacker to execute arbitrary code with root privileges on the firewall. Cloud NGFW, Panorama appliances, and Prisma Access are not impacted by this vulnerability.

Metrics

CVSS Version 4.0CVSS Version 3.xCVSS Version 2.0

NVD enrichment efforts reference publicly available information to associate vector strings. CVSS information contributed by other sources is also displayed.

CVSS 3.x Severity and Vector Strings:

 **CNA:** Palo Alto Networks, Inc.

Base Score: 10.0 CRITICAL

Vector: CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H

QUICK INFO

CVE Dictionary Entry: CVE-2024-3400

NVD Published Date: 04/12/2024

NVD Last Modified: 11/29/2024

Source: Palo Alto Networks, Inc.

CVSS v3.1 Severity and Metrics:

Base Score: 10.0 CRITICAL

Vector: AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H

Impact Score: 6.0

Exploitability Score: 3.9

Attack Vector (AV): Network

Attack Complexity (AC): Low

Privileges Required (PR): None

User Interaction (UI): None

Scope (S): Changed

Confidentiality (C): High

Integrity (I): High

Availability (A): High

References to Advisories, Solutions, and Tools

By selecting these links, you will be leaving NIST webspace. We have provided these links to other web sites because they may have information that would be of interest to you. No inferences should be drawn on account of other sites being referenced, or not, from this page. There may be other web sites that are more appropriate for your purpose. NIST does not necessarily endorse the views expressed, or concur with the facts presented on these sites. Further, NIST does not endorse any commercial products that may be mentioned on these sites. Please address comments about this page to nvd@nist.gov.

Figure 3.5: CVE-2024-3400

CVE-2024-3400 is a command injection vulnerability in the GlobalProtect feature of Palo Alto Networks PAN-OS (versions 10.2, 11.0, and 11.1). According to the NVD entry as shown in Fig. 3.5, the vulnerability has been assigned the maximum CVSS v3.1 base score of 10.0 (*Critical*), reflecting its ease of exploitation and severe impact.

Table 3.4: CVE-2024-3400, a critical command injection vulnerability in Palo Alto Networks.

Attribute	Value
CVE	CVE-2024-3400
CVE Description	A command injection vulnerability in the GlobalProtect feature of Palo Alto Networks PAN-OS (versions 10.2, 11.0, 11.1) when a GlobalProtect gateway or portal is configured. An unauthenticated attacker can execute arbitrary commands as root on the firewall.
Attack Sequences	The attack requires two key steps: - Command Injection (b_1): attacker sends a crafted HTTP(S) request targeting the GlobalProtect interface. - Improper Input Validation (b_2): lack of sanitisation allows arbitrary command execution, leading to file creation in telemetry paths and execution with root privileges. Post exploitation activity includes deployment of the UPSTYLE Python backdoor, persistence via cron jobs, credential and configuration exfiltration, and interactive shell access (Operation MidnightEclipse).
Mitigations and Controls	We consider three primary mitigation options: - Patch management (b'_1): upgrade PAN-OS to fixed releases (10.2.9-h1, 11.0.4-h1, 11.1.2-h3, and later). - Threat prevention (b'_2): enable Threat Prevention signatures 95187, 95189, and 95191 on GlobalProtect interfaces to block exploit attempts. - Configuration hardening (b'_3): disable Device Telemetry to prevent malicious use of telemetry paths for payload execution. Note: Disabling telemetry is no longer an effective as updated by the vendor.
Characteristics	• Authentication: Not Required • Availability Impact: Critical (Remote, No User Interaction) • Classification: Command Injection via Improper Input Validation • Credibility: Vendor Confirmed, Exploited in the Wild • Ease: Exploit Available (Metasploit module released) • Application Type: PAN-OS, GlobalProtect Feature • Vendors: Palo Alto Networks • LocalCode: No, RemoteCode: Yes (root) • CWE: CWE-77 (Command Injection), CWE-20 (Improper Input Validation) • CVSS: - CVSS3.1 Vector: AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H - Score: 10.0 (Critical)

Table 3.4 presents the vulnerability profile of CVE-2024-3400 contained in the dataset, which provides input to the rational verification tool (StEVE). The attack vector is network based and requires no authentication, allowing a remote adversary to execute arbitrary code with *root* privileges. Exploitation enables full system compromise, persistence through backdoors, and potential lateral movement. Exploit modules with Metasploit [Kennedy et al., 2011, Rapid7, LLC, 2025] are available for CVE-2024-3400 [Tenable, Inc., 2024, Rapid7, 2025]. Mitigations recommended by the vendor include applying immediate workarounds, enabling Threat Prevention signatures on the GlobalProtect interface, and upgrading to patched PAN-OS releases. The profile and its associated metrics are transformed into an ADT and then into an SSG model.

```

prism — test.sh CVE-2024-3400 — 86x30

EXTRACTING A CVE AND A CVSS INTO AN ADT(1)...

ID: CVE-2024-3400

Description: A command injection as a result of arbitrary file creation vulnerability in the Glo
balProtect feature of Palo Alto Networks PAN-OS software for specific PAN-OS versions and distin
ct feature configurations may enable an unauthenticated attacker to execute arbitrary code with
root privileges on the firewall.

Cloud NGFW, Panorama appliances, and Prisma Access are not impacted by this vulnerability.

Impact Score: 6

Solutions:
b'1: Upgrading PAN-OS to 10.2.9-h1, 11.0.4-h1, 11.1.2-h3, and in all later PAN-OS versions
b'2: Using Threat Prevention to Block Attacks with Threat IDs 95187, 95189, and 95191
b'3: Disabling Device Telemetry

Exploits:
b1: Command Injection
b2: Improper Input Validation

-----
Enter the defender actions (use '&' for conjunctive, '|' for disjunctive):
Default: b'1|b'2 (demo version)
Enter your choice or press Enter to use default: █

```

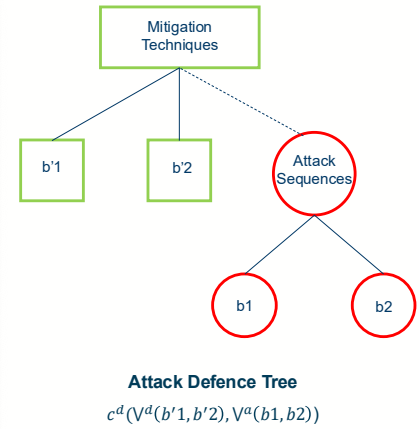


Figure 3.6: Extracting a CVE into an ADT

1. Extracting a CVE into an ADT

Defender's Actions

- Upgrading PAN-OS to 10.2.9-h1, 11.0.4-h1, 11.1.2-h3, and in all later PAN-OS versions (b'_1)
- Using Threat Prevention to block attacks with threat IDs 95187, 95189, and 95191 (b'_2)
- Disabling Device Telemetry (b'_3)

Attacker's Actions

- Command Injection (b_1)
- Improper Input Validation (b_2)

The tool automatically extracts this information, as illustrated on the left side of Fig. 3.6, where the CVE is transformed into an ADT. Users enter the defender actions (use '&' for conjunctive, '|' for disjunctive). In this case, we enter the $b'_1|b'_2$ for the defender and $b_1|b_2$ for the attacker. Note that due to the defender action b'_3 disabling telemetry is no longer as effective as updated by

the vendor, we omit them for the experiment. As a result, the right side of the figure shows the resulting ADT, both as a graphical structure and as the corresponding ADTerms, expressed as $c^d(\vee_d(b'_1, b'_2), \vee_a(b_1, b_2))$.

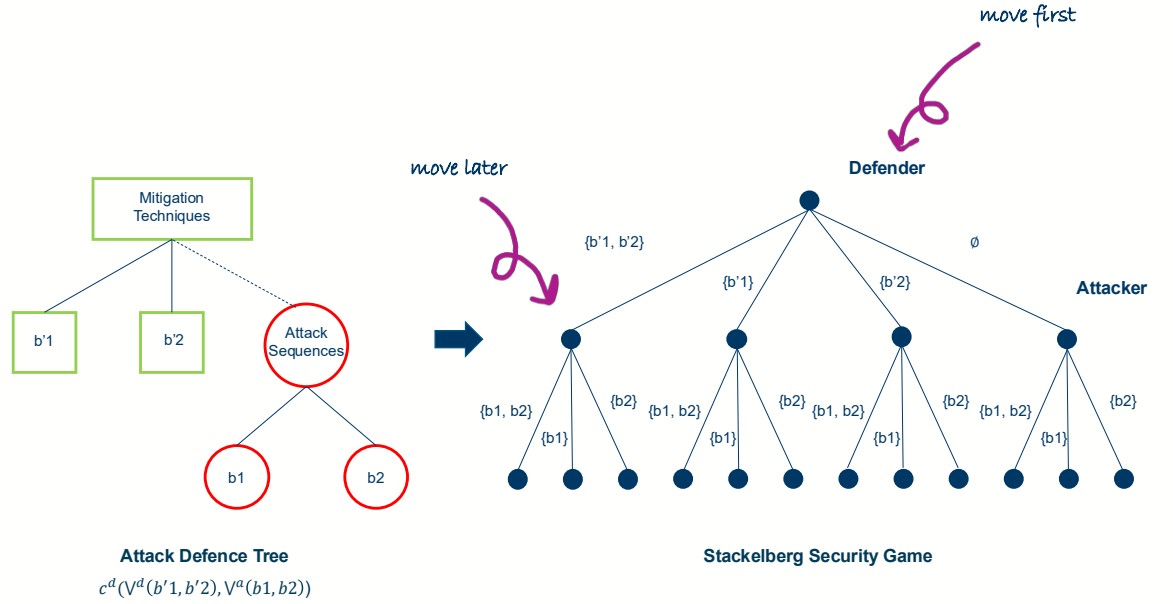


Figure 3.7: Transforming an ADT to SSGs

2. Transforming an ADT to an SSG

The defender takes the following disjunctive actions: b'_1, b'_2 . The attacker uses the following disjunctive actions: b_1, b_2 . The ADTerm encoding this scenario is $c^d(\vee_d(b'_1, b'_2), \vee_a(b_1, b_2))$ and graphically represented in Fig. 3.7, which is then transformed into an SSG with payoffs derived from adjustment costs. Following Algorithm 1, each of the defender's strategies $\sigma_1 \in 2^{\{b'_1, b'_2\}}$ is verified against the security requirements $\rho_1 = \text{F upgradePanOS}$ and $\rho_2 = \text{F threatPrevention}$ corresponding to the actions b'_1, b'_2 respectively. Then the tool generates the two corresponding rPATL formulae $\langle\langle C \rangle\rangle R_{SE=?}^r[\rho_i]$, for $i \in \{1, 2\}$. On the other hand, each of the attacker's strategies $\sigma_2 \in 2^{\{b_1, b_2\}}$ is obtained from the disjunctive actions b_1 and b_2 in the ADTerm. The combined behaviours of the defender and attacker can be enumerated as follows:

- $\{b'_1, b'_2\} \rightarrow \{b_1, b_2\}$
- $\{b'_1, b'_2\} \rightarrow \{b_1\}$
- $\{b'_1, b'_2\} \rightarrow \{b_2\}$
- $\{b'_1\} \rightarrow \{b_1, b_2\}$
- $\{b'_1\} \rightarrow \{b_1\}$
- $\{b'_1\} \rightarrow \{b_2\}$
- $\{b'_2\} \rightarrow \{b_1, b_2\}$
- $\{b'_2\} \rightarrow \{b_1\}$
- $\{b'_2\} \rightarrow \{b_2\}$
- $\emptyset \rightarrow \{b_1, b_2\}$
- $\emptyset \rightarrow \{b_1\}$
- $\emptyset \rightarrow \{b_2\}$

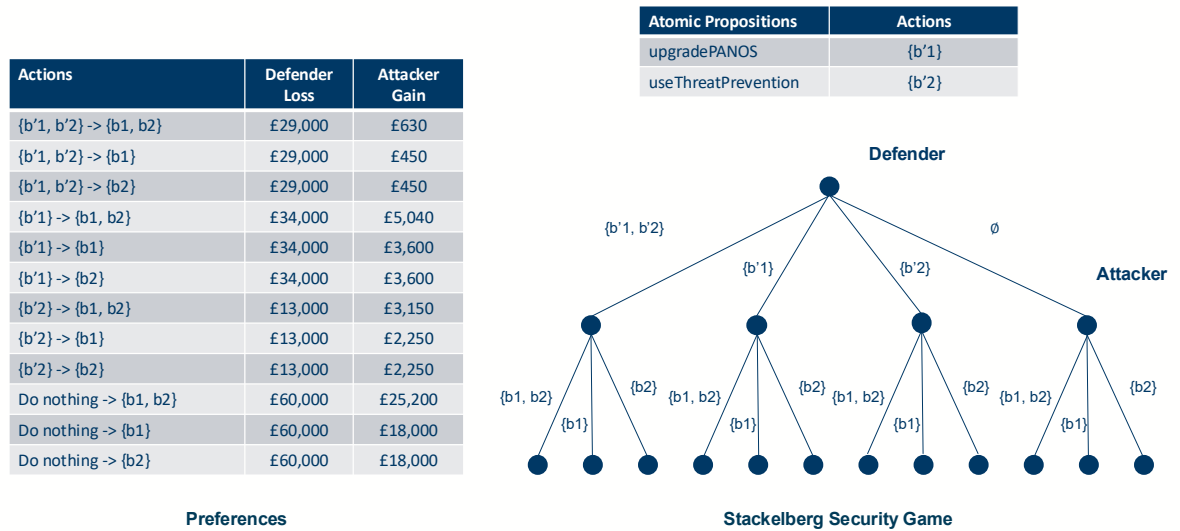


Figure 3.8: Assigning Preferences and Atomic Propositions

Fig. 3.8 demonstrates the process of assigning preferences. Furthermore, on the top right show that atomic propositions are labelled for each action, with `upgradePanOS` corresponding to b'_1 and `threatPrevention` corresponding to b'_2 . In practice, preferences can be derived either from tables provided by the organisation or through an analysis of the associated costs and effects. The following section demonstrates how these values are obtained in this experiment.

Initial Cost and Impact Values

- Defender's Asset Value: £100,000
- Attacker's Maximum Gain: £50,000
- Attacker's Gain Multipliers: (b_1, b_2) : 84%, (b_1) : 60%, (b_2) : 60% No Action: 0%
- Patch (b'_1): Estimated Cost £4,500 - £22,000 annually, Impact Reduction: 70-90%
- Patch (b'_2): Estimated Cost £800 - £5,500 annually, Impact Reduction: 80-95%
- CVSS Score: 6

Fig. 3.9 depicts the SSG for CVE-2024-3400. For example, if the defender applies threat prevention (b'_2), the attacker observes this commitment and may respond by launching a command injection (b_1), exploiting improper input validation (b_2), or attempting both actions in combination. Full details of the cost estimation and preferences are provided in the Appendix.

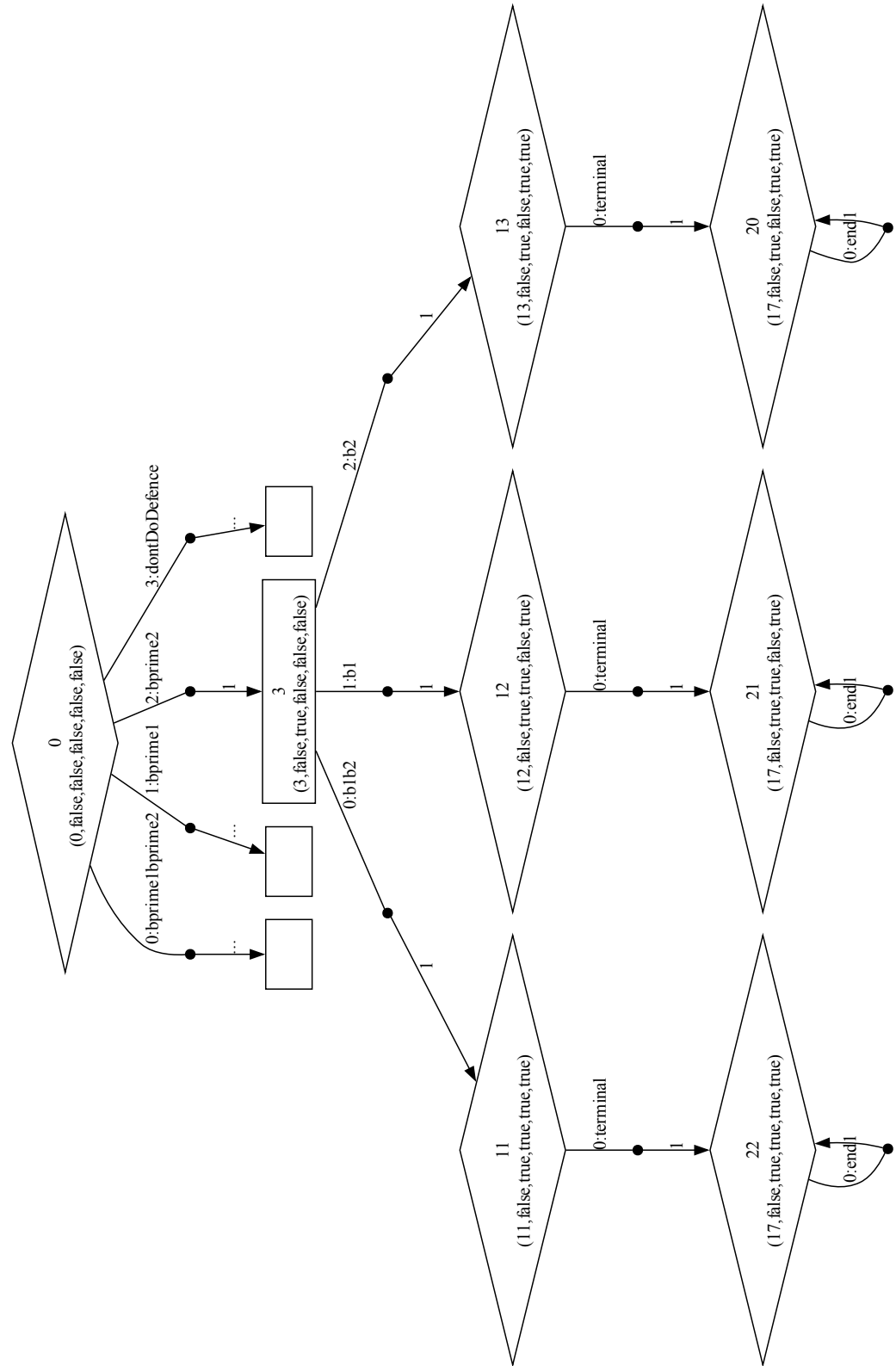


Figure 3.9: Graphical representation of the SSG for the CVE-2024-3400 example, as generated from the PRISM-games extension in STEVE. In this diagram, diamonds represent defenders and rectangles represent attackers. The strategy relies on the Stackelberg equilibrium, where the defender applies only threat prevention (action b'_2), which will cost the defender £13,000 and the attacker earns £3,150. Some states are omitted from the diagram; these are indicated by “...”.

3.4 Repeated Stackelberg Security Games (RSSGs)

An RSSG expands the one-time SSG $(\mathcal{G}, AP, L)$ by enabling the stage game $\mathcal{G}$ to be played over a limited number of T rounds. While the initial extensive form game is structured over nodes $H \cup Z$, the repeated game requires a more comprehensive concept of state. Therefore, a derived state space S is defined, where each state $s_t \in S$ encapsulates details at round t , including previous actions and system conditions. Unlike nodes in $H \cup Z$, which signify individual decision points or results, states in S encapsulate the developing game history and facilitate strategic adaptation analysis. The RSSG framework is stateful unlike repeated one-shot SSGs. This means that the optimal strategy at round t depends on the current state s_t , reflecting past interactions. This adaptive defence against evolving attacks is where static strategies are insufficient.

Round by Round Stackelberg Equilibrium. In each round $t \in \{1, \dots, T\}$ of an RSSG, the defender (leader) selects an action a_1^t according to the current state s_t , which reflects the game history. The attacker (follower) then observes a_1^t and responds with an action a_2^t that maximises their utility. We consider the case where the attacker always selects a best response in every round, given the defender's action and the current state. To reason about utility in a repeated setting, we extend the notion from one-shot games. In the extensive form model, the utility function $u_i : Z \rightarrow \mathbb{R}_{\geq 0}$ assigns payoffs to terminal nodes. We define $U_i(s_t, a_1^t, a_2^t)$ as the expected cumulative utility for player i starting from state s_t , given the actions a_1^t and a_2^t taken in round t , and assuming equilibrium strategies are followed thereafter. This generalises the terminal node utility to account for accumulated outcomes across multiple rounds. We also define a reward function $R_i : S \times A_1 \times A_2 \rightarrow \mathbb{R}_{\geq 0}$, where $R_i(s_t, a_1^t, a_2^t)$ gives the immediate reward received by player i in round t , based on the current state s_t , the defender's action a_1^t , and the attacker's action a_2^t . These per round rewards form the basis for computing expected cumulative utility. To model temporal preferences, we adopt the notion of discounted utility. Given a discount factor $\beta \in [0, 1]$, the total discounted utility for player i over horizon T is:

$$R_i^T = \sum_{t=1}^T \beta^{t-1} R_i(s_t, a_1^t, a_2^t)$$

This prioritises earlier rewards when $\beta < 1$, which reflects practical urgency in security contexts. Formally, a strategy profile $\sigma = (\sigma_1, \sigma_2)$, where $\sigma_i = (\sigma_i^1, \dots, \sigma_i^T)$ dictates the actions a_i^t chosen by player i in round t , constitutes a *round by round Stackelberg equilibrium* if for every round $t \in \{1, \dots, T\}$:

1. The attacker's action a_2^t maximises their expected utility, given the defender's action a_1^t and the current state s_t :

$$a_2^t \in \arg \max_{a \in A_2(s_t)} \mathbb{E}[U_2(s_t, a_1^t, a)]$$

where $A_2(s_t)$ are the attacker's available actions in state s_t , and U_2 represents the attacker's expected utility function which might consider immediate reward $R_2(s_t, a_1^t, a)$ or future discounted rewards.

2. The defender's action a_1^t is chosen as part of a strategy σ_1 that maximises their total expected discounted utility over the horizon T , assuming the attacker plays a best response strategy $\sigma_2^* \in BR(\sigma_1)$, where $BR(\sigma_1)$ denotes the set of best responses to σ_1 . Formally:

$$\sigma_1 \in \arg \max_{\sigma_1'} \mathbb{E}_{\sigma_1', \sigma_2^*} [R_1^T] \quad \text{where} \quad R_1^T = \sum_{t=1}^T \beta^{t-1} R_1(s_t, a_1^t, a_2^t)$$

Here, $R_1(s_t, a_1^t, a_2^t)$ is the immediate reward for the defender in round t ; (s_t, a_1^t, a_2^t) belongs to the sequence of states and actions resulting from the strategy profile (σ_1', σ_2^*) , where σ_1' is a candidate defender strategy and $\sigma_2^* \in BR(\sigma_1')$; and $\beta \in [0, 1]$ is the discount factor.

This definition means the defender commits to a sequence of actions that is optimal over the horizon T , under the constraint that the attacker will rationally counter each move in every round.

The Role of Discounting (β). The discount factor $\beta \in [0, 1]$, captures how the defender values rewards over time. When $\beta < 1$, an *exponential decay* is applied, giving higher weight to immediate outcomes; future rewards are reduced by the factor β^{t-1} , capturing urgency, uncertainty, or the time value of defensive actions. The expression β^{t-1} means that a reward received in round t is multiplied by β^{t-1} , so rewards later in time are given less weight. This is typical in cybersecurity where addressing threats promptly is critical. Another option is *no discounting* for $\beta = 1$, when all rounds are valued equally. In this case the total payoff for each player i is the sum of rewards $R_i^T = \sum_{t=1}^T R_i(s_t, a_1^t, a_2^t)$. It is suitable for objectives focused on average performance over the fixed horizon T .

Computational Verification and Stability. Determining whether a strategy profile is a round by round Stackelberg equilibrium, or synthesising the optimal defender strategy, usually relies on methods such as backward induction or value iteration [Leyton-Brown and Shoham, 2008] adapted for finite repeated Stackelberg games. These techniques compute the expected cumulative discounted rewards and determine the optimal choice at each decision point.

3.4.1 Strategy Synthesis

Our framework supports both the reasoning and synthesis of defender strategies in the RSSG setting. The objective is to derive a defender strategy σ_1 that maximises the total expected discounted utility R_1^T under the round by round assumption. Table 3.5 summarises the notation used for defender and attacker strategies in the repeated game.

Static strategy

The *optimal static strategy* corresponds to finding a single strategy $\mu_1 : S \rightarrow \Delta(A_1)$, where μ_1 maps each state $s \in S$ to a probability distribution over defender actions. Here, S is the set of states in the repeated game, and $\Delta(A_1)$ denotes the set of probability distributions over the defender's action set A_1 . The same strategy μ_1 is applied in every round $t = 1, \dots, T$, and the defender's expected utility R_1^T is evaluated assuming that, in each round, the attacker selects a best response to the action chosen according to $\mu_1(s_t)$. While the strategy itself is fixed, its performance reflects the full sequence of interactions.

Dynamic strategy

The *optimal dynamic strategy* corresponds to finding an optimal sequence of actions $\sigma_1 = (\sigma_1^1, \dots, \sigma_1^T)$, where the action σ_1^t applied in round t is determined by optimising the remaining discounted utility from round t onwards, depending on the round t and the current state s_t .

Table 3.5: Notation summary for strategy types in RSSGs.

Symbol	Description
Σ_1, Σ_2	Strategy sets for the defender (1) and attacker (2)
$\sigma_1 \in \Sigma_1$	Defender's strategy across all T rounds
σ_1^t	Defender's action (or decision rule) in round t
$\mu_1 : S \rightarrow \Delta(A_1)$	Static Strategy: maps states to action distributions, reused each round
$\sigma_2^* \in BR(\sigma_1)$	Best response strategy of the attacker to σ_1

Computational complexity. The model checking problem for rPATL over stochastic multiplayer games is in $\text{NP} \cap \text{coNP}$ for simple bounded until properties and in $\text{NEXP} \cap \text{coNEXP}$ for the general case [Chen et al., 2013]. Our extension introducing the Stackelberg equilibrium operator does not increase this upper bound, since the additional leader follower reasoning is achieved through an iterative evaluation of best response sets. However, in practical settings with finite horizons and restricted strategy spaces, the computation time empirically scales polynomially with the number of reachable states under a given strategy and linearly with the number of rounds in repeated SSGs.

3.4.2 Case Study 2: Modelling CVE-2024-4947 as an RSSG

In May 2024, a vulnerability spread silently through the web. Hidden inside Chrome, a flaw in type handling allowed a single malicious page to take control of a browser session. To the

victim, it looked like an ordinary click: a link in an email, an ad on a familiar site, a page that loaded just a little too slowly. The exploit was already working, corrupting memory and preparing to run foreign code inside the sandbox. Google rushed to patch it after active exploitation was confirmed in the wild [NVD, 2024]. For defenders, the situation carried an unsettling weight. Unlike server side flaws this attack required no direct compromise of corporate networks. Should they rely on patch management, trusting every machine would be updated in time? Or should they double down on browser isolation, content filtering, and monitoring for exploitation? The decisions reflected the trade offs common in repeated: attackers adjust their tactics, defenders adapt their filters, and each side reconsiders their strategy round after round.

🚩 CVE-2024-4947 Detail

Description

Type Confusion in V8 in Google Chrome prior to 125.0.6422.60 allowed a remote attacker to execute arbitrary code inside a sandbox via a crafted HTML page. (Chromium security severity: High)

Metrics

CVSS Version 4.0CVSS Version 3.xCVSS Version 2.0

NVD enrichment efforts reference publicly available information to associate vector strings. CVSS information contributed by other sources is also displayed.

CVSS 3.x Severity and Vector Strings:

 NIST: NVD	Base Score: 9.6 CRITICAL	Vector: CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:H/I:H/A:H
ADP: CISA-ADP	Base Score: 9.6 CRITICAL	Vector: CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:H/I:H/A:H

QUICK INFO

CVE Dictionary Entry:
[CVE-2024-4947](#)

NVD Published Date:
05/15/2024

NVD Last Modified:
11/27/2024

Source:
Chrome

CVSS v3.1 Severity and Metrics:
Base Score: 9.6 CRITICAL
Vector: AV:N/AC:L/PR:N/UI:R/S:C/C:H/I:H/A:H
Impact Score: 6.0
Exploitability Score: 2.8

Attack Vector (AV): Network
Attack Complexity (AC): Low
Privileges Required (PR): None
User Interaction (UI): Required
Scope (S): Changed
Confidentiality (C): High
Integrity (I): High
Availability (A): High

References to Advisories, Solutions, and Tools

By selecting these links, you will be leaving NIST webspace. We have provided these links to other web sites because they may have information that would be of interest to you. No inferences should be drawn on account of other sites being referenced, or not, from this page. There may be other web sites that are more appropriate for your purpose. NIST does not necessarily endorse the views expressed, or concur with the facts presented on these sites. Further, NIST does not endorse any commercial products that may be mentioned on these sites. Please address comments about this page to nvd@nist.gov.

Figure 3.10: CVE-2024-4947

Fig. 3.10 presents CVE-2024-4947, a type confusion vulnerability in Chrome’s V8 JavaScript engine that Enables remote code execution via crafted HTML content. The flaw has been assigned a CVSS v3.1 base score of 9.6 (*Critical*), reflecting its ease of exploitation and severe impact. This case presents the dynamics of an attacker returning with new phishing tactics and exploit chains. Table 3.6 summarises key attributes including its description, common attack sequences, potential defender mitigations, and technical characteristics relevant for threat modelling and constructing the RSSG model used in this thesis. Information collated from sources such as the National Vulnerability Database (NVD), vendor advisories such as Google Chrome Releases, and public security reporting on its exploitation. The structure follows a template adapted from Kuppa et al. [Kuppa et al., 2021]. This CVE was selected due to its critical severity, documented

Table 3.6: CVE-2024-4947, a type confusion vulnerability in Google Chrome.

Attribute	Value
CVE	CVE-2024-4947
CVE Description	A type confusion flaw in the V8 JavaScript engine in Google Chrome prior to version 125.0.6422.60. Exploitation via crafted HTML can result in arbitrary code execution within the browser. The issue was exploited as a zero day in May 2024, actively exploited, and included in CISA's KEV catalogue.
Attack Sequences	The attacker must complete the following steps: - AttemptPhishing (b_1): lure the victim to visit a maliciously crafted website. - AttemptExploitV8 (b_2): trigger the V8 type confusion vulnerability when the page loads. - ExecutePayload (b_3): run arbitrary code inside the browser context, potentially achieving full compromise with a sandbox escape.
Mitigations and Controls	The defender may apply these countermeasures: - PatchChrome (b'_1): update Chrome to version 125.0.6422.60 or later. - BlockMaliciousSites (b'_2): restrict or block access to domains known to host exploits. - EnableEnhancedMonitoring (b'_3): deploy script and memory monitoring to detect anomalous activity targeting V8.
Characteristics	• Authentication: Not Required • Impact: Critical (Remote, User Interaction Required) • Classification: Type Confusion (Memory Corruption) • Credibility: Vendor Confirmed, Exploited in the Wild • Ease: Exploit Available (Zero-day, Targeted Use) • Application Type: Browser, JavaScript Engine • Vendors: Google (Chromium) • LocalCode: No, RemoteCode: Yes • CWE: CWE-843 (Access of Resource Using Incompatible Type) • CVSS: - CVSS3.1 Vector: AV:N/AC:L/PR:N/UI:R/S:C/C:H/I:H/A:H - Score: 9.6 (Critical) • CPE: - cpe:2.3:a:google:chrome:<125.0.6422.60:*:*:*:*:*:* - cpe:2.3:a:chromium:chromium:*:*:*:*:*:*

exploitation in the part of CISA KEV, and multistep characteristics suitable for modelling adaptive attacker defender interactions over repeated rounds.

In the ADT, the defender may apply mitigation actions such as updating Chrome to version 125.0.6422.60 (`PatchChrome`), blocking known malicious (`BlockMaliciousSites`), or enabling enhanced script monitoring (`EnableEnhancedMonitoring`); we denote these actions by b'_1 , b'_2 , and b'_3 , respectively. The attacker must complete a multi-step process: first, luring the victim to a malicious site (`AttemptPhishing`, b_1); second, exploiting the V8 vulnerability upon visit (`AttemptExploitV8`, b_2); and finally, executing a malicious payload to achieve code execution (`ExecutePayload`, b_3).

This scenario is expressed as the ADTerm $c^d(\vee_d(b'_1, b'_2, b'_3), \wedge_a(b_1, b_2, b_3))$, where the attacker must complete all three steps – phishing, exploiting, and payload execution – to compromise the system, while the defender can prevent the attack by applying any one of the mitigation actions. Although $\wedge_a$ typically denotes conjunction without sequence, in this case the steps are inherently sequential due to technical dependencies.

In our RSSG model, these actions are mapped directly from the ADT: defender actions `PatchChrome`, `BlockMaliciousSites`, and `EnableEnhancedMonitoring` map to b'_1, b'_2, b'_3 , with an additional `DoNothing` option. On the other hand, the attacker actions comprise `AttemptPhishing`, `AttemptExploitV8`, and `ExecutePayload`, which map to b_1, b_2, b_3 , along with `Wait` as a passive choice. This alignment ensures consistency between the threat model and the repeated game as shown in Table 3.7.

Table 3.7: States and Actions

Category	Details / Variables
States	Key states capture security posture and attacker progress: <code>Initial</code> , <code>Vulnerable_Unpatched</code> , <code>Attacker_Phishing</code> , <code>Attacker_Exploiting</code> , <code>Compromised</code> , <code>Patched</code> , <code>SiteBlocked</code> , <code>EnhancedMonitoringActive</code> , <code>Game_Over</code> (terminal).
Defender Actions (A_1)	Available actions for the defender: <code>DoNothing</code> , <code>PatchChrome</code> , <code>BlockMaliciousSites</code> , <code>EnableEnhancedMonitoring</code> .
Attacker Actions (A_2)	Available actions for the rational attacker (follower): <code>Wait</code> , <code>AttemptPhishing</code> , <code>AttemptExploitV8</code> , <code>ExecutePayload</code> .

The ADT model is translated into the RSSG. The graphical model can be seen at the repository⁵. The defender and attacker actions identified in the ADT such as `PatchChrome` and `AttemptExploitV8` directly define the action sets used in the game. The logical structure of the ADT, which dictates the necessary steps for a successful compromise, informs the state space such as `Vulnerable_Unpatched`, `Attacker_Exploiting`, `Compromised`, and the transition probabilities between them. Finally, the security outcomes are quantified as reward (or cost) structures. For example, a successful attack corresponds to a significant negative effect for the defender (`incident_cost` in Table 3.8), while defender actions have their own associated costs (`defence_cost`). This stateful, adaptive model, along with its associated transition

⁵<https://zenodo.org/records/13338608>

Table 3.8: Transition Probabilities and Payoffs

Element	Value / Description
Transition Probabilities	• Patch Success Probability: 0.98 per round (if <code>PatchChrome</code> chosen). • Phishing Success Probability: 0.4 (if <code>Vulnerable_Unpatched</code> and attacker chooses <code>AttemptPhishing</code>). • Exploit Success Probability: 0.75 (if phishing successful and attacker chooses <code>AttemptExploitV8</code>). • Site Blocking Effectiveness (prevents phishing): 0.9 (if <code>BlockMaliciousSites</code> chosen). • Compromise occurs if <code>ExecutePayload</code> follows successful exploitation. • <code>Game_Over</code> reached after $T = 10$ rounds or upon compromise.
Defender Rewards/Costs	Defined via reward structures: • “incident_cost”: -200 (upon reaching <code>Compromised</code>). • “defence_cost” (per round): <code>PatchChrome</code> = -5, <code>BlockMaliciousSites</code> = -3, <code>EnableEnhancedMonitoring</code> = -2. • “total_cost”: Sum of “incident_cost” and accumulated “defence_cost”. • “steps”: Accumulates 1 per round.
Attacker Utility	Defined via “attacker_utility” structure: • Success: +100 (upon reaching <code>Compromised</code>). • Failed Attempt: -1 (for failed <code>Phishing</code> or <code>Exploit</code>).

probabilities and payoffs, is the formal object that we will analyse in Chapter 4 and evaluate in Chapter 5 to demonstrate the quantitative value of dynamic defence.

In the following chapter, we introduce rational verification and show how the *Stackelberg equilibrium semantics* extend the rPATL reward operator to evaluate quantitative properties under Stackelberg equilibrium. Let $\mathcal{G}_C = \langle \{1, 2\}, S, A, (S_1, S_2), \Delta, AP, \chi \rangle$ be the coalition game induced by C , and let $\text{rew}(r, \mathbf{c}, \text{Sat}(\phi))$ denote the cumulative reward until a state satisfying ϕ is reached, following the standard rPATL definition [Chen et al., 2013] and its implementation in our tool [Phetmanee et al., 2025]. The formula

$$\langle\langle C \rangle\rangle R_{SE=?}^r [F\phi]$$

returns the expected cumulative reward for the coalition C until ϕ holds, assuming Stackelberg play. We define:

$$\begin{aligned}
 \langle\langle C \rangle\rangle R_{SE=?}^r [F\phi] &\stackrel{\text{def}}{=} \mathbb{E}_{\mathcal{G}_{C,S}}^{\text{SE}} [\text{rew}(r, \mathbf{c}, \text{Sat}(\phi))] \\
 &= \sup_{\sigma_1 \in \Sigma_1} \sup_{\sigma_2 \in BR(\sigma_1)} \mathbb{E}_{\mathcal{G}_{C,S}}^{\sigma_1, \sigma_2} [\text{rew}(r, \mathbf{c}, \text{Sat}(\phi))],
 \end{aligned} \tag{3.1}$$

where $BR(\sigma_1)$ denotes the set of the follower’s best response strategies to the leader’s strategy σ_1 . This corresponds to the Stackelberg equilibrium assumption, in which the follower breaks ties in favour of the leader. Therefore, the standard rPATL reward operator $R_{=?}^r$ becomes a Stackelberg, computed by nesting the leader’s optimisation and the follower’s best response procedure. The operator evaluates the expected reward under Stackelberg rationality: the defender (leader)

commits to a strategy first, and the attacker (follower) responds optimally, yielding an equilibrium outcome consistent with the semantics of Eq. 4 in this thesis.

3.5 Summary

This chapter has introduced the methodology for bridging raw vulnerability data with game models. We began with unstructured sources such as CVEs and showed how attack sequences and defensive measures can be systematically expressed as ADTs. These structures provide the foundation for models, their transformation into SSGs, and their RSSGs.

Through CVE-2017-8759, we demonstrated the transformation pipeline step by step, showing how descriptive technical details such as malicious document delivery and patch management, map into conjunctive and disjunctive actions in ADTs and then into strategies with payoffs. This highlighted how costs, risks, and impact reductions can be expressed and used to calculate equilibrium outcomes. We then explored CVE-2024-3400 as the first case study, demonstrating how an SSG captures the critical decision faced by defenders when patches, signatures, or configurations must be applied under immediate threat. The analysis highlighted how Stackelberg games are unbalanced with the defender committing first and the attacker adjusting in response. Finally, we modelled CVE-2024-4947 as an RSSG, moving beyond the single-shot setting. This case illustrated how repeated interactions change the dynamics: attackers return with phishing and exploit attempts, while defenders must weigh the long term costs of patching, blocking, and monitoring. We captured the adaptive round by round nature of the game, incorporating states, transition probabilities, and discounted utilities.

Together, these examples show how our methodology supports both static and dynamic analysis of threats. In the next chapter, these models become the input to the rational verification framework where properties expressed in rPATL are checked against Stackelberg equilibrium strategies, allowing us to evaluate defender commitments in both one-shot and repeated settings. Although the evaluation in this thesis focuses on three representative vulnerabilities, the transformation pipeline is designed to be general. Because the ADT to SSG translation operates on structural patterns rather than CVE specific content, the same procedure can be applied to any vulnerability that exposes attack defence relationships captured in standard CVE or CWE descriptions. Future work will extend this validation across a broader dataset to quantify modelling consistency and performance scalability.

Chapter 4

Rational Verification of Security Games

Rational verification [Wooldridge et al., 2016] is a formal verification method used to check whether a property ϕ , written in rPATL [Chen et al., 2013], is satisfied under the assumption that the participating agents behave rationally. Here, rationality means that each agent selects strategies that optimise its own expected utility, while considering how the choices of other agents may affect the outcome.

In this thesis, we make use of an extended version of rPATL, introduced in our prior work on security games [Phetmanee et al., 2025, Phetmanee et al., 2026], which incorporates Stackelberg reasoning. This extension adapts the logic so that properties can be evaluated in settings where a defender commits to a strategy first and an attacker subsequently adapts with a rational best response. Formally, given a game $\mathcal{G}$ and an rPATL property ϕ , rational verification checks whether ϕ holds when players follow some or all equilibrium strategies in $\mathcal{G}$, such as Stackelberg equilibria. We verify the ongoing optimality of a defender’s strategy under the Stackelberg assumption and expressed it in rPATL as the formula:

$$\langle\langle C \rangle\rangle R_{SE=?}^r [F\phi] \stackrel{def}{=} \sup_{\sigma_1 \in \Sigma_1} \sup_{\sigma_2 \in BR(\sigma_1)} \mathbb{E}^{\sigma_1, \sigma_2} [F\phi] \quad (4.1)$$

Here, $\langle\langle C \rangle\rangle$ refers to the coalition of the two players, in our case, the defender and the attacker; $R_{SE=?}^r$ is the reward operator under the Stackelberg equilibrium; F is the temporal operator specifying a reachability goal; ϕ denotes a desirable system property, for example *the system not being compromised*.

The formula therefore represents the expected reward accumulated by the defender until a state satisfying ϕ is reached, under the assumption that the attacker reacts strategically. Here, σ_1 denotes the strategy chosen by the defender, while σ_2 corresponds to the attacker’s rational best response. The defender needs to assess whether their strategies remain effective when facing adaptive and strategic attackers.

Subgame Perfect versus Stackelberg Equilibrium. PRISM-games natively supports the computation of subgame-perfect Nash equilibria (SPNE) in turn-based stochastic multiplayer games, where equilibrium strategies are computed by backward induction over all subgames. While this notion guarantees rational behaviour at every state, it assumes that all players optimise simultaneously with mutual knowledge of each other's strategies. In contrast, many security domains exhibit a clear sequential hierarchy in which the defender first commits to a protection policy and the attacker subsequently chooses a best response action. The Stackelberg equilibrium explicitly encodes this asymmetry: the leader fixes a strategy σ_1^* before play begins, and the follower selects an optimal reaction $\sigma_2^* \in BR(\sigma_1^*)$ after observing the leader's commitment. Although both SPNE and Stackelberg equilibrium can be represented within turn-based semantics, the Stackelberg setting introduces an additional verification problem: to prove that the defender's committed strategy remains optimal when the follower's rational adaptation is explicitly modelled. Therefore, the Stackelberg operator extends the rPATL reward semantics to reason quantitative properties under this hierarchical form of rationality.

4.1 Verifying Stackelberg Equilibria

Through rational verification we can determine whether temporal specifications hold, given that agents conform to an equilibrium concept. In this work we adopt rPATL syntax [Chen et al., 2013], under the assumption that the defender aims to maximise expected discounted payoff across T rounds, while the attacker chooses optimal counter-strategies in response. rPATL formulae include state formulae (ϕ), path formulae (ψ), and reward path formulae (ρ), but their semantics are interpreted under the round by round Stackelberg equilibrium assumption. Temporal operators relevant to our analysis include both reward and probability expressed in rPATL.

- The operator $\langle\langle C \rangle\rangle R_{SE=?}^r[\rho]$ denotes the expected cumulative reward that coalition C can guarantee under Stackelberg equilibrium, for a given reward structure r and reward path formula ρ . In our setting, the coalition is always $C = \{1\}$, representing the defender (player 1). The formula ρ typically takes the form $F\phi$, meaning that the reward is accumulated along a path until a state satisfying ϕ is reached.
- The operator $\langle\langle C \rangle\rangle P_{\sim p}[\psi]$ expresses whether the defender can ensure that the probability of satisfying path formula ψ meets a bound $\sim p$, where $\sim \in \{<, \leq, \geq, >\}$. For example, $P_{\geq 0.9}$ asks whether the probability is at least 90%. Valid path formulae ψ include temporal operators such as $X\phi$ (next state), or $\phi_1 U^{\leq T} \phi_2$ (bounded until within T steps). The query $P_{=?}[\psi]$ computes the optimal probability the defender can guarantee.

In our framework rPATL properties are evaluated under the semantics of Stackelberg equilibrium. This means that the defender is assumed to choose a strategy that optimises their expected payoff,

Table 4.1: Security metrics expressed in rPATL for rational verification in RSSGs, showing their mapping to properties under Stackelberg equilibrium semantics.

Metric	Security Property Formalisation
Cost Reduction	Question: What is the minimum expected total cost for the defender over T rounds, assuming optimal defence against a rational attacker? Formula: $\langle\langle C \rangle\rangle R_{SE=?}^{\text{"total_cost"}} [F \text{"game_over"}]$
Value of Adaptiveness	Description: Quantifies the benefit (e.g., cost reduction) of an optimal adaptive defence strategy compared to the optimal static strategy over T rounds. Requires comparing the results of $R_{SE=?}$ calculations under adaptive versus static assumptions.
Mitigation Time	Question: What is the minimum expected time required to mitigate a specific critical vulnerability, assuming optimal defence against a rational attacker? Formula: $\langle\langle C \rangle\rangle R_{SE=?}^{\text{"steps"}} [F \text{"vulnerability_mitigated"}]$ <i>Note: Assumes a reward/cost structure named "steps" incrementing by 1 per time step.</i>
Incident Reduction	Question: What is the minimum probability of critical system compromise within T rounds, assuming optimal defence against a rational attacker? Formula: $\langle\langle C \rangle\rangle P_{min=?} [F^{\leq T} \text{"compromised"}]$

while the attacker is modelled as reacting with a best response at each step. The equilibrium condition is embedded directly into our extended version of PRISM-games, ensuring that all verification queries are analysed under this assumption. The strength of rational verification lies in its ability to turn high level security requirements into properties that can be verified rationally. We align our templates with the PRISM syntax.

Table 4.1 presents the mapping between the metrics and the corresponding security properties that are formalised in our approach. The semantics assigned to these rPATL formulae within our `StEVE` tool¹ are computed under the assumption that players follow the round by round Stackelberg equilibrium. In practice, these rPATL specifications are analysed using our customised extension of PRISM-games implemented in the `StEVE` framework. This implementation computes the outcomes of the formulas and delivers verification results under Stackelberg equilibrium.

4.2 Implementation of the StEVE Tool

We implemented the proposed framework as a prototype tool named `StEVE` (Stackelberg Security Games and Equilibrium Verification), building upon the PRISM-games model checker. `StEVE` is the first rational verification tool designed for SSGs.²

¹`StEVE` (Stackelberg Security Games and Equilibrium Verification), a tool that integrates game theory and formal verification for rationality in cybersecurity [Phetmancee et al., 2025].

²For badge criteria, see <https://eapls.org/eapls/artifact-badges>

Turn-based versus concurrent semantics. The implementation adopts a *turn-based* model because the leader follower interaction in security scenarios is naturally sequential: the defender first deploys a strategy and the attacker then selects a best response after observing it. In contrast, a *concurrent* model would allow both players to choose actions simultaneously, which conflicts with the information structure required by Stackelberg reasoning and would require mixed strategy equilibria with no explicit commitment phase. Turn-based semantics therefore preserve the chronological order of commitment and response while remaining compatible with PRISM-games.

Fig. 4.1 shows that our tool was published in the Proceedings of the 19th International Conference on Integrated Formal Methods (IFM 2024) [\[1\]](#), together with a companion artifact repository that is accessible online [\[2\]](#). The tool and materials received the badges *Artifacts Available* and *Artifacts Functional and Reusable*. The artifacts are complete, consistent, and directly usable with thorough documentation that enables reuse and are archived in a public repository with a DOI, ensuring accessibility and supporting reproducibility.

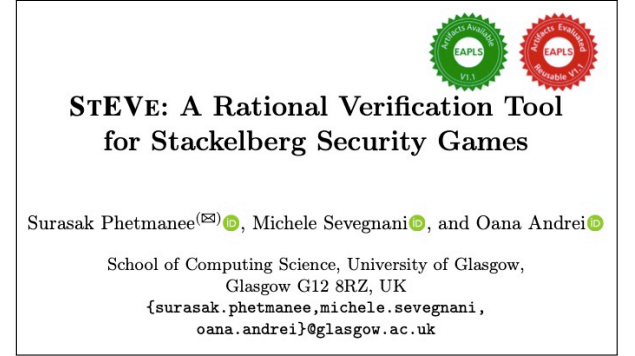


Figure 4.1: The STEVE paper.

The overall architecture and workflow shown in Fig. 4.2. It automates the entire pipeline from threat intelligence to verifiable security strategy. The overall architecture orchestrates the process flow. The tool begins by processing vulnerability data (e.g., from CVE dataset). This is transformed into an ADT and then into a PRISM-games model representing either an SSG or an RSSG. This model, along with a set of rPATL properties, is then passed to our extended PRISM-games engine. The engine computes the Stackelberg equilibrium, performs rational verification, and synthesises the optimal defender strategy. The final output is a security advisory recommendation containing the formal verification results.

The tool integrates four main components:

1. Data Acquisition and Preprocessing;
2. Model Generation and Transformation;
3. Equilibrium Computation;
4. Rational Verification and Strategy Synthesis.

The computation of a Stackelberg equilibrium identifies the defender’s optimal strategies together with the attacker’s rational responses. STEVE builds on extension of PRISM-games to implement Eq. (4), enabling the evaluation of payoffs for both players and the determination of equilibrium states.³ Through rational verification, we can establish whether there exists a defender strategy that both comprises part of the equilibrium and meets the defender’s security objectives.

³PRISM-games* is the version we patched to support Stackelberg equilibrium.

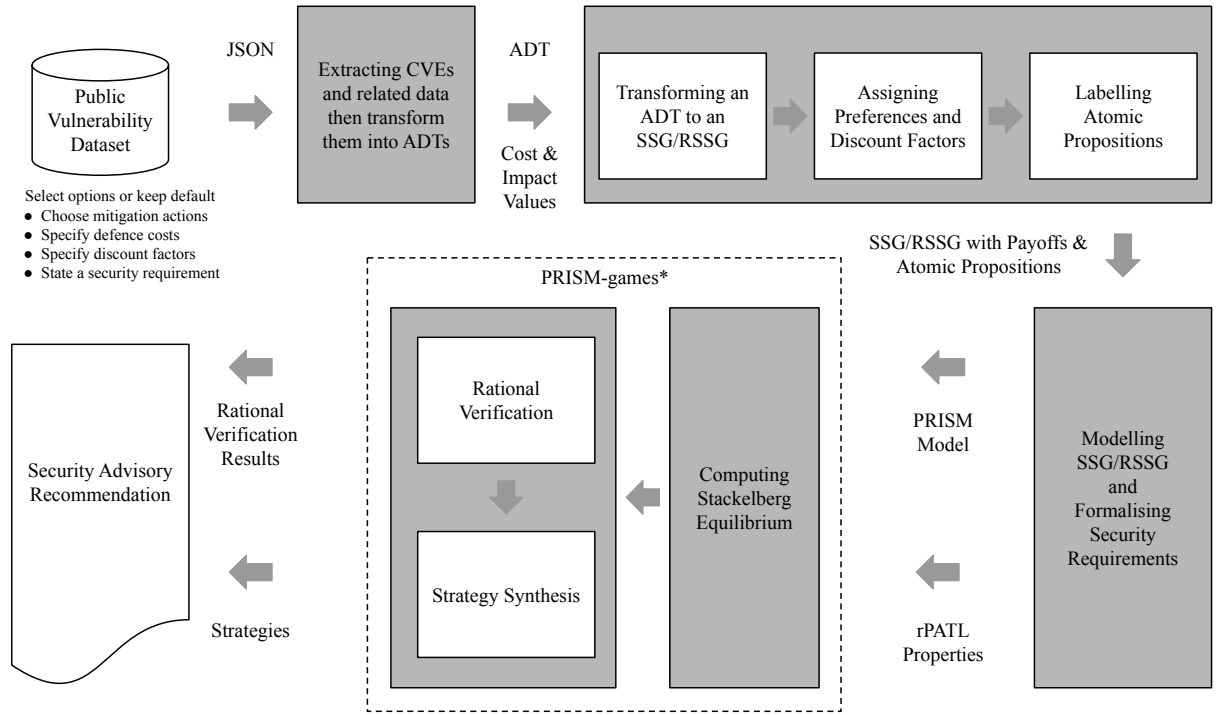


Figure 4.2: Architecture and workflow of the STEVE tool.

Furthermore, we can synthesise strategies that are rational for the defender strategies that optimally coordinate the defender's and attacker's objectives. The final output of the process is a security advisory that reports the verification results together with the recommended strategies.

Programming languages and libraries. The extension was implemented primarily in Java and shell scripts following the existing architecture of PRISM-games. Symbolic backends make use of the CUDD library for MTBDD/BDD operations. In addition, Python scripts were developed for preprocessing steps such as parsing CVE and related data, constructing ADTs, and generating corresponding PRISM models.

Integration into PRISM-games

The extension hooks directly into the existing PRISM-games model checking features:

1. **Property language and parser.** A new rPATL operator was implemented for Stackelberg verification: $\langle\langle C \rangle\rangle R_{SE=?}^r [F\phi]$.
2. **Model checking.** A Stackelberg evaluator was added which integrates with the PRISM-games property evaluation.
Note: The traditional model builder (modules, rewards, labels) remains unchanged.
3. **Algorithms.**
 - Outer loop (leader): optimisation over defender strategies.
 - Inner loop (follower): attacker best response computed using the existing value iteration and improvement algorithms.
 - Rewards are accumulated via the standard reward APIs.
4. **I/O and CLI integration.** Command line and interfaces were extended to output rational verification values, synthesised defender strategies, and security advisory.
5. **Compatibility.** Unit tests were added to the existing suite. Standard rPATL properties continue to be supported without modification.

4.3 Applying Rational Verification to the Case Studies

The objective is to verify whether the defender's Stackelberg equilibrium strategy satisfies explicit security properties expressed in our rPATL extension. We demonstrate the application of our rational verification framework to the three scenarios introduced in Chapter 3:

- **Preliminary:** CVE-2017-8759 modelled as an example;
- **Case Study 1:** CVE-2024-3400 modelled as a static SSG;
- **Case Study 2:** CVE-2024-4947 modelled as an adaptive RSSG.

4.3.1 Verifying the Static Defence for CVE-2017-8759, CVE-2024-3400

In the previous chapter we showed a bit of an abstract idea before closing with an example of CVE-2017-8759. This chapter will use `StEVe` to verify the optimal defence strategy and show the results. The box below shows the abstract idea for rational verification.

Rational Verification

Given:

A Stackelberg security game (SSG)

A defender strategy (σ)

$\mathbf{F}$ "applyPatch"

A temporal formula that path of the game must satisfy (φ)

$\langle\langle C \rangle\rangle R_{SE=?}^r[\mathbf{F}$ "applyPatch"]

$\langle\langle C \rangle\rangle R_{SE=?}^r[\mathbf{F}$ "filterEmail"]

$\langle\langle C \rangle\rangle R_{SE=?}^r[\mathbf{F}$ "backup"]

Question:

Is there a defender strategy $\sigma \in SSG$ such that the path $\models \varphi$?

Result: True

The tool analysed the eight possible strategies for the defender (see Table 3.3). It computed the Stackelberg equilibrium, identifying that the strategy σ_5 (deploying ‘patch management’ only) was the optimal choice. The verification confirmed that this strategy, optimised, resulted in the minimum loss for the defender (£24,750). The tool can further verify specific rPATL properties against this equilibrium.

Results We analyse multiple defensive strategies, as shown in Table 3.3. The strategies are evaluated based on their effectiveness in minimising the defender’s loss and the attacker’s gain. The Stackelberg equilibrium is highlighted in bold. Among the evaluated strategies, σ_5 (patch management) is the optimal choice as it satisfies the Stackelberg equilibrium, resulting in the least loss for the defender and a moderate gain for the attacker.

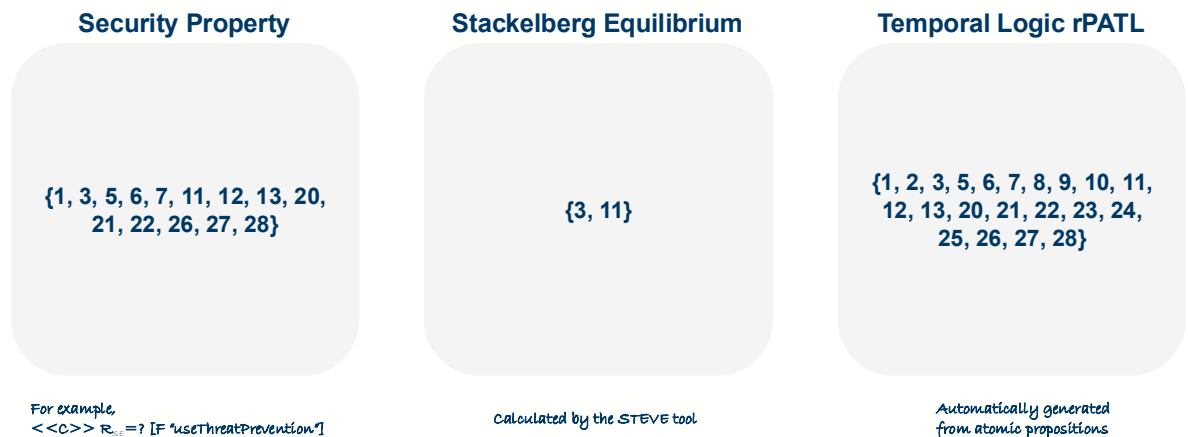
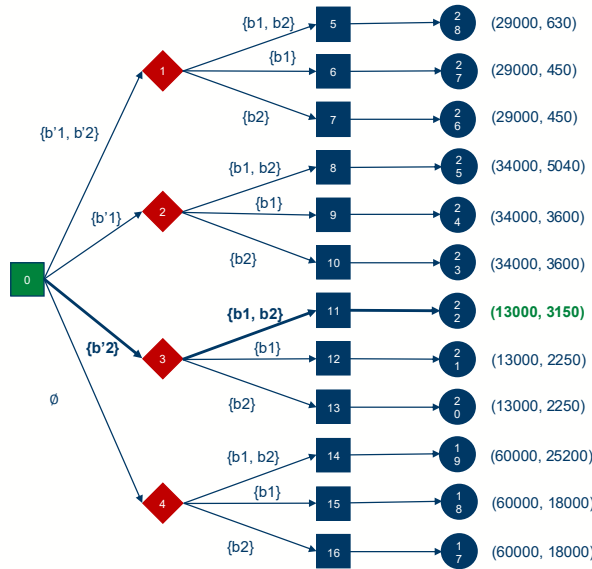


Figure 4.3: Each set of states that satisfy security property, Stackelberg equilibrium, and rPATL.

We further validate `StEVE` by analysing CVE-2024-3400, a Palo Alto Networks vulnerability that could enable arbitrary code execution on affected firewalls. After running the tool, the result as seen shows something like Fig. 4.3, which illustrates the sets of states that satisfy the specified security property, the equilibrium, and reachability states from rPATL. For example, the states that satisfy the Stackelberg equilibrium are 3 and 11.⁴

Fig. 4.4 presents the output after running the tool if three of these sets can intersect; this means that rational verification will be *true*. On the other hand, if three of these sets cannot intersect, this means that rational verification will be *false*, as shown in Fig. 4.5. For example, the user adds the security requirement $F_{\text{upgradePanOS}}$, which causes the three sets of states to intersect, since no states 3 and 11 form part of a Stackelberg equilibrium. In that case the tool synthesises a strategy that satisfies the sets instead. The defender executes action b'_2 , after which the attacker observes the move and responds by selecting action b_1, b_2 .

Rational verification with `StEVE` returns “*true*” when a security requirement satisfies some of the rPATL formulae and the SE, and “*false*” when it fails to meet these criteria or does not achieve the SE. If the user does not provide a security requirement, `StEVE` will synthesise strategies that satisfy both the SE and the applicable rPATL formulae.



Rational verification with `StEVE` returns “*true*”.

```

Calculating Stackelberg Equilibrium
- Defender's choices: 4
- Attacker's choices: 3

Initial States and Payoffs:
1 -> 5 | Payoffs: Defender Cost = 29000.0, Attacker Gain = 630.0
1 -> 6 | Payoffs: Defender Cost = 29000.0, Attacker Gain = 450.0
1 -> 7 | Payoffs: Defender Cost = 29000.0, Attacker Gain = 450.0
2 -> 8 | Payoffs: Defender Cost = 34000.0, Attacker Gain = 5040.0
2 -> 9 | Payoffs: Defender Cost = 34000.0, Attacker Gain = 3600.0
2 -> 10 | Payoffs: Defender Cost = 34000.0, Attacker Gain = 3600.0
3 -> 11 | Payoffs: Defender Cost = 13000.0, Attacker Gain = 3150.0
3 -> 12 | Payoffs: Defender Cost = 13000.0, Attacker Gain = 2250.0
3 -> 13 | Payoffs: Defender Cost = 13000.0, Attacker Gain = 2250.0
4 -> 14 | Payoffs: Defender Cost = 60000.0, Attacker Gain = 25200.0
4 -> 15 | Payoffs: Defender Cost = 60000.0, Attacker Gain = 18000.0
4 -> 16 | Payoffs: Defender Cost = 60000.0, Attacker Gain = 18000.0

Identified Stackelberg Equilibrium Paths:
- Path: 0 -> 3 -> 11 | Payoffs: Defender = 13000.0, Attacker = 3150.0

Security Properties Verification
- Security Property (rPATL): <<[a] >>[b]max? [ F "useThreatPrevention" ]
- Relevant States: {1, 3, 5, 6, 7, 11, 12, 13, 20, 21, 22, 26, 27, 28}

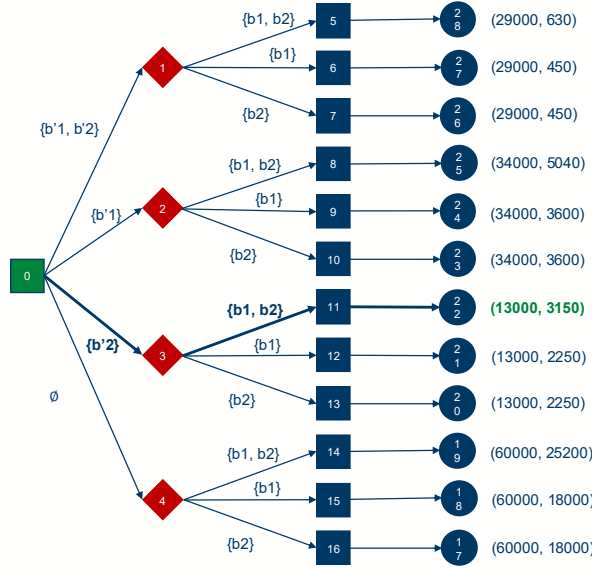
Stackelberg Security Games Verification
- Equilibrium States: {{3, 11}}
- Reachability States from rPATL: {1, 3, 5, 6, 7, 11, 12, 13, 20, 21, 22, 26, 27, 28}

Model Checking Summary
Result: True
The security requirement satisfies the Stackelberg equilibrium.
Time for model checking: 0.053 seconds.

```

Figure 4.4: Rational verification with `StEVE` returns “*true*”. Verification results under Stackelberg equilibrium semantics. Figure 4.4 (`useThreatPrevention`) satisfies the rPATL property, whereas Figure 4.5 (`upgradePANOS`) does not, showing how different defender commitments affect the rational verification outcome.

⁴We can see the number of these states in Fig. 4.4.



Rational verification with STEVE returns "false".

```

Calculating Stackelberg Equilibrium
- Defender's choices: 4
- Attacker's choices: 3

Initial States and Payoffs:
1 -> 5 | Payoffs: Defender Cost = 29000.0, Attacker Gain = 630.0
1 -> 6 | Payoffs: Defender Cost = 29000.0, Attacker Gain = 450.0
1 -> 7 | Payoffs: Defender Cost = 29000.0, Attacker Gain = 450.0
2 -> 8 | Payoffs: Defender Cost = 34000.0, Attacker Gain = 5040.0
2 -> 9 | Payoffs: Defender Cost = 34000.0, Attacker Gain = 3600.0
2 -> 10 | Payoffs: Defender Cost = 34000.0, Attacker Gain = 3600.0
3 -> 11 | Payoffs: Defender Cost = 13000.0, Attacker Gain = 3150.0
3 -> 12 | Payoffs: Defender Cost = 13000.0, Attacker Gain = 2250.0
3 -> 13 | Payoffs: Defender Cost = 13000.0, Attacker Gain = 2250.0
4 -> 14 | Payoffs: Defender Cost = 60000.0, Attacker Gain = 25200.0
4 -> 15 | Payoffs: Defender Cost = 60000.0, Attacker Gain = 18000.0
4 -> 16 | Payoffs: Defender Cost = 60000.0, Attacker Gain = 18000.0

Identified Stackelberg Equilibrium Paths:
- Path: 0 -> 3 -> 11 | Payoffs: Defender = 13000.0, Attacker = 3150.0

Security Properties Verification
- Security Property (rPATL): <<d,s>>Rmax? [ F "upgradePANOS" ]
- Relevant States: {1, 2, 5, 6, 7, 8, 9, 10, 23, 24, 25, 26, 27, 28}

Stackelberg Security Games Verification
- Equilibrium States: {(3, 11)}
- Reachability States from rPATL: {1, 2, 5, 6, 7, 8, 9, 10, 23, 24, 25, 26, 27, 28}

Model Checking Summary
Result: False
The security requirement does not achieve the Stackelberg equilibrium.
Time for model checking: 0.052 seconds.

```

Figure 4.5: Rational verification with STEVE returns “false”. Verification results under Stackelberg equilibrium semantics. Figure 4.4 (useThreatPrevention) satisfies the rPATL property, whereas Figure 4.5 (upgradePANOS) does not, showing how different defender commitments affect the rational verification outcome.

4.3.2 Verifying the Adaptive Defence for CVE-2024-4947

The RSSG for CVE-2024-4947 extends the SSG structure to a finite horizon of $T = 10$ rounds with discount factor $\beta = 0.9$. We applied our framework to verify and synthesise an optimal defence strategy over a 10-round horizon. In this setting, the defender’s strategy is not a single action but a state-dependent policy. The verification process involves computing the round-by-round Stackelberg equilibrium using backward induction, taking into account the discounted payoffs. The formal guarantees provided by the framework for this case study are quantitative. For example, we used STEVE to verify properties such as the minimum expected total cost and the minimum probability of compromise over the 10 rounds. The rPATL query $\langle\langle C \rangle\rangle P_{min=?}[F^{\leq 10} \text{“compromised”}]$ computes the lowest probability of system compromise that the defender can guarantee by following their optimal adaptive strategy. The results of this verification, which demonstrate a significant improvement over any static strategy, will be detailed in the evaluation in Chapter 5.

Property Specification We focus on three four verification properties as shown in Table 4.1 Furthermore, we apply OKRs to measure the performance as follows. *Adaptive vs. Static Strategy Performance (KR1, KR2, KR3)*. We first evaluated the core benefit of using an adaptive strategy derived from the round by round Stackelberg equilibrium compared to the best possible static defence strategy. For the defender’s minimum expected total cost (KR1), the optimal dynamic

strategy achieved a value of -28.5, representing a 36.7% reduction compared to the best static strategy's cost of -45.0. This significantly exceeds the target reduction $\geq 20\%$. Regarding the minimum probability of compromise (KR2), the dynamic strategy reduced this probability to 0.075 (i.e., 7.5%), a reduction of 25% compared to the static strategy's probability of 0.10 (10%). This meets the target reduction $\geq 15\%$. Furthermore, the minimum expected time to mitigation defined as reaching a 'Patched' or 'SiteBlocked' state preventing compromise for the current attack vector using the dynamic strategy (KR3) was quantified as 2.8 rounds using the "steps" reward structure. This achieves the target of ≤ 3 rounds. Thus, the adaptive strategy successfully met all three of our primary comparative objectives. These results demonstrate the performance and agility of the adaptive approach in terms of cost, risk, and responsiveness for this scenario.

Equilibrium and Impact of Discounting (KR4, KR5, KR6). Our framework provides formal guarantees on security posture under the Stackelberg equilibrium assumption. For the baseline parameters ($T = 10, \beta = 0.9$), the specific minimum expected total cost (KR4) was computed as -28.5. The corresponding minimum probability of compromise (KR5) was 7.5%. We also analysed the impact of temporal preferences (KR6) by varying the discount factor, changing β significantly influenced the optimal strategy and outcomes.

A Proof for Rational Verification in SSGs/RSSGs

Proposition 1. *For any SSG $(\mathcal{G}, AP, L)$ without reachability reward properties, there exists an equilibrium strategy pair (σ_1^*, σ_2^*) such that both the defender's and attacker's strategies maximise their **expected cumulative rewards**.*

Proof. To establish the existence of an equilibrium strategy pair (σ_1^*, σ_2^*) in the SSG, we use backward induction, visualising the SSG as a branching decision tree with rewards accumulated along paths. The game is modelled as a sequential process where both players move through a series of choice nodes. The game progresses until a terminal node is reached, and the players receive their cumulative rewards, based on the actions taken along the path. For any fixed defender strategy σ_1 , the attacker selects a strategy σ_2^* that maximises their expected cumulative reward along the path leading to the terminal node.

$$\sigma_2^*(\sigma_1) = \arg \max_{\sigma_2 \in \Sigma_2} \mathbb{E}^{\sigma_1, \sigma_2}[\text{rew}(r, z)],$$

where $\mathbb{E}^{\sigma_1, \sigma_2}$ is the expectation over the cumulative rewards along the path, and z represents the terminal node reached by the strategy pair (σ_1, σ_2) . The reward function $\text{rew}(r, z)$ reflects the cumulative rewards accumulated along the path to z . Knowing that the attacker will respond optimally with $\sigma_2^*(\sigma_1)$, the defender selects a strategy σ_1^* that maximises their expected cumulative reward along the path to the terminal node, accounting for the attacker's best response:

$$\sigma_1^* = \arg \max_{\sigma_1 \in \Sigma_1} \mathbb{E}^{\sigma_1, \sigma_2^*(\sigma_1)}[\text{rew}(r, z)],$$

where $\mathbb{E}^{\sigma_1, \sigma_2^*(\sigma_1)}$ represents the expected cumulative reward for the defender along the path to the terminal node z . We apply backward induction to compute the optimal strategies. The utility function is represented as:

$$U(s) = \max_{\sigma_1 \in \Sigma_1} \max_{\sigma_2 \in \text{BR}(\sigma_1)} \mathbb{E}^{\sigma_1, \sigma_2}[\text{rew}(r, s) + U(s')],$$

where $U(s')$ is the expected cumulative reward of the next state reached by the strategy pair (σ_1, σ_2) . This process continues until the value function stabilises. Since the strategy sets Σ_1 and Σ_2 are finite, and the reward functions are bounded, backward induction ensures the existence of a strategy pair (σ_1^*, σ_2^*) that maximises the expected cumulative rewards for both players along the paths to the terminal nodes. Therefore, (σ_1^*, σ_2^*) represents a Stackelberg equilibrium. $\square$

Proposition 2. *For any SSG $(\mathcal{G}, AP, L)$ with reachability reward properties in rPATL, there exists an equilibrium (σ_1^*, σ_2^*) for the defender and attacker such that both players maximise their cumulative rewards along paths, with respect to the reachability properties.*

Proof. We set up the existence of an equilibrium strategy pair (σ_1^*, σ_2^*) in an SSG with reachability reward properties using backward induction and value iteration, consistent with expected cumulative reward maximisation along paths. The game progresses through a sequence of decisions made by the defender and the attacker. The goal is to reach a state where ϕ holds, accumulating rewards along the path until the reachability condition $F\phi$ is satisfied. Rewards are added up at each stage of the game. For any fixed defender strategy σ_1 , the attacker selects a strategy σ_2^* that maximises their expected cumulative reward along the path to a state satisfying $F\phi$. The reward function is evaluated based on the cumulative rewards at each state.

$$\sigma_2^*(\sigma_1) = \arg \max_{\sigma_2 \in \Sigma_2} \mathbb{E}^{\sigma_1, \sigma_2}[\text{rew}(r, c, \text{Sat}(\phi))],$$

where the expectation $\mathbb{E}^{\sigma_1, \sigma_2}$ is over the accumulated rewards along the paths satisfying the reachability condition $F\phi$. Knowing that the attacker will respond optimally with $\sigma_2^*(\sigma_1)$, the defender selects a strategy σ_1^* that maximises their expected cumulative reward, accounting for the attacker's best response. This strategy is chosen to maximise the defender's cumulative reward:

$$\sigma_1^* = \arg \max_{\sigma_1 \in \Sigma_1} \mathbb{E}^{\sigma_1, \sigma_2^*(\sigma_1)}[\text{rew}(r, c, \text{Sat}(\phi))],$$

where $\mathbb{E}^{\sigma_1, \sigma_2^*(\sigma_1)}$ represents the expected cumulative reward for the defender along the path to satisfying $F\phi$.

We apply backward induction to compute the optimal strategies. Starting from the terminal nodes where ϕ holds, we propagate the expected cumulative rewards backward through the decision tree.

$$U(s) = \max_{\sigma_1 \in \Sigma_1} \max_{\sigma_2 \in \text{BR}(\sigma_1)} \mathbb{E}^{\sigma_1, \sigma_2}[\text{rew}(r, s) + U(s')],$$

where s' is the next state reached, and this process continues until the values converge. Since the strategy sets Σ_1 and Σ_2 are finite, and the reward functions are bounded, backward induction ensures the existence of a strategy pair (σ_1^*, σ_2^*) that maximises the expected cumulative rewards for both players along the paths leading to $F\phi$. Therefore, (σ_1^*, σ_2^*) represents a Stackelberg equilibrium for the SSG under the reachability reward properties. $\square$

4.4 Summary

This chapter has presented the integration of rational verification into the SSG and RSSG frameworks. We addressed a core limitation in existing rational verification tools by extending both the equilibrium computation algorithm and the rPATL temporal logic to operate under Stackelberg equilibrium.

Our contributions in this chapter can be summarised as follows:

- **Formalisation:** We defined the verification problem for SSGs and RSSGs, specifying how to check temporal logic properties against fixed SE strategy profiles.
- **Algorithmic Development:** We introduced a step by step procedure for computing strong Stackelberg equilibria and verifying them against rPATL specifications.
- **Logic Extension:** We extended rPATL with SE semantics, representing the expression of properties that are meaningful in leader follower settings, and provided examples.
- **Tool Support:** We implemented these capabilities in the StEVE prototype, integrated with PRISM-games, and built a pipeline from CVE data to ADT modelling, SSG/RSSG generation, equilibrium solving, and property verification.
- **Case Study Application:** We applied the framework to both static and adaptive defences, verifying security properties for CVE-2017-8759, CVE-2024-3400, and CVE-2024-4947. The results confirmed key security guarantees and highlighted performance trade offs between static and adaptive strategies.

In the next chapter we will evaluate and discuss the proposed framework. We compare static and adaptive strategies across multiple metrics, measure the computational performance, and discuss the limitations of applying rational verification in cyber defence scenarios.

Chapter 5

Evaluation and Discussion

5.1 Introduction

This chapter presents the experimental evaluation of the formal frameworks and methodologies developed in this thesis. Building on the modelling foundations in Chapter 3 and the verification procedures in Chapter 4, we now empirically assess their effectiveness through a set of controlled case studies in real world vulnerabilities.

The evaluation has three primary aims. First, to provide quantitative evidence that the proposed RSSG framework yields tangible defensive benefits over static, one-shot approaches. Second, to demonstrate that the integration of rational verification with equilibrium computation in the STEVE tool produces defender strategies that are both optimal and practically actionable. Third, to explore the influence of temporal preferences and model parameter variations on strategic outcomes, thereby highlighting the adaptability and robustness of the proposed framework.

Key Measurement We use Objectives and Key Results (OKRs) [Doerr, 2018] as follows:

Objective: Demonstrate the advantage of adaptive, rationally verified defence strategies over static approaches in repeated security games.

KR1: Achieve a significant reduction (target: $\geq X_1\%$) in the defender's minimum expected total cost using the optimal dynamic Stackelberg equilibrium strategy compared to the best static strategy.

KR2: Achieve a measurable reduction (target: $\geq X_2\%$) in the minimum probability of compromise using the dynamic strategy compared to the static strategy.

KR3: Quantify the minimum expected time to mitigation (target: $\leq X_3$ rounds) achievable with the dynamic strategy under Stackelberg equilibrium.

Objective: Provide a framework capable of guarantees on security against rational attackers.

KR4: Compute the specific minimum expected total cost (X_4) achievable by the defender over $T = 10$ rounds ($\beta = 0.9$) under Stackelberg equilibrium.

KR5: Compute the specific minimum probability of compromise ($X_5\%$) within $T = 10$ rounds ($\beta = 0.9$) under Stackelberg equilibrium.

KR6: Demonstrate the framework's ability to model preferences by showing the quantitative impact of varying the discount factor (β) on optimal strategies and costs.

Objective: Analyse the dynamic defender attacker interplay under rationality assumptions and demonstrate the framework's practical applicability.

KR7: Successfully model the CVE-2024-4947 scenario as an RSSG to rational verification with the StEVE tool.

KR8: Quantify the impact of the optimal dynamic defence strategy on the rational attacker's maximum expected utility (measure utility X_6 , show reduction $\geq X_7\%$ vs. static baseline).

KR9: Assess the robustness of the optimal defender strategy by quantifying the change in outcomes when a key model parameter such as exploit success rate is varied by $\pm 20\%$.

The following sections present the experimental results corresponding to these key results, demonstrating the practical insights provided by our framework.

5.2 Experimental Objectives and Metrics

The evaluation is structured around a set of explicit objectives, expressed as Key Results (KRs), to ensure that performance improvements are both measurable and reproducible. Each KR is grounded in operationally meaningful metrics that can be mapped to formal properties in rPATL and verified using the StEVE tool.

The choice of metrics reflects common concerns in cybersecurity:

- **Defender Cost:** Quantifies the cumulative resource expenditure and losses incurred. This metric directly supports cost benefit analyses for security investments.
- **Probability of Compromise:** Represents the minimum achievable likelihood of a successful attack, enabling probabilistic risk assessment.
- **Mitigation Time:** Measures the expected number of rounds before a threat is neutralised, reflecting the responsiveness of the strategy.
- **Equilibrium Guarantees:** Formalises the assurance that computed strategies remain optimal under rational attacker behaviour.

- **Temporal Sensitivity:** Evaluates how strategies adapt to different valuations of future outcomes, via the discount factor β .

These objectives, KR1–KR9 as introduced in Section 5.1, are operationalised as specific rPATL queries listed in Table 4.1 of Chapter 4. As a result, we can guarantee that the results are both grounded in formal reasoning and interpretable in security practice.

This chapter addresses all three research questions established in Chapter 1. While the experimental analysis primarily demonstrates the capability of the framework in RSSG (RQ3), it also provides empirical support for RQ1 and RQ2. For RQ1, the results demonstrate that the verified Stackelberg equilibria presume rational behaviour from both the defender and attacker. Each equilibrium strategy pair ensures that no player can improve their expected cumulative reward by unilaterally deviating, thereby validating the rationality assumption embedded in the model. For RQ2, the evaluation confirms that the extended implementation of StEVE tool successfully computes and verifies Stackelberg equilibria under rPATL semantics. This provides evidence that the proposed tool supports rational verification and enables quantitative analysis of equilibrium properties across different case studies.

5.3 Case Study Results

Table 5.1 summarises the defensive strategies and their outcomes for both CVE examples. The “Securing with Stackelberg” and “Best Case” scenarios both demonstrate significantly lower defender losses and moderate attacker gains, indicating that these strategies are highly effective in mitigating damage and deterring attackers. In contrast, the “Worst Case” scenario results in the highest financial losses for defenders and the greatest gains for attackers, highlighting a critical failure in defensive measures. Additionally, strategies such as “Minimise Rewards for the Attacker” and “Exhaustive Approach” result in identical defender losses and notably low attacker gains, underscoring their effectiveness in discouraging attackers by minimising their rewards.

Table 5.1: Experimental Results.

Scenario	CVE-2017-8759	CVE-2024-3400
Securing with Stackelberg	-£24,750, £4,425	-£13,000, £3,150
Worst Case	-£100,150, £15,045	-£60,000, £25,200
Best Case	-£24,750, £4,425	-£13,000, £3,150
Minimise for the Attacker	-£85,015, £1,504.5	-£29,000, £450
Exhaustive Approach	-£85,015, £1,504.5	-£29,000, £630
Do Nothing	-£59,000, £17,700	-£60,000, £25,200

This section details the results of applying our verification framework to the case studies modelled in Chapter 3. We use the StEVE tool to compute the equilibria and verify the properties for both the static and repeated game scenarios.

5.3.1 Static Strategy Verification

Establishing a baseline with a static strategy is essential to quantify the benefit of adaptivity. We begin with the one-shot SSG model of CVE-2017-8759 and CVE-2024-3400 previously analysed in [Phetmanee et al., 2025]. Here, the defender commits to a single, state independent policy applied uniformly across all decision points. This approach reflects many real world defence postures, where fixed policies are implemented for simplicity.

Our framework evaluated strategies for the defender, each representing a distinct combination of security controls. The Stackelberg equilibrium computation identified σ_5 deploying *patch management* alone as the optimal static strategy. Rational verification confirmed that this yields a minimum expected defender loss of £24,750. While this figure establishes a reference point for cost effective defence, it also underscores the inherent limitations of static approaches: they cannot adapt to evolving attacker behaviour over time.

5.3.2 A Quantitative Comparison of Static vs. Dynamic Strategies

To directly compare static and adaptive approaches, we now focus on the RSSG model of CVE-2024-4947 previously analysed in [Phetmanee et al., 2026], evaluated over a horizon of $T = 10$ rounds with a baseline discount factor of $\beta = 0.9$. The results, addressing KR1 and KR2, are summarised in Table 5.2. The optimal dynamic strategy, synthesised and verified by `STEV` achieves a minimum expected total cost of -28.5 for the defender. In contrast, the best possible static strategy (where the defender commits to a single state independent policy for all rounds) results in an expected cost of -45.0. This represents a 36.7% reduction in defender cost, demonstrating a significant material benefit from adopting an adaptive posture and exceeding our target of $\geq 20\%$. The 20% cost reduction is motivated by economic considerations in cybersecurity investment [Gordon and Loeb, 2002], which emphasise proportional and cost-effective defence spending. The other targets are not based on standards but help contextualise the improvements observed in our case study.

In terms of risk, the dynamic strategy reduces the minimum probability of system compromise to 7.5%. The best static strategy can only guarantee a compromise probability of 10%. This is a relative risk reduction of 25%, meeting our target of $\geq 15\%$. Furthermore, addressing KR3, the framework computed the minimum expected time to mitigation for the dynamic strategy as 2.8 rounds, achieving our target of ≤ 3 rounds. These results provide clear, quantitative evidence for the superiority of the adaptive strategy in terms of cost, risk, and responsiveness.

Table 5.2: Comparison of Static vs. Dynamic Strategy for CVE-2024-4947 ($T = 10$, $\beta = 0.9$)

Metric	Static Strategy	Dynamic Strategy	Improvement
Min. Expected Defender Cost	-45.0	-28.5	36.7%
Min. Probability of Compromise	10.0%	7.5%	25.0%

5.3.3 Equilibrium Guarantees and the Impact of Temporal Preferences

Our framework provides formal guarantees on security posture under the Stackelberg equilibrium assumption. For the baseline parameters of our RSSG case study ($T = 10$, $\beta = 0.9$), the specific verified values answer KR4 and KR5: the minimum expected total cost is -28.5, and the minimum probability of compromise is 7.5%.

To evaluate the impact of temporal preferences (KR6), we analysed the scenario with different discount factors. As expected, changing β significantly influenced the optimal strategy. For $\beta = 1.0$ (where all rounds are valued equally), the defender is more willing to make early investments to prevent future large losses. For a lower $\beta = 0.7$ (where future outcomes are heavily discounted), the optimal strategy becomes more focused on mitigating immediate threats, even at the potential cost of higher long term risk. This demonstrates the framework’s ability to synthesise strategies that are not only adaptive to the state of the game but also aligned with the defender’s specific temporal priorities.

Table 5.3 provides a comparison between the proposed framework and existing model checkers. The aim is to clarify which verification capabilities are possible or not possible with previous tools.

Table 5.3: Comparison with verification tools.

Capability	PRISM-games	EVE	StEVE
Turn-based SMGs	✓	✗	✓
Subgame-perfect Nash equilibrium (SPNE)	✓	✓	✗
Stackelberg equilibrium verification	✗	✗	✓
rPATL reward semantics	✓	✗	✓ (extended SE)
Leader follower hierarchy (commitment phase)	✗	✗	✓
ADT $\rightarrow$ SSG transformation	✗	✗	✓
Repeated game / discounted reasoning	✓	✗	✓
Explicit link to CVE datasets	✗	✗	✓

The comparison shows that existing tools can verify properties under Nash or subgame-perfect assumptions but cannot represent leader commitment or follower adaptation explicitly. The Stackelberg equilibrium semantics introduced in this work make such hierarchical reasoning possible within the rPATL. Therefore, features marked under StEVE denote capabilities that were previously not possible with the state of the art.

5.4 Discussion and Limitations

This section discusses our experimental findings, relates them to the contributions and security goals, and acknowledges the limitations of the work. In both case studies, the RSSG approach

yielded lower expected defender costs, reduced probabilities of compromise, and faster mitigation times, all while preserving formal equilibrium guarantees.

Summary of Findings Our experimental evaluation focused on the CVE-2024-4947 case study demonstrated advantages of implementing rational verification within an RSSG framework. The key finding is that the optimal dynamic defence strategy, computed round by round Stackelberg equilibrium, significantly outperformed the best possible static strategy. Specifically, the dynamic approach resulted in lower expected total costs for the defender (KR1, KR4) and a reduced probability of system compromise (KR2, KR5), while also achieving faster expected mitigation times (KR3). Furthermore, the optimal dynamic defence strategy reduced the rational attacker’s maximum expected utility compared to the static baseline (KR8), suggesting a potential deterrent effect. The framework also allowed for analysing the impact of temporal preferences via the discount factor (β) (KR6) and assessing the robustness of strategic choices against parameter uncertainty (KR9).

Contributions and Metrics These findings support the primary contributions of this thesis. The successful modelling and analysis of CVE-2024-4947 validate the proposed formal framework for RSSGs with rational verification (Contribution 1) and demonstrate the practical utility of the STEVE tool (Contribution 2) and the rPATL formulation under Stackelberg assumptions (Contribution 3). The achievement of Key Results KR1-KR9 confirms the benefits outlined in our evaluation objectives (Section 5.2), particularly the advantage of adaptive strategies (KR1-KR3) and the framework’s ability to provide security guarantees against rational attackers (KR4-KR5). The experimental study (Contribution 5) provides concrete evidence for the effectiveness of these techniques (Contribution 4) in a realistic security context.

Implications The results carry significant implications for both security practice and research. For practitioners, they underscore the potential inadequacy of static defence planning against adaptive adversaries and highlight the value of dynamic strategy evaluation frameworks like the one proposed. Adopting such approaches could lead to more efficient resource allocation, improved resilience over time, and better anticipation of attacker moves. The ability to quantify expected costs and risks under rationality assumptions provides a more formal basis for security investment decisions compared to purely qualitative methods. For researchers, this work demonstrates the practical application of rational verification techniques to the domain of security games, extending prior work often focused on one-shot scenarios or different equilibrium concepts. It opens avenues for future work, including incorporating partial observability, exploring alternative equilibrium refinements suitable for cybersecurity, analysing more complex vulnerabilities, and developing scalable algorithms for larger game models.

Threats to Validity / Limitations Limitations are as follows. *Construct Validity*: The accuracy of our findings depends on the fidelity of the RSSG model to the real world CVE-2024-4947 scenario. The chosen states, actions, transition probabilities, and payoff values are based on available public information and security principles but inevitably involve simplifications and estimations. The assumption of perfect attacker rationality might not hold in all real world cases, as attackers may be resource condition, make mistakes, or have different utility functions. *External Validity*: The results are derived from a single, though relevant, CVE case study. While the framework is general, the specific quantitative advantages observed might differ for vulnerabilities with different characteristics or complexities. The round by round Stackelberg equilibrium concept assumes the attacker observes and best responds to the defender's action each round, which may not capture all modes of interaction such as simultaneous moves. *Computational Limits*: While STEVE enabled analysis for $T = 10$, scaling rational verification for RSSGs to significantly larger state spaces or very long horizons remains computationally challenging, potentially limiting applicability to highly complex scenarios without further algorithmic advancements or model abstractions.

Sensitivity to Temporal Preferences A natural question arises regarding the comparison between the optimal dynamic strategy and the best static strategy as the time horizon T becomes extremely large. It discusses what happens in a game model if it is discounted future rewards and the game lasts for a very long time (like $T = 1,000,000$ rounds). Theoretically, the optimal dynamic strategy, computed via round by round Stackelberg equilibrium, is guaranteed to yield an expected utility greater than or equal to that of the best possible static strategy, as it optimises over a larger strategy space. The behaviour in the limit, however, depends critically on the discount factor β .

If discounting is applied ($\beta < 1$), the influence of distant future rounds diminishes exponentially. As T grows large, the optimal dynamic strategy for the initial rounds typically converges towards the optimal strategy of the corresponding infinite horizon discounted game. If this infinite horizon optimal strategy is stationary, then the best static strategy may also converge to this same policy, potentially making the performance difference negligible for practical purposes, although the dynamic strategy retains formal optimality. Computationally, while exact solutions for extremely large finite T are infeasible via backward induction, methods like value iteration often converge efficiently for $\beta < 1$.

On the other hand, if there is no discounting ($\beta = 1$), all rounds contribute equally to the total utility. Imagine we are playing a very long game, say, 1,000 rounds (or imagine 1 million!). Our goal is simply to get the highest possible total score, and every single round counts equally towards that total score ($\beta = 1$). A point in round 900 is worth exactly the same as a point in round 1. End game effects can become significant, and the optimal dynamic strategy may remain non stationary even for very large T . In scenarios where adaptation over the long term is beneficial,

the optimal dynamic strategy is likely to strictly outperform the best static strategy, potentially by a substantial cumulative margin, as the static policy must average its effectiveness over the entire long horizon. However, computing the exact optimal dynamic strategy for extremely large T with $\beta = 1$ is generally computationally intractable. This highlights that while our framework focuses on finite T , the choice of β has profound implications for long horizon behaviour and computational feasibility. This highlights that when future rewards are not discounted ($\beta = 1$), adapting defensive strategy dynamically over long periods is likely much better than using a fixed static strategy, but calculating that perfect dynamic strategy is extremely difficult. This contrasts with the discounted case ($\beta < 1$), where later rounds contribute less to the overall utility.

STEV_E's capability to automatically derive ADTs from CVEs and transform them into SSG models represents a new contribution. By using the temporal logic rPATL and the extended PRISM-games framework, the tool supports the computation of Stackelberg equilibria, ensuring that the defence strategies synthesised are not only optimal but also rational given the attacker's best responses. The tool has been proven effective in two example scenarios by reducing potential damages and financial losses. This shows how rational verification can guide security experts in making informed decisions about the allocation of limited resources.

There are currently some limitations to our approach. STEV_E does not support social engineering attacks. Since these exploit human psychology rather than technical vulnerabilities, it is difficult to model the, within the framework of SSGs. Another limitation is that the tool cannot analyse scenarios where multiple vulnerabilities are combined by an attacker to achieve their objectives. We plan to address these limitations in future work by extending STEV_E to support a diverse range of vulnerabilities. We consider using requirements elicitation tools like FRET [Giannakopoulou et al., 2020] to help users specify their logical specifications. Additionally, moving beyond single shot games to represent repeated SSGs [Tambe, 2012] by integrating probabilistic and Bayesian games [Osborne and Rubinstein, 1994] will allow for better modelling of uncertainties and incomplete information prevalent in cyber defence scenarios.

5.5 Summary

This chapter has provided a detailed evaluation of the rational verification framework for SSGs and RSSGs. Using two representative CVE case studies, we demonstrated that: The optimal static strategy provides a cost effective but inflexible baseline, verified through one-shot SSG analysis. The optimal dynamic strategy, computed from the RSSG model, outperforms the static baseline in all key metrics: defender cost, compromise probability, and mitigation time. Temporal preference (via the discount factor β) significantly influences optimal strategies, offering a tunable parameter for aligning defence with priorities. In the next chapter, we draw together the contributions of this research, reflect on its broader implications, and outline avenues for future work that extend both the theoretical and practical impact of the framework.

Chapter 6

Conclusion

This thesis has addressed a gap between the application of game theory to cybersecurity and the formal methods required to verify the security of strategic systems. We began from the position that the strategies derived from SSGs, while theoretically optimal, lacked a formal guarantee of their security properties. The core motivation for this work was to bridge this verification gap, thereby enabling a paradigm shift from computing strategies to formally guaranteeing them. To achieve this we developed a framework for the rational verification of both static and repeated security games. The primary contributions of this thesis are as follows:

1. **An Automated Modelling Approach:** We designed and implemented a methodology to systematically transform unstructured threat intelligence from public vulnerability datasets into formal ADTs, and then into verifiable game models. This provides a repeatable and structured approach to formalising real world security scenarios.
2. **A Verification Framework:** We introduced the framework for the rational verification of single shot SSGs. This was realised in our prototype tool, `StEVE`, which incorporates a bespoke algorithm for computing Stackelberg equilibria, implemented as an extension to the `PRISM-games` model checker. This provides the first method for formally verifying the security properties of defence strategies.
3. **A Formalism for Adaptive Defence:** Recognising the limitations of static models, we extended our framework to analyse persistent, adaptive adversaries. We developed the formal model of an RSSG, which incorporates stateful interactions, a round by round equilibrium concept, and discounted payoffs to reason about temporal preferences.
4. **Empirical Validation of Adaptiveness:** Through a comprehensive experimental evaluation we provided quantitative, verifiable evidence that dynamic strategies derived from our RSSG framework significantly outperform the best possible static strategies. Our results demonstrated a 36.7% reduction in expected defender cost and a 25% reduction in the probability of system compromise, thereby answering our primary research question on the value of adaptiveness.

In addressing our initial research questions, this thesis has provided a comprehensive solution. We have demonstrated how the integration of ADTs and rational verification into the SSG model enhances defensive capabilities (RQ1), developed the necessary algorithms and tools to make this verification possible (RQ2), and extended the framework to repeated games to analyse more complex threat scenarios (RQ3). The successful development and evaluation of the `StEVE` tool provides a concrete proof of concept for the entire framework. This work delivers a methodology and a tool for the design and analysis of adaptive security strategies. This research provides a significant approach to managing cyber risk. The datasets, models, and artefacts supporting this study are open source and available in the online repository [↗](#).

6.1 Future Work

The framework presented in this thesis while providing a complete solution to the Stackelberg verification problem, also opens several promising avenues for future research. The limitations acknowledged in Chapter 5 directly inform these potential directions.

- **Enhancing Model:** The current framework relies on a restricted form of ADT for tractability. Future work could focus on extending the modelling pipeline to handle more general. This would allow for the representation of more complex in depth scenarios but would require the development of more sophisticated algorithms to manage the resulting increase in state space complexity. Furthermore, incorporating Bayesian game models could address uncertainty and incomplete information, providing a more realistic abstraction of scenarios where the defender may not have perfect knowledge of attacker types or payoffs.
- **Addressing Scalability:** The state space explosion remains a fundamental challenge in model checking. While `StEVE` was able to analyse a 10-round repeated game, applying the framework to scenarios with very long time horizons or a large number of concurrent actions will require advances in scalability. Future research could investigate the use of refinement techniques, symmetry reduction, or statistical model checking to analyse more complex RSSGs.
- **Exploring Alternative Game Concepts:** Our work has focused exclusively on the Stackelberg equilibrium. Future research could explore the verification of other equilibrium concepts relevant to security. For example, integrating different commitment structures or allowing for simultaneous moves in certain rounds could provide insights into a range of attacker defender interactions. Investigating the implications of different solution concepts for follower and leader assumption would also be a valuable extension.
- **User Studies:** To bridge the gap between academic research and practical security operations, future work should focus on the usability and real world application of the `StEVE`. This could involve conducting user studies with security professionals to refine the tool's interface and the presentation of its verification results.

Appendix A

Proofs of Correctness for the Procedures

A.1 Transforming an ADT to an SSG model: Correctness

Base Case: Attacker Action is ‘Send a Virus’

- Statement $P(n)$: The algorithm correctly transforms an ADT to an SSG model when the attacker has n actions.
- $P(1)$: The attacker has 1 action: ‘send a virus’.

Proof:

- Let $\psi_{\text{attacker}} = \{\text{‘send a virus’}\}$.
- According to Algorithm 1, $\text{Act}_{\text{attacker}} = \text{POWERSET}(\psi_{\text{attacker}})$.
- The algorithm should create nodes and assign actions accordingly, establishing terminal nodes and appending preference values.

Inductive Step: Attacker Sends a Virus and Password

- Inductive Hypothesis $P(k)$: Assume the statement $P(n)$ is true for some arbitrary number k , i.e., the algorithm correctly transforms an ADT to an SSG model when the attacker has k actions, including ‘send a virus’.

Proof:

- Assume $P(k)$ is true, where $\psi_{\text{attacker}} = \{\text{‘send a virus’}, a_2, a_3, \dots, a_k\}$ and the algorithm correctly generates an SSG model. Now, consider $P(k+1)$, where we add ‘send a password’ to the actions, i.e., $\psi_{\text{attacker}} = \{\text{‘send a virus’}, \text{‘send a password’}, a_2, a_3, \dots, a_k\}$.
- According to Algorithm 1, $\text{Act}_{\text{attacker}} = \text{POWERSET}(\psi_{\text{attacker}})$.
- The algorithm will identify this new action, add it to the powerset of actions, and create corresponding nodes and actions in the SSG model.

- The algorithm should then proceed to create nodes for the attacker, assign possible actions, map nodes and actions, and add preference values, just as it did in the base case and under the inductive hypothesis.
- If the algorithm correctly transforms the ADT to an SSG model with the new action included, then $P(k+1)$ is true, and by the principle of mathematical induction, $P(n)$ is true for all $n \geq 1$.

Defender Action is ‘Invest High Performance Firewalls’

- Statement $P(n)$: The algorithm correctly transforms an ADT to an SSG model when the defender has n actions.
- $P(1)$: The defender has 1 action: ‘invest high performance firewalls’.

Proof:

- Let $\psi_{\text{defender}} = \{\text{‘invest high performance firewalls’}\}$.
- According to Algorithm 1, $\text{Act}_{\text{defender}} = \text{POWERSET}(\psi_{\text{defender}})$.
- The algorithm should create nodes and assign actions accordingly, establishing terminal nodes and appending preference values.

Inductive Step: Defender Adds Action ‘Hire Cybersecurity Experts’

- Inductive Hypothesis $P(k)$: Assume the statement $P(n)$ is true for some arbitrary number k , i.e., the algorithm correctly transforms an ADT to an SSG model when the defender has k actions, including ‘invest high performance firewalls’.

Proof:

- Assume $P(k)$ is true, where $\psi_{\text{defender}} = \{\text{‘invest high performance firewalls’}, d_2, d_3, \dots, d_k\}$ and the algorithm correctly generates an SSG model.
- Now, consider $P(k+1)$, where we add ‘hire cybersecurity experts’ to the actions, i.e., $\psi_{\text{defender}} = \{\text{‘invest high performance firewalls’}, \text{‘hire cybersecurity experts’}, d_2, d_3, \dots, d_k\}$.
- According to Algorithm 1, $\text{Act}_{\text{defender}} = \text{POWERSET}(\psi_{\text{defender}})$.
- The algorithm will identify this new action, add it to the powerset of actions, and create corresponding nodes and actions in the SSG model.
- Specifically, for each new action d in $\text{Act}_{\text{defender}}$, the algorithm will:
 - Add a new node h_{new} to H_{defender} .
 - Assign $\text{Act}_{\text{defender}}$ to $\chi(h_{\text{new}})$.
 - Map the choice node and action to the corresponding node $\sigma(h_1, d) = h_{\text{new}}$.
 - For each action a in $\text{Act}_{\text{attacker}}$, create a new terminal node z_{new} and map $\sigma(h_{\text{new}}, a) = z_{\text{new}}$.
- If the algorithm correctly transforms the ADT to an SSG model with the new action included, then $P(k+1)$ is true, and by the principle of mathematical induction, $P(n)$ is true for all $n \geq 1$.

Base Case: Disjunctively Refined Nodes are ‘Send a Virus’ and ‘Password’

- Statement $P(n)$: The algorithm correctly transforms an ADT to an SSG model when the attacker has n disjunctively refined nodes.
- $P(1)$: The attacker has 2 disjunctively refined nodes: ‘send a virus’ and ‘password’.

Proof:

- Let $\psi_{\text{attacker}} = \{\text{‘send a virus’}, \text{‘password’}\}$.
- According to Algorithm 1, $\text{Act}_{\text{attacker}} = \text{POWERSET}(\psi_{\text{attacker}})$.
 - So, $\text{Act}_{\text{attacker}} = \{\emptyset, \{\text{‘send a virus’}\}, \{\text{‘password’}\}, \{\text{‘send a virus’}, \text{‘password’}\}\}$.

Inductive Step: Add ‘Social Engineering’ to the Disjunctively Refined Node

- Inductive Hypothesis $P(k)$: Assume the statement $P(n)$ is true for some arbitrary number k , i.e., the algorithm correctly transforms an ADT to an SSG model when the attacker has k disjunctively refined nodes, including ‘send a virus’ and ‘password’.

Proof:

- Assume $P(k)$ is true, where $\psi_{\text{attacker}} = \{\text{‘send a virus’}, \text{‘password’}, a_3, \dots, a_k\}$ and the algorithm correctly generates an SSG model.
- Now, consider $P(k+1)$, where we add ‘social engineering’ to the disjunctively refined nodes, i.e., $\psi_{\text{attacker}} = \{\text{‘send a virus’}, \text{‘password’}, \text{‘social engineering’}, a_3, \dots, a_k\}$.
- According to Algorithm 1, $\text{Act}_{\text{attacker}} = \text{POWERSET}(\psi_{\text{attacker}})$.
 - So, $\text{Act}_{\text{attacker}}$ will contain all possible combinations of the three actions, which are 8 sets (from $\emptyset$ to the set of all three actions).
- The algorithm will identify this new action, add it to the powerset of actions, and create corresponding nodes and actions in the SSG model.
- Specifically, for each new action a in $\text{Act}_{\text{attacker}}$, the algorithm will:
 - Add a new node h_{new} to H_{attacker} .
 - Assign $\text{Act}_{\text{attacker}}$ to $\chi(h_{\text{new}})$.
 - Map the choice node and action to the corresponding node $\sigma(h_1, a) = h_{\text{new}}$.
 - For each action b in $\text{Act}_{\text{defender}}$, create a new terminal node z_{new} and map $\sigma(h_{\text{new}}, b) = z_{\text{new}}$.
- If the algorithm correctly transforms the ADT to an SSG model with the new action included, then $P(k+1)$ is true, and by the principle of mathematical induction, $P(n)$ is true for all $n \geq 1$.

Base Case: Conjunctively Refined Nodes are ‘Send a Virus’ and ‘Password’

- Statement $P(n)$: The algorithm correctly transforms an ADT to an SSG model when the attacker has n conjunctively refined nodes.
- $P(1)$: The attacker has 2 conjunctively refined nodes: ‘send a virus’ and ‘password’.

Proof:

- Let $\psi_{\text{attacker}} = \{\text{‘send a virus’}, \text{‘password’}\}$.
- According to Algorithm 1, $\text{Act}_{\text{attacker}} = \text{POWERSET}(\psi_{\text{attacker}})$.
 - So, $\text{Act}_{\text{attacker}} = \{\emptyset, \{\text{‘send a virus’}\}, \{\text{‘password’}\}, \{\text{‘send a virus’}, \text{‘password’}\}\}$.

Inductive Step: Add ‘Social Engineering’ to the Conjunctively Refined Node

- Inductive Hypothesis $P(k)$: Assume the statement $P(n)$ is true for some arbitrary number k , i.e., the algorithm correctly transforms an ADT to an SSG model when the attacker has k conjunctively refined nodes, including ‘send a virus’ and ‘password’.

Proof:

- Assume $P(k)$ is true, where $\psi_{\text{attacker}} = \{\text{‘send a virus’}, \text{‘password’}, a_3, \dots, a_k\}$ and the algorithm correctly generates an SSG model.
- Now, consider $P(k+1)$, where we add ‘social engineering’ to the conjunctively refined nodes, i.e., $\psi_{\text{attacker}} = \{\text{‘send a virus’}, \text{‘password’}, \text{‘social engineering’}, a_3, \dots, a_k\}$.
- According to Algorithm 1, $\text{Act}_{\text{attacker}} = \text{POWERSET}(\psi_{\text{attacker}})$.
 - So, $\text{Act}_{\text{attacker}}$ will contain all possible combinations of the three actions, which are 8 sets (from $\emptyset$ to the set of all three actions).
- The algorithm will identify this new action, add it to the powerset of actions, and create corresponding nodes and actions in the SSG model.
- Specifically, for each new action a in $\text{Act}_{\text{attacker}}$, the algorithm will:
 - Add a new node h_{new} to H_{attacker} .
 - Assign $\text{Act}_{\text{attacker}}$ to $\chi(h_{\text{new}})$.
 - Map the choice node and action to the corresponding node $\sigma(h_1, a) = h_{\text{new}}$.
 - For each action b in $\text{Act}_{\text{defender}}$, create a new terminal node z_{new} and map $\sigma(h_{\text{new}}, b) = z_{\text{new}}$.
- If the algorithm correctly transforms the ADT to an SSG model with the new action included, then $P(k+1)$ is true, and by the principle of mathematical induction, $P(n)$ is true for all $n \geq 1$.

Base Case: Disjunctively Refined Nodes are ‘Invest High Performance Firewalls’ and ‘Hire Cybersecurity Experts’

- Statement $P(n)$: The algorithm correctly transforms an ADT to an SSG model when the defender has n disjunctively refined nodes.
- $P(1)$: The defender has 2 disjunctively refined nodes: ‘invest high performance firewalls’ and ‘hire cybersecurity experts’.

Proof:

- Let $\psi_{\text{defender}} = \{\text{‘invest high performance firewalls’}, \text{‘hire cybersecurity experts’}\}$.
- According to Algorithm 1, $\text{Act}_{\text{defender}} = \text{POWERSET}(\psi_{\text{defender}})$.
 - So, $\text{Act}_{\text{defender}} = \{\emptyset, \{\text{‘invest high performance firewalls’}\}, \{\text{‘hire cybersecurity experts’}\}, \{\text{‘invest high performance firewalls’}, \text{‘hire cybersecurity experts’}\}\}$.

Inductive Step: Add ‘Training’ to the Disjunctively Refined Node

- Inductive Hypothesis $P(k)$: Assume the statement $P(n)$ is true for some arbitrary number k , i.e., the algorithm correctly transforms an ADT to an SSG model when the defender has k disjunctively refined nodes, including ‘invest high performance firewalls’ and ‘hire cybersecurity experts’.

Proof:

- Assume $P(k)$ is true, where $\psi_{\text{defender}} = \{\text{‘invest high performance firewalls’}, \text{‘hire cybersecurity experts’}, d_3, \dots, d_k\}$ and the algorithm correctly generates an SSG model.
- Now, consider $P(k+1)$, where we add ‘training’ to the disjunctively refined nodes, i.e., $\psi_{\text{defender}} = \{\text{‘invest high performance firewalls’}, \text{‘hire cybersecurity experts’}, \text{‘training’}, d_3, \dots, d_k\}$.
- According to Algorithm 1, $\text{Act}_{\text{defender}} = \text{POWERSET}(\psi_{\text{defender}})$.
 - So, $\text{Act}_{\text{defender}}$ will contain all possible combinations of the three actions, which are 8 sets (from $\emptyset$ to the set of all three actions).
- The algorithm will identify this new action, add it to the powerset of actions, and create corresponding nodes and actions in the SSG model.
- Specifically, for each new action d in $\text{Act}_{\text{defender}}$, the algorithm will:
 - Add a new node h_{new} to H_{defender} .
 - Assign $\text{Act}_{\text{defender}}$ to $\chi(h_{\text{new}})$.
 - Map the choice node and action to the corresponding node $\sigma(h_1, d) = h_{\text{new}}$.
 - For each action a in $\text{Act}_{\text{attacker}}$, create a new terminal node z_{new} and map $\sigma(h_{\text{new}}, a) = z_{\text{new}}$.

- If the algorithm correctly transforms the ADT to an SSG model with the new action included, then $P(k+1)$ is true, and by the principle of mathematical induction, $P(n)$ is true for all $n \geq 1$.

Base Case: Conjunctively Refined Nodes are ‘Invest High Performance Firewalls’ and ‘Hire Cybersecurity Experts’

- Statement $P(n)$: The algorithm correctly transforms an ADT to an SSG model when the defender has n conjunctively refined nodes.
- $P(1)$: The defender has 2 conjunctively refined nodes: ‘invest high performance firewalls’ and ‘hire cybersecurity experts’.

Proof:

- Let $\psi_{\text{defender}} = \{\text{‘invest high performance firewalls’}, \text{‘hire cybersecurity experts’}\}$.
- According to Algorithm 1, $\text{Act}_{\text{defender}} = \text{POWERSET}(\psi_{\text{defender}})$.
 - So, $\text{Act}_{\text{defender}} = \{\emptyset, \{\text{‘invest high performance firewalls’}\}, \{\text{‘hire cybersecurity experts’}\}, \{\text{‘invest high performance firewalls’}, \text{‘hire cybersecurity experts’}\}\}$.

Inductive Step: Add ‘Training’ to the Conjunctively Refined Node

- Inductive Hypothesis $P(k)$: Assume the statement $P(n)$ is true for some arbitrary number k , i.e., the algorithm correctly transforms an ADT to an SSG model when the defender has k conjunctively refined nodes, including ‘invest high performance firewalls’ and ‘hire cybersecurity experts’.

Proof:

- Assume $P(k)$ is true, where $\psi_{\text{defender}} = \{\text{‘invest high performance firewalls’}, \text{‘hire cybersecurity experts’}, d_3, \dots, d_k\}$ and the algorithm correctly generates an SSG model.
- Now, consider $P(k+1)$, where we add ‘training’ to the conjunctively refined nodes, i.e., $\psi_{\text{defender}} = \{\text{‘invest high performance firewalls’}, \text{‘hire cybersecurity experts’}, \text{‘training’}, d_3, \dots, d_k\}$.
- According to Algorithm 1, $\text{Act}_{\text{defender}} = \text{POWERSET}(\psi_{\text{defender}})$.
 - So, $\text{Act}_{\text{defender}}$ will contain all possible combinations of the three actions, which are 8 sets (from $\emptyset$ to the set of all three actions).
- The algorithm will identify this new action, add it to the powerset of actions, and create corresponding nodes and actions in the SSG model.
- Specifically, for each new action d in $\text{Act}_{\text{defender}}$, the algorithm will:
 - Add a new node h_{new} to H_{defender} .
 - Assign $\text{Act}_{\text{defender}}$ to $\chi(h_{\text{new}})$.

- Map the choice node and action to the corresponding node $\sigma(h_1, d) = h_{\text{new}}$.
- For each action a in $\text{Act}_{\text{attacker}}$, create a new terminal node z_{new} and map $\sigma(h_{\text{new}}, a) = z_{\text{new}}$.
- If the algorithm correctly transforms the ADT to an SSG model with the new action included, then $P(k+1)$ is true, and by the principle of mathematical induction, $P(n)$ is true for all $n \geq 1$.

Base Case: Attack Node has a Child ‘Send a Virus’ and a Countermeasure ‘Invest High Performance Firewalls’

- Statement $P(n)$: The algorithm correctly transforms an ADT to an SSG model when the attacker has n nodes with a countermeasure.
- $P(1)$: The attacker has 1 node with a countermeasure: ‘send a virus’ (attack refinement) and ‘invest high performance firewalls’ (countermeasure).

Proof:

- Let $\psi_{\text{attacker}} = \{\text{‘send a virus’}\}$ and $\psi_{\text{defender}} = \{\text{‘invest high performance firewalls’}\}$.
- Accor. to Alg. 1, $\text{Act}_{\text{attacker}} = \text{POWERSET}(\psi_{\text{attacker}})$ and $\text{Act}_{\text{defender}} = \text{POWERSET}(\psi_{\text{defender}})$.
 - So, $\text{Act}_{\text{attacker}} = \{\emptyset, \{\text{‘send a virus’}\}\}$ and $\text{Act}_{\text{defender}} = \{\emptyset, \{\text{‘invest high perform. firewalls’}\}\}$.

Inductive Step: Add $\vee^{\text{attacker}}$ with ‘Send a Virus’ and ‘Send a Password’

- Inductive Hypothesis $P(k)$: Assume the statement $P(n)$ is true for some arbitrary number k , i.e., the algorithm correctly transforms an ADT to an SSG model when the attacker has k nodes with a countermeasure, including ‘send a virus’ and ‘invest high performance firewalls’.

Proof:

- Assume $P(k)$ is true, where $\psi_{\text{attacker}} = \{\text{‘send a virus’}, a_3, \dots, a_k\}$ and $\psi_{\text{defender}} = \{\text{‘invest high performance firewalls’}, d_3, \dots, d_k\}$ and the algorithm correctly generates an SSG model.
- Now, consider $P(k+1)$, where we add $\vee^{\text{attacker}}$ with ‘send a virus’ and ‘send a password’ to the attack nodes, i.e., $\psi_{\text{attacker}} = \{\text{‘send a virus’}, \text{‘send a password’}, a_3, \dots, a_k\}$.
- According to Algorithm 1,
 - $\text{Act}_{\text{attacker}} = \text{POWERSET}(\psi_{\text{attacker}})$ and $\text{Act}_{\text{defender}} = \text{POWERSET}(\psi_{\text{defender}})$.
 - So, $\text{Act}_{\text{attacker}}$ will contain all possible combinations of the two actions, which are 4 sets (from $\emptyset$ to the set of both actions).
- The algorithm will identify this new action, add it to the powerset of actions, and create corresponding nodes and actions in the SSG model.
- Specifically, for each new action a in $\text{Act}_{\text{attacker}}$, the algorithm will:
 - Add a new node h_{new} to H_{attacker} .

- Assign $\text{Act}_{\text{attacker}}$ to $\chi(h_{\text{new}})$.
- Map the choice node and action to the corresponding node $\sigma(h_1, a) = h_{\text{new}}$.
- For each action b in $\text{Act}_{\text{defender}}$, create a new terminal node z_{new} and map $\sigma(h_{\text{new}}, b) = z_{\text{new}}$.
- If the algorithm correctly transforms the ADT to an SSG model with the new action included, then $P(k+1)$ is true, and by the principle of mathematical induction, $P(n)$ is true for all $n \geq 1$.

Base Case: Attack Node has a Child ‘Send a Virus’ and a Countermeasure ‘Invest High Performance Firewalls’

- Statement $P(n)$: The algorithm correctly transforms an ADT to an SSG model when the attacker has n nodes with a countermeasure.
- $P(1)$: The attacker has 1 node with a countermeasure: ‘send a virus’ (attack refinement) and ‘invest high performance firewalls’ (countermeasure).

Proof:

- Let $\psi_{\text{attacker}} = \{\text{‘send a virus’}\}$ and $\psi_{\text{defender}} = \{\text{‘invest high performance firewalls’}\}$.
- According to Algorithm 1,
 $\text{Act}_{\text{attacker}} = \text{POWERSET}(\psi_{\text{attacker}})$ and
 $\text{Act}_{\text{defender}} = \text{POWERSET}(\psi_{\text{defender}})$.
- So, $\text{Act}_{\text{attacker}} = \{\emptyset, \{\text{‘send a virus’}\}\}$ and
 $\text{Act}_{\text{defender}} = \{\emptyset, \{\text{‘invest high performance firewalls’}\}\}$.

Inductive Step: Add $\wedge^{\text{attacker}}$ with ‘Send a Virus’ and ‘Send a Password’

- Inductive Hypothesis $P(k)$: Assume the statement $P(n)$ is true for some arbitrary number k , i.e., the algorithm correctly transforms an ADT to an SSG model when the attacker has k nodes with a countermeasure, including ‘send a virus’ and ‘invest high performance firewalls’.

Proof:

- Assume $P(k)$ is true, where $\psi_{\text{attacker}} = \{\text{‘send a virus’}, a_3, \dots, a_k\}$ and
 $\psi_{\text{defender}} = \{\text{‘invest high performance firewalls’}, d_3, \dots, d_k\}$ and the algorithm correctly generates an SSG model.
- Now, consider $P(k+1)$, where we add $\wedge^{\text{attacker}}$ with ‘send a virus’ and ‘send a password’ to the attack nodes, i.e., $\psi_{\text{attacker}} = \{\text{‘send a virus’}, \text{‘send a password’}, a_3, \dots, a_k\}$.
- According to Algorithm 1,
 $\text{Act}_{\text{attacker}} = \text{POWERSET}(\psi_{\text{attacker}})$ and $\text{Act}_{\text{defender}} = \text{POWERSET}(\psi_{\text{defender}})$.
- So, $\text{Act}_{\text{attacker}}$ will contain all possible combinations of the two actions, which are 4 sets (from $\emptyset$ to the set of both actions).

- The algorithm will identify this new action, add it to the powerset of actions, and create corresponding nodes and actions in the SSG model.
- Specifically, for each new action a in $\text{Act}_{\text{attacker}}$, the algorithm will:
 - Add a new node h_{new} to H_{attacker} .
 - Assign $\text{Act}_{\text{attacker}}$ to $\chi(h_{\text{new}})$.
 - Map the choice node and action to the corresponding node $\sigma(h_1, a) = h_{\text{new}}$.
 - For each action b in $\text{Act}_{\text{defender}}$, create a new terminal node z_{new} and map $\sigma(h_{\text{new}}, b) = z_{\text{new}}$.
- If the algorithm correctly transforms the ADT to an SSG model with the new action included, then $P(k+1)$ is true, and by the principle of mathematical induction, $P(n)$ is true for all $n \geq 1$.

Base Case: Defence Node has a Child ‘Invest High Perform. Firewalls’ and a Countermeasure ‘Social Engineering’

- Statement $P(n)$: The algorithm correctly transforms an ADT to an SSG model when the defender has n nodes with a countermeasure.
- $P(1)$: The defender has 1 node with a countermeasure: ‘invest high performance firewalls’ (defence refinement) and ‘social engineering’ (countermeasure).

Proof:

- Let $\psi_{\text{defender}} = \{\text{‘invest high performance firewalls’}\}$ and $\psi_{\text{attacker}} = \{\text{‘social engineering’}\}$.
- According to Algorithm 1,

$$\text{Act}_{\text{defender}} = \text{POWERSET}(\psi_{\text{defender}}) \text{ and } \text{Act}_{\text{attacker}} = \text{POWERSET}(\psi_{\text{attacker}}).$$
 - So, $\text{Act}_{\text{defender}} = \{\emptyset, \{\text{‘invest high performance firewalls’}\}\}$ and $\text{Act}_{\text{attacker}} = \{\emptyset, \{\text{‘social engineering’}\}\}.$

Inductive Step: Add $\vee^{\text{defender}}$ with ‘Invest High Performance Firewalls’ and ‘Training’

- Inductive Hypothesis $P(k)$: Assume the statement $P(n)$ is true for some arbitrary number k , i.e., the algorithm correctly transforms an ADT to an SSG model when the defender has k nodes with a countermeasure, including ‘invest high performance firewalls’ and ‘social engineering’.

Proof:

- Assume $P(k)$ is true, where $\psi_{\text{defender}} = \{\text{‘invest high performance firewalls’}, d_3, \dots, d_k\}$ and $\psi_{\text{attacker}} = \{\text{‘social engineering’}, a_3, \dots, a_k\}$ and the algorithm correctly generates an SSG model.
- Now, consider $P(k+1)$, where we add $\vee^{\text{defender}}$ with ‘invest high performance firewalls’ or ‘training’ to the defence nodes, i.e., $\psi_{\text{defender}} = \{\text{‘invest high performance firewalls’}, \text{‘training’}, d_3, \dots, d_k\}.$

- According to Algorithm 1,
 $\text{Act}_{\text{defender}} = \text{POWERSET}(\psi_{\text{defender}})$ and $\text{Act}_{\text{attacker}} = \text{POWERSET}(\psi_{\text{attacker}})$.
 - So, $\text{Act}_{\text{defender}}$ will contain all possible combinations of the two actions, which are 4 sets (from $\emptyset$ to the set of both actions).
- The algorithm will identify this new action, add it to the powerset of actions, and create corresponding nodes and actions in the SSG model.
- Specifically, for each new action d in $\text{Act}_{\text{defender}}$, the algorithm will:
 - Add a new node h_{new} to H_{defender} .
 - Assign $\text{Act}_{\text{defender}}$ to $\chi(h_{\text{new}})$.
 - Map the choice node and action to the corresponding node $\sigma(h_1, d) = h_{\text{new}}$.
 - For each action a in $\text{Act}_{\text{attacker}}$, create a new terminal node z_{new} and map $\sigma(h_{\text{new}}, a) = z_{\text{new}}$.
- If the algorithm correctly transforms the ADT to an SSG model with the new action included, then $P(k+1)$ is true, and by the principle of mathematical induction, $P(n)$ is true for all $n \geq 1$.

Base Case: Defence Node has a Child ‘Invest High Perform. Firewalls’ and a Countermeasure ‘Social Engineering’

- Statement $P(n)$: The algorithm correctly transforms an ADT to an SSG model when the defender has n nodes with a countermeasure.
- $P(1)$: The defender has 1 node with a countermeasure: ‘invest high performance firewalls’ (defence refinement) and ‘social engineering’ (countermeasure).

Proof:

- Let $\psi_{\text{defender}} = \{\text{‘invest high performance firewalls’}\}$ and $\psi_{\text{attacker}} = \{\text{‘social engineering’}\}$.
- According to Algorithm 1,
 $\text{Act}_{\text{defender}} = \text{POWERSET}(\psi_{\text{defender}})$ and $\text{Act}_{\text{attacker}} = \text{POWERSET}(\psi_{\text{attacker}})$.
 - So, $\text{Act}_{\text{defender}} = \{\emptyset, \{\text{‘invest high performance firewalls’}\}\}$ and
 $\text{Act}_{\text{attacker}} = \{\emptyset, \{\text{‘social engineering’}\}\}$.

Inductive Step: Add $\wedge^{\text{defender}}$ with ‘Invest High Performance Firewalls’ and ‘Training’

- Inductive Hypothesis $P(k)$: Assume the statement $P(n)$ is true for some arbitrary number k , i.e., the algorithm correctly transforms an ADT to an SSG model when the defender has k nodes with a countermeasure, including ‘invest high performance firewalls’ and ‘social engineering’.

Proof:

- Assume $P(k)$ is true, where $\psi_{\text{defender}} = \{\text{'invest high performance firewalls'}, d_3, \dots, d_k\}$ and $\psi_{\text{attacker}} = \{\text{'social engineering'}, a_3, \dots, a_k\}$ and the algorithm correctly generates an SSG model.
- Now, consider $P(k+1)$, where we add $\wedge^{\text{defender}}$ with 'invest high performance firewalls' and 'training' to the defence nodes, i.e., $\psi_{\text{defender}} = \{\text{'invest high performance firewalls'}, \text{'training'}, d_3, \dots, d_k\}$.
- According to Algorithm 1,
 $\text{Act}_{\text{defender}} = \text{POWERSET}(\psi_{\text{defender}})$ and $\text{Act}_{\text{attacker}} = \text{POWERSET}(\psi_{\text{attacker}})$.
 - So, $\text{Act}_{\text{defender}}$ will contain all possible combinations of the two actions, which are 4 sets (from $\emptyset$ to the set of both actions).
- The algorithm will identify this new action, add it to the powerset of actions, and create corresponding nodes and actions in the SSG model.
- Specifically, for each new action d in $\text{Act}_{\text{defender}}$, the algorithm will:
 - Add a new node h_{new} to H_{defender} .
 - Assign $\text{Act}_{\text{defender}}$ to $\chi(h_{\text{new}})$.
 - Map the choice node and action to the corresponding node $\sigma(h_1, d) = h_{\text{new}}$.
 - For each action a in $\text{Act}_{\text{attacker}}$, create a new terminal node z_{new} and map $\sigma(h_{\text{new}}, a) = z_{\text{new}}$.
- If the algorithm correctly transforms the ADT to an SSG model with the new action included, then $P(k+1)$ is true, and by the principle of mathematical induction, $P(n)$ is true for all $n \geq 1$.

Base Case: Defence Node with a Child 'Invest High Perform. Firewalls' and a Countermeasure 'Social Engineering'

- Statement $P(n)$: The algorithm correctly transforms an ADT to an SSG model when the defender has n nodes with a countermeasure, and assigns preference values using utility functions.
- $P(1)$: The defender has 1 node with a countermeasure: 'invest high performance firewalls' (defence refinement) and 'social engineering' (countermeasure).

Proof:

- Let $\psi_{\text{defender}} = \{\text{'invest high performance firewalls'}\}$ and $\psi_{\text{attacker}} = \{\text{'social engineering'}\}$.
- According to Algorithm 1,
 $\text{Act}_{\text{defender}} = \text{POWERSET}(\psi_{\text{defender}})$ and $\text{Act}_{\text{attacker}} = \text{POWERSET}(\psi_{\text{attacker}})$.
 - So, $\text{Act}_{\text{defender}} = \{\emptyset, \{\text{'invest high performance firewalls'}\}\}$ and $\text{Act}_{\text{attacker}} = \{\emptyset, \{\text{'social engineering'}\}\}$.

Inductive Step: Add $\wedge^{\text{defender}}$ with 'Invest High Performance Firewalls' and 'Training'

- Inductive Hypothesis P(k): Assume the statement P(n) is true for some arbitrary number k, i.e., the algorithm correctly transforms an ADT to an SSG model when the defender has k nodes with a countermeasure, and assigns preference values using utility functions.

Proof:

- Assume P(k) is true, where $\psi_{\text{defender}} = \{\text{'invest high performance firewalls'}, d_3, \dots, d_k\}$ and $\psi_{\text{attacker}} = \{\text{'social engineering'}, a_3, \dots, a_k\}$ and the algorithm correctly generates an SSG model and assigns preference values.
- Now, consider P(k+1), where we add $\wedge^{\text{defender}}$ with 'invest high performance firewalls' and 'training' to the defence nodes, i.e., $\psi_{\text{defender}} = \{\text{'invest high performance firewalls'}, \text{'training'}, d_3, \dots, d_k\}$.
- According to Algorithm 1,
 $\text{Act}_{\text{defender}} = \text{POWERSET}(\psi_{\text{defender}})$ and $\text{Act}_{\text{attacker}} = \text{POWERSET}(\psi_{\text{attacker}})$.
 - So, $\text{Act}_{\text{defender}}$ will contain all possible combinations of the two actions, which are 4 sets (from $\emptyset$ to the set of both actions).
- The algorithm will identify this new action, add it to the powerset of actions, create corresponding nodes and actions in the SSG model, and assign preference values using utility functions.
- Specifically, for each new action d in $\text{Act}_{\text{defender}}$, the algorithm will:
 - Add a new node h_{new} to H_{defender} .
 - Assign $\text{Act}_{\text{defender}}$ to $\chi(h_{\text{new}})$.
 - Map the choice node and action to the corresponding node $\sigma(h_1, d) = h_{\text{new}}$.
 - For each action a in $\text{Act}_{\text{attacker}}$, create a new terminal node z_{new} and map $\sigma(h_{\text{new}}, a) = z_{\text{new}}$.
 - Assign preference values using utility functions: $u_{\gamma}(\delta) = \text{Preferences}$, where $\gamma \in N$ and $\delta \in Z$.
- If the algorithm correctly transforms the ADT to an SSG model with the new action included and assigns preference values using utility functions, then P(k+1) is true, and by the principle of mathematical induction, P(n) is true for all $n \geq 1$.

A.2 Cost Estimation and Preferences

1. Defender: b'_1, b'_2 , Attacker: b_1, b_2

Defender's Cost: £22,000 + £5,500 = £27,500

Given:

Patch (b'_1): Impact Reduction: 70-90%

Patch (b'_2): Impact Reduction: 80-95%

Calculation Steps:

1. Start with 100% risk.
2. Apply Patch b'_1 (70-90% reduction):
Using the average, let's say 80% reduction.
New risk = 100% - 80% of 100% = 20%
3. Apply Patch b'_2 (80-95% reduction) to the remaining risk:
Using the average, let's say 87.5% reduction.
Final risk = 20% - 87.5% of 20% = 2.5%

Total Impact Reduction Calculation:

Original Risk: 100%

Final Risk: 2.5%

Total Impact Reduction: 100% - 2.5% = 97.5%

The impact score from CVSS will be reduced to $6 * (1 - 0.975) = 0.15$.

Defender's adjusted loss: $(0.15 / 10) * £100,000 = £1,500$

Attacker's gain: $(0.15 / 10) * £50,000 * 0.84 = £630$

 Defender's payoff: $-£27,500 - £1,500 = -£29,000$

 Attacker's payoff: £630

2. Defender: b'_1, b'_2 , Attacker: b_1

Defender's Cost: $£22,000 + £5,500 = £27,500$

Given:

Patch (b'_1): Impact Reduction: 70-90%

Patch (b'_2): Impact Reduction: 80-95%

Calculation Steps:

1. Start with 100% risk.
2. Apply Patch b'_1 (70-90% reduction):
Using the average, let's say 80% reduction.
New risk = 100% - 80% of 100% = 20%
3. Apply Patch b'_2 (80-95% reduction) to the remaining risk:
Using the average, let's say 87.5% reduction.
Final risk = 20% - 87.5% of 20% = 2.5%

Total Impact Reduction Calculation:

Original Risk: 100%

Final Risk: 2.5%

Total Impact Reduction: 100% - 2.5% = 97.5%

The impact score from CVSS will be reduced to $6 * (1 - 0.975) = 0.15$.

Defender's adjusted loss: $(0.15 / 10) * £100,000 = £1,500$

Attacker's gain: $(0.15 / 10) * £50,000 * 0.6 = £450$

 Defender's payoff: $-\text{£}27,500 - \text{£}1,500 = -\text{£}29,000$

 Attacker's payoff: $\text{£}450$

3. Defender: b'_1, b'_2 , Attacker: b_2

Defender's Cost: $\text{£}22,000 + \text{£}5,500 = \text{£}27,500$

Given:

Patch (b'_1): Impact Reduction: 70-90%

Patch (b'_2): Impact Reduction: 80-95%

Calculation Steps:

1. Start with 100% risk.
2. Apply Patch b'_1 (70-90% reduction):
Using the average, let's say 80% reduction.
New risk = $100\% - 80\% \text{ of } 100\% = 20\%$
3. Apply Patch b'_2 (80-95% reduction) to the remaining risk:
Using the average, let's say 87.5% reduction.
Final risk = $20\% - 87.5\% \text{ of } 20\% = 2.5\%$

Total Impact Reduction Calculation:

Original Risk: 100%

Final Risk: 2.5%

Total Impact Reduction: $100\% - 2.5\% = 97.5\%$

The impact score from CVSS will be reduced to $6 * (1 - 0.975) = 0.15$.

Defender's adjusted loss: $(0.15 / 10) * \text{£}100,000 = \text{£}1,500$

Attacker's gain: $(0.15 / 10) * \text{£}50,000 * 0.6 = \text{£}450$

 Defender's payoff: $-\text{£}27,500 - \text{£}1,500 = -\text{£}29,000$

 Attacker's payoff: $\text{£}450$

4. Defender: b'_1 , Attacker: b_1, b_2

Defender's Cost: $\text{£}22,000$

Given:

Patch (b'_1): Impact Reduction: 70-90%

Calculation Steps:

1. Start with 100% risk.
2. Apply Patch b'_1 (70-90% reduction):
Using the average, let's say 80% reduction.
Final risk = $100\% - 80\% \text{ of } 100\% = 20\%$

Total Impact Reduction Calculation:

Original Risk: 100%

Final Risk: 20%

Total Impact Reduction: $100\% - 20\% = 80\%$

The impact score from CVSS will be reduced to $6 * (1-0.8) = 1.2$.

Defender's adjusted loss: $(1.2 / 10) * £100,000 = £12,000$

Attacker's gain: $(1.2 / 10) * £50,000 * 0.84 = £5,040$

 Defender's payoff: $-£22,000 - £12,000 = -£34,000$

 Attacker's payoff: £5,040

5. Defender: b'_1 , Attacker: b_1

Defender's Cost: £22,000

Given:

Patch (b'_1): Impact Reduction: 70-90%

Calculation Steps:

1. Start with 100% risk.
2. Apply Patch b'_1 (70-90% reduction):
Using the average, let's say 80% reduction.
Final risk = $100\% - 80\% \text{ of } 100\% = 20\%$

Total Impact Reduction Calculation:

Original Risk: 100%

Final Risk: 20%

Total Impact Reduction: $100\% - 20\% = 80\%$

The impact score from CVSS will be reduced to $6 * (1-0.8) = 1.2$.

Defender's adjusted loss: $(1.2 / 10) * £100,000 = £12,000$

Attacker's gain: $(1.2 / 10) * £50,000 * 0.6 = £3,600$

 Defender's payoff: $-£22,000 - £12,000 = -£34,000$

 Attacker's payoff: £3,600

6. Defender: b'_1 , Attacker: b_2

Defender's Cost: £22,000

Given:

Patch (b'_1): Impact Reduction: 70-90%

Calculation Steps:

1. Start with 100% risk.
2. Apply Patch b'_1 (70-90% reduction):
Using the average, let's say 80% reduction.

$$\text{Final risk} = 100\% - 80\% \text{ of } 100\% = 20\%$$

Total Impact Reduction Calculation:

Original Risk: 100%

Final Risk: 20%

Total Impact Reduction: $100\% - 20\% = 80\%$

The impact score from CVSS will be reduced to $6 * (1-0.8) = 1.2$.

Defender's adjusted loss: $(1.2 / 10) * £100,000 = £12,000$

Attacker's gain: $(1.2 / 10) * £50,000 * 0.6 = £3,600$

 Defender's payoff: $-£22,000 - £12,000 = -£34,000$

 Attacker's payoff: £3,600

7. Defender: b'_2 , Attacker: b_1, b_2

Defender's Cost: £5,500

Given:

Patch (b'_2): Impact Reduction: 80-95%

Calculation Steps:

1. Start with 100% risk.
2. Apply Patch b'_2 (80-95% reduction):
Using the average, let's say 87.5% reduction.
Final risk = $100\% - 87.5\% \text{ of } 100\% = 12.5\%$

Total Impact Reduction Calculation:

Original Risk: 100%

Final Risk: 12.5%

Total Impact Reduction: $100\% - 12.5\% = 87.5\%$

The impact score from CVSS will be reduced to $6 * (1-0.875) = 0.75$.

Defender's adjusted loss: $(0.75 / 10) * £100,000 = £7,500$

Attacker's gain: $(0.75 / 10) * £50,000 * 0.84 = £3,150$

 Defender's payoff: $-£5,500 - £7,500 = -£13,000$

 Attacker's payoff: £3,150

8. Defender: b'_2 , Attacker: b_1

Defender's Cost: £5,500

Given:

Patch (b'_2): Impact Reduction: 80-95%

Calculation Steps:

1. Start with 100% risk.

2. Apply Patch b'_2 (80-95% reduction):

Using the average, let's say 87.5% reduction.

Final risk = 100% - 87.5% of 100% = 12.5%

Total Impact Reduction Calculation:

Original Risk: 100%

Final Risk: 12.5%

Total Impact Reduction: 100% - 12.5% = 87.5%

The impact score from CVSS will be reduced to $6 * (1 - 0.875) = 0.75$.

Defender's adjusted loss: $(0.75 / 10) * £100,000 = £7,500$

Attacker's gain: $(0.75 / 10) * £50,000 * 0.6 = £2,250$

 Defender's payoff: $-£5,500 - £7,500 = -£13,000$

 Attacker's payoff: £2,250

9. Defender: b'_2 , Attacker: b_2

Defender's Cost: £5,500

Given:

Patch (b'_2): Impact Reduction: 80-95%

Calculation Steps:

1. Start with 100% risk.

2. Apply Patch b'_2 (80-95% reduction):

Using the average, let's say 87.5% reduction.

Final risk = 100% - 87.5% of 100% = 12.5%

Total Impact Reduction Calculation:

Original Risk: 100%

Final Risk: 12.5%

Total Impact Reduction: 100% - 12.5% = 87.5%

The impact score from CVSS will be reduced to $6 * (1 - 0.875) = 0.75$.

Defender's adjusted loss: $(0.75 / 10) * £100,000 = £7,500$

Attacker's gain: $(0.75 / 10) * £50,000 * 0.6 = £2,250$

 Defender's payoff: $-£5,500 - £7,500 = -£13,000$

 Attacker's payoff: £2,250

10. Defender: {}, Attacker: b_1, b_2

Defender's Cost: £0

Total Impact Reduction Calculation:

Original Risk: 100%

Final Risk: 100%

Total Impact Reduction: 0%

The impact score from CVSS will be reduced to $6 * (1-0) = 6$.

Defender's adjusted loss: $(6 / 10) * £100,000 = £60,000$

Attacker's gain: $(6 / 10) * £50,000 * 0.84 = £25,200$

 Defender's payoff: $-£0 - £60,000 = -£60,000$

 Attacker's payoff: $£25,200$

11. Defender: {}, Attacker: b_1

Defender's Cost: £0

Total Impact Reduction Calculation:

Original Risk: 100%

Final Risk: 100%

Total Impact Reduction: 0%

The impact score from CVSS will be reduced to $6 * (1-0) = 6$.

Defender's adjusted loss: $(6 / 10) * £100,000 = £60,000$

Attacker's gain: $(6 / 10) * £50,000 * 0.6 = £18,000$

 Defender's payoff: $-£0 - £60,000 = -£60,000$

 Attacker's payoff: $£18,000$

12. Defender: {}, Attacker: b_2

Defender's Cost: £0

Total Impact Reduction Calculation:

Original Risk: 100%

Final Risk: 100%

Total Impact Reduction: 0%

The impact score from CVSS will be reduced to $6 * (1-0) = 6$.

Defender's adjusted loss: $(6 / 10) * £100,000 = £60,000$

Attacker's gain: $(6 / 10) * £50,000 * 0.6 = £18,000$

 Defender's payoff: $-£0 - £60,000 = -£60,000$

 Attacker's payoff: $£18,000$

Bibliography

- [Abate et al., 2021] Abate, A., Gutierrez, J., Hammond, L., Harrenstein, P., Kwiatkowska, M., Najib, M., Perelli, G., Steeples, T., and Wooldridge, M. J. (2021). Rational verification: Game-theoretic verification of multi-agent systems. *Appl. Intell.*, 51(9):6569–6584.
- [Alder, 2025] Alder, S. (2025). Patient death linked to ransomware attack on pathology services provider. Accessed: 2025-08-08.
- [Arias et al., 2020] Arias, J., Budde, C. E., Penczek, W., Petrucci, L., Sidoruk, T., and Stoelinga, M. (2020). Hackers vs. security: Attack defence trees as asynchronous multi-agent systems. In *Formal Methods and Software Engineering (ICFEM 2020)*, volume 12531 of *LNCS*, pages 3–19. Springer.
- [Aslanyan et al., 2016] Aslanyan, Z., Nielson, F., and Parker, D. (2016). Quantitative verification and synthesis of attack defence scenarios. In *CSF’16*, pages 105–119. IEEE Computer Society.
- [Baier and Katoen, 2008] Baier, C. and Katoen, J.-P. (2008). *Principles of Model Checking*. The MIT Press.
- [BBC News, 2023] BBC News (2023). Payroll provider zellis hit by moveit cyber attack. Accessed: 2025-06-21.
- [BBC News, 2025] BBC News (2025). M&s hit by ransomware attack affecting stores and online shopping. <https://www.bbc.co.uk/news/articles/ckgnndrgxv3o>. Accessed: 18 June 2025.
- [Beyer and Podelski, 2022] Beyer, D. and Podelski, A. (2022). *Software Model Checking: 20 Years and Beyond*, pages 554–582. Springer Nature Switzerland, Cham.
- [Buczak and Seeker, 2016] Buczak, A. L. and Seeker, E. (2016). A survey of data mining and machine learning methods for cyber security. *IEEE Communications Surveys & Tutorials*, 18(2):1153–1176.
- [Bulasara et al., 2025] Bulasara, P. K., Sahoo, S., Gupta, N., Han, Z., and Kumar, N. (2025). The internet of bio-nano things with insulin-glucose, security and research challenges: A survey. *ACM Comput. Surv.*, 57(5).

- [Chen et al., 2013] Chen, T., Forejt, V., Kwiatkowska, M. Z., Parker, D., and Simaitis, A. (2013). Automatic verification of competitive stochastic systems. *FMSD*, 43(1):61–92.
- [Cybersecurity and Infrastructure Security Agency (CISA), 2020] Cybersecurity and Infrastructure Security Agency (CISA) (2020). Top 10 routinely exploited vulnerabilities. Cybersecurity Advisory, Alert Code AA20-133A. Last revised: May 12, 2020 — Identifies CVE-2017-8759 among the top 10 most exploited vulnerabilities by foreign cyber actors between 2016–2019.
- [Cybersecurity and Infrastructure Security Agency (CISA), 2024] Cybersecurity and Infrastructure Security Agency (CISA) (2024). Palo Alto Networks Releases Guidance on PAN-OS Vulnerability CVE-2024-3400. Alert, Accessed: 22 August 2025.
- [Cybersecurity and Infrastructure Security Agency (CISA), 2025] Cybersecurity and Infrastructure Security Agency (CISA) (2025). Known Exploited Vulnerabilities Catalog. <https://www.cisa.gov/known-exploited-vulnerabilities-catalog>. Accessed July 31, 2025.
- [Doerr, 2018] Doerr, J. (2018). *Measure What Matters: How Google, Bono, and the Gates Foundation Rock the World with OKRs*. Portfolio/Penguin, New York.
- [Financial Times, 2025] Financial Times (2025). Ransomware costs at nhs provider synnovis far outstrip profits. Accessed: 2025-08-08.
- [Firesmith, 2003] Firesmith, D. (2003). Engineering security requirements. *Object Tech.*, 2(1):53–68.
- [Giannakopoulou et al., 2020] Giannakopoulou, D., Mavridou, A., Rhein, J., Pressburger, T., Schumann, J., and Shi, N. (2020). Formal requirements elicitation with FRET. In *Proceedings of the International Working Conference on Requirements Engineering: Foundation for Software Quality (REFSQ 2020)*. NASA ARC-E-DAA-TN77785.
- [Gordon and Loeb, 2002] Gordon, L. and Loeb, M. (2002). The economics of information security investment. *ACM Trans. Inf. Syst. Secur.*, 5(4):438–457.
- [Gutierrez et al., 2019a] Gutierrez, J., Najib, M., Perelli, G., and Wooldridge, M. (2019a). Equilibrium design for concurrent games. In *30th International Conference on Concurrency Theory (CONCUR 2019)*, volume 140 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 20:1–20:17. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.
- [Gutierrez et al., 2019b] Gutierrez, J., Najib, M., Perelli, G., and Wooldridge, M. (2019b). On computational tractability for rational verification. In *Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence (IJCAI 2019)*, pages 1160–1166. ijcai.org.

- [Gutierrez et al., 2020] Gutierrez, J., Najib, M., Perelli, G., and Wooldridge, M. (2020). Automated temporal equilibrium analysis: Verification and synthesis of multi-player games. *Artificial Intelligence*, 287:103353.
- [Gutierrez et al., 2018] Gutierrez, J., Najib, M., Perelli, G., and Wooldridge, M. J. (2018). EVE: A tool for temporal equilibrium analysis. In *Proc. of ATVA'18*, volume 11138 of *LNCS*, pages 551–557. Springer.
- [Hutchins et al., 2011] Hutchins, E. M., Cloppert, M. J., and Amin, R. M. (2011). Intelligence-driven computer network defense informed by analysis of adversary campaigns and intrusion kill chains. *Leading Issues in Information Warfare & Security Research*, 1(1):80–106.
- [International Organisation for Standardisation, 2022] International Organisation for Standardisation (2022). Iso/iec 27001:2022 information security, cybersecurity and privacy protection — information security management systems — requirements.
- [Jaewon Lee et al., 2014] Jaewon Lee, C., CGEIT, C., and CIA, C. (2014). An enhanced risk formula for software security vulnerabilities. *ISACA Journal*.
- [Kennedy et al., 2011] Kennedy, D., O’Gorman, J., Kearns, D., and Aharoni, M. (2011). *Metasploit: The Penetration Tester’s Guide*. No Starch Press.
- [Khakpour and Parker, 2024] Khakpour, N. and Parker, D. (2024). Partially observable security games for attack defence analysis in software systems. In *Software Engineering and Formal Methods: 22nd International Conference, SEFM 2024, Aveiro, Portugal, November 6-8, 2024, Proceedings*, page 144–161, Berlin, Heidelberg. Springer-Verlag.
- [Kim, 2018] Kim, P. (2018). *The Hacker Playbook 3: Practical Guide to Penetration Testing*. Secure Planet, LLC, kindle ebook edition. Kindle Edition, as referenced on Amazon UK.
- [Kordy et al., 2013] Kordy, B., Kordy, P., Mauw, S., and Schweitzer, P. (2013). ADTool: Security analysis with attack defence trees. In *Proc. of QEST 2013*, volume 8054 of *LNCS*, pages 173–176. Springer.
- [Kordy et al., 2010] Kordy, B., Mauw, S., Melissen, M., and Schweitzer, P. (2010). Attack defence trees and two-player binary zero-sum extensive form games are equivalent. In *Proc. of GameSec 2010*.
- [Kordy et al., 2014] Kordy, B., Mauw, S., Radomirovic, S., and Schweitzer, P. (2014). Attack Defence Trees. *J. Log. Comput.*, 24(1):55–87.
- [Krombholz et al., 2015] Krombholz, K., Hobel, H., Huber, M., and Weippl, E. (2015). Advanced social engineering attacks. *Journal of Information Security and Applications*, 22:113–122.

- [Kuhn, 1953] Kuhn, H. W. (1953). Extensive games and the problem of information. In Kuhn, H. W. and Tucker, A. W., editors, *Contributions to the Theory of Games*, vol. 2, number 28 in Annals of Mathematics Studies, pages 193–216. Princeton University Press.
- [Kulik et al., 2022] Kulik, T., Dongol, B., Larsen, P. G., Macedo, H. D., Schneider, S., Tran-Jørgensen, P. W., and Woodcock, J. (2022). A survey of practical formal methods for security. *Form. Asp. Comput.*, 34(1):1–39.
- [Kuppa et al., 2021] Kuppa, A., Aouad, L. M., and Le-Khac, N. (2021). Linking CVE’s to MITRE ATT&CK techniques. In *Proc. 16th Int. ARES Conf.*, pages 21:1–21:12. ACM.
- [Kwiatkowska et al., 2011] Kwiatkowska, M., Norman, G., and Parker, D. (2011). PRISM 4.0: Verification of probabilistic real-time systems. In *Proc. of CAV’11*, volume 6806 of *LNCS*, pages 585–591. Springer.
- [Kwiatkowska et al., 2022a] Kwiatkowska, M., Norman, G., and Parker, D. (2022a). Probabilistic model checking and autonomy. *Annu. Rev. Control. Robotics Auton. Syst.*, 5:385–410.
- [Kwiatkowska et al., 2019] Kwiatkowska, M., Norman, G., Parker, D., and Santos, G. (2019). Equilibria-based probabilistic model checking for concurrent stochastic games. In *Proc. 23rd International Symposium on Formal Methods*, volume 11800 of *LNCS*, pages 298–315. Springer.
- [Kwiatkowska et al., 2020] Kwiatkowska, M., Norman, G., Parker, D., and Santos, G. (2020). PRISM-games 3.0: Stochastic game verification with concurrency, equilibria and time. In *Proc. of CAV’2020*, volume 12225 of *LNCS*, pages 475–487. Springer.
- [Kwiatkowska et al., 2022b] Kwiatkowska, M., Norman, G., Parker, D., and Santos, G. (2022b). Correlated equilibria and fairness in concurrent stochastic games. In *Proc. 28th International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS’22)*, volume 13244 of *LNCS*, page 60–78. Springer.
- [Langner, 2011] Langner, R. (2011). Stuxnet: Dissecting a cyberwarfare weapon. *IEEE Security & Privacy*, 9(3):49–51.
- [Leyton-Brown and Shoham, 2008] Leyton-Brown, K. and Shoham, Y. (2008). *Essentials of game theory: A concise multidisciplinary introduction*. Morgan & Claypool.
- [Liedtke, 2024] Liedtke, T. (2024). *Functional Safety (Safety)*, pages 81–96. Springer Fachmedien Wiesbaden, Wiesbaden.
- [Lim, 2012] Lim, D. (2012). Hacking Cars to Keep Them Safe. *MIT Technology Review*.

- [Mozilla Foundation, 2025] Mozilla Foundation (2025). Mozilla security advisories. <https://www.mozilla.org/en-US/security/advisories/>. A continuously updated repository of Mozilla security bulletins and impact assessments.
- [Najib, 2020] Najib, M. (2020). *Rational verification in multi-agent systems*. PhD thesis, UK.
- [Nash, 1950] Nash, J. F. (1950). Equilibrium points in n-person games. *Proceedings of the National Academy of Sciences*, 36(1):48–49.
- [Nash, 1951] Nash, J. F. (1951). Non-cooperative games. *Annals of Mathematics*, 54(2):286–295.
- [National Cyber Security Centre, 2021] National Cyber Security Centre (2021). Bring your own device (byod). <https://www.ncsc.gov.uk/collection/device-security-guidance/bring-your-own-device>. Version 2.1, reviewed 13 May 2025.
- [National Cyber Security Centre, 2025a] National Cyber Security Centre (2025a). Moveit vulnerability. Accessed: 2025-08-08.
- [National Cyber Security Centre, 2025b] National Cyber Security Centre (2025b). National cyber security centre. <https://www.ncsc.gov.uk>. Accessed: 2025-08-19.
- [National Institute of Standards and Technology, 2018] National Institute of Standards and Technology (2018). Framework for improving critical infrastructure cybersecurity, version 1.1. Cybersecurity white paper, NIST.
- [National Institute of Standards and Technology (NIST), 2021] National Institute of Standards and Technology (NIST) (2021). Cve-2021-44228 detail — nvd. <https://nvd.nist.gov/vuln/detail/CVE-2021-44228>. NVD published date: 12 December 2021; last modified: 8 August 2025.
- [National Institute of Standards and Technology (NIST), 2024] National Institute of Standards and Technology (NIST) (2024). CVE-2024-3400 Detail. <https://nvd.nist.gov/vuln/detail/CVE-2024-3400>. Accessed: 22 August 2025.
- [Ng and Hasan, 2025] Ng, C. Y. and Hasan, M. K. B. (2025). Cybersecurity serious games development: A systematic review. *Comput. Secur.*, 150:104307.
- [NHS England, 2024] NHS England (2024). Synnovis cyber attack – statement from nhs england. Accessed: 2025-08-08.
- [NIST, 2017] NIST (2017). NVD - CVE-2017-0144. <https://nvd.nist.gov/vuln/detail/cve-2017-0144>.

- [NordPass, 2023] NordPass (2023). Top 200 most common passwords. <https://nordpass.com/most-common-passwords-list/>. Sixth annual list of the world’s most common passwords, based on analysis of a 2.5 TB database of compromised credentials.
- [NVD, 2024] NVD (2024). CVE-2024-4947. <https://nvd.nist.gov/vuln/detail/CVE-2024-4947>.
- [Osborne and Rubinstein, 1994] Osborne, M. J. and Rubinstein, A. (1994). *A course in game theory*. MIT Press.
- [OWASP Foundation, 2025] OWASP Foundation (2025). Owasp top ten. <https://owasp.org/www-project-top-ten/>. Project page. As of July 2025, the OWASP Top Ten:2025 release is anticipated in late summer or early fall 2025.
- [Palo Alto Networks, 2024] Palo Alto Networks (2024). PAN-OS: Command Injection Vulnerability in GlobalProtect (CVE-2024-3400). <https://security.paloaltonetworks.com/CVE-2024-3400>. Security advisory, Accessed: 22 August 2025.
- [PcWinTech.com, 2021] PcWinTech.com (2021). Default router & modem passwords a–l. <https://www.pcwintech.com/default-router-modem-passwords>. Site last updated December 27, 2021.
- [Phetmanee et al., 2025] Phetmanee, S., Sevegnani, M., and Andrei, O. (2025). Steve: A rational verification tool for stackelberg security games. In Kosmatov, N. and Kovács, L., editors, *Integrated Formal Methods*, pages 267–275, Cham. Springer Nature Switzerland.
- [Phetmanee et al., 2026] Phetmanee, S., Sevegnani, M., and Andrei, O. (2026). Rational verification in repeated security games. In Törngren, M., Gallina, B., Schoitsch, E., Troubitsyna, E., and Bitsch, F., editors, *Computer Safety, Reliability, and Security. SAFECOMP 2025 Workshops*, pages 312–326, Cham. Springer Nature Switzerland.
- [Pita et al., 2009] Pita, J., Jain, M., Ordóñez, F., Portway, C., Tambe, M., Western, C., Paruchuri, P., and Kraus, S. (2009). ARMOR security for los angeles international airport. In *Proceedings of the 22nd International Joint Conference on Artificial Intelligence (IJCAI-09)*, pages 125–132. International Joint Conferences on Artificial Intelligence Organization.
- [Pnueli, 1977] Pnueli, A. (1977). The temporal logic of programs. In *Proc. of the 18th Annual Symposium on Foundations of Computer Science*, pages 46–57. IEEE Computer Society.
- [Progress Software, 2023] Progress Software (2023). Moveit transfer critical vulnerability — 31 may 2023. Accessed: 2025-08-08.

- [Rapid7, 2025] Rapid7 (2025). Metasploit 2024 annual wrap-up. <https://www.rapid7.com/blog/post/2025/01/03/metasploit-2024-annual-wrap-up/>. Includes exploit module added for CVE-2024-3400, Accessed: 22 August 2025.
- [Rapid7, LLC, 2025] Rapid7, LLC (2003–2025). Metasploit framework. <https://www.metasploit.com/>. Open-source penetration testing framework, Accessed: 22 August 2025.
- [Schneier, 1999] Schneier, B. (1999). Attack trees. *Dr. Dobbs's Journal*, 24(12):21–29.
- [Sharma et al., 2023] Sharma, A., Gupta, B. B., Singh, A. K., and Saraswat, V. (2023). Advanced persistent threats (APT): evolution, anatomy, attribution and countermeasures. *Journal of Ambient Intelligence and Humanized Computing*, 14(7):9355–9381.
- [Shieh et al., 2012] Shieh, E., An, B., Yang, R., Tambe, M., Baldwin, C., DiRenzo, J., Maule, B., and Meyer, G. (2012). PROTECT: A deployed game theoretic system to protect the ports of the united states. In *Proceedings of the 11th International Conference on Autonomous Agents and Multiagent Systems (AAMAS 2012)*, pages 13–20. International Foundation for Autonomous Agents and Multiagent Systems.
- [Shostack, 2014] Shostack, A. (2014). *Threat modeling: Designing for security*. John Wiley & Sons.
- [Sinha et al., 2018] Sinha, A., Fang, F., An, B., Kiekintveld, C., and Tambe, M. (2018). Stackelberg security games: Looking beyond a decade of success. In *Proc. IJCAI18*.
- [Tambe, 2012] Tambe, M. (2012). *Security and game theory*. Cambridge University Press.
- [Tambe, 2025] Tambe, M. (2025). Teamcore biography — milind tambe. <https://teamcore.seas.harvard.edu/tambe/>. Accessed August 12, 2025.
- [Tenable, Inc., 2024] Tenable, Inc. (2024). Nessus Plugin ID 193255: Palo Alto Networks PAN-OS CVE-2024-3400. <https://www.tenable.com/plugins/nessus/193255>. Exploit availability: Metasploit, Accessed: 22 August 2025.
- [The MITRE Corporation, 2024] The MITRE Corporation (2024). MITRE ATT&CK Framework. <https://attack.mitre.org/>.
- [Tsai et al., 2009] Tsai, J., Rathi, S., Kiekintveld, C., Ordóñez, F., and Tambe, M. (2009). IRIS: A tool for strategic security allocation in transportation networks. In *Proceedings of the 8th International Conference on Autonomous Agents and Multiagent Systems (AAMAS 2009)*, pages 37–44. International Foundation for Autonomous Agents and Multiagent Systems.

- [Verizon Business, 2024] Verizon Business (2024). 2024 data breach investigations report. Technical report, Verizon, Basking Ridge, NJ.
- [Volexity, 2024] Volexity (2024). Zero-Day Exploitation of Unauthenticated Remote Code Execution Vulnerability in GlobalProtect (CVE-2024-3400). Accessed: 22 August 2025.
- [Von Neumann, 1928] Von Neumann, J. (1928). Zur theorie der gesellschaftsspiele. *Mathematische Annalen*, 100(1):295–320.
- [von Stackelberg, 2011] von Stackelberg, H. (2011). *Market structure and equilibrium*. Springer.
- [Widela et al., 2019] Widela, W., Audinot, M., Fila, B., and Pinchinat, S. (2019). Beyond 2014: Formal methods for attack tree–based security modeling. *ACM Comput. Surv.*, 52(4).
- [Wooldridge et al., 2016] Wooldridge, M., Gutierrez, J., Harrenstein, P., Marchioni, E., Perelli, G., and Toumi, A. (2016). Rational verification: From model checking to equilibrium checking. *Proc. of the AAAI Conf. on Artificial Intelligence*, 30(1).
- [Wuyts et al., 2014] Wuyts, K., Scandariato, R., and Joosen, W. (2014). Linddun privacy threat tree catalog. In *2014 IEEE Security and Privacy Workshops*, pages 182–191. IEEE.
- [Zheng et al., 2022] Zheng, W., Jung, T., and Lin, H. (2022). The stackelberg equilibrium for one-sided zero-sum partially observable stochastic games. *Automatica*, 140:110231.